Foundations and Automation for Monadic Program Verifiers

VLADIMIR GLADSHTEIN

(B.Sc., Saint Petersburg State University)

A THESIS SUBMITTED
FOR THE DEGREE OF
DOCTOR OF PHILOSOPHY

SCHOOL OF COMPUTING
NATIONAL UNIVERSITY OF SINGAPORE

2026

Thesis Advisor:
Ilya Sergey

Thesis Committee:
Olivier Danvy
Ranjit Jhala

Declaration

I hereby declare that this thesis is my original work and it has been written by me in its entirety. I have duly acknowledged all the sources of information which have been used in the thesis.

This thesis has also not been submitted for any degree in any university previously.

Vladimir Gladshtein
August 4, 2026

Abstract

With the rise of AI-assisted programming, software is now produced faster than it can be read: machine-generated code arrives at a scale no human review can keep up with, while the cost of finding exploitable bugs keeps dropping. Program verifiers, tools that mathematically prove programs correct with respect to their specifications, are the natural answer, but the demands of this new reality are steep. A verifier must be *foundational*, resting on a small trusted computing base; *general*, accommodating diverse languages, effects, and specification styles; and *scalable*, keeping pace with the rate at which code is written. Existing verification frameworks achieve at most two of these criteria at once.

This thesis defends the claim that all three can coexist, by building program verifiers as monadic shallow embeddings in the Lean proof assistant. We present Loom, a unified foundation for such verifiers, comprising three components. The first is a metatheory derived from first principles: a single type-class interface, `WPMonad`, that equips monadic computations with weakest-precondition transformers, together with non-axiomatic constructions for effects such as state, exceptions, partiality, non-determinism, and erasable ghost code. This interface is implemented in the Lean standard library and ships with the proof assistant by default. The second component is a collection of reasoning techniques that make the foundations general and reusable: compositional treatment of monad transformers, proofs of partial and total correctness, demonic and angelic readings of non-determinism, and a simplified construction of instances via ordered monad algebras. The third component is `vcgen`: a high-performance, extensible tactic for generating and discharging verification conditions, built on Lean's SymM framework for scalable symbolic execution and integrated with the `grind` proof automation.

To demonstrate the practicality of these foundations, we build Velvet: an auto-active verifier for effectful imperative programs in the style of Dafny. Velvet combines SMT-based automation, interactive Lean proofs, and property-based testing in a single multi-modal environment with access to Lean's mathematical libraries. On the VERINA benchmark suite, Velvet matches the effectiveness and performance of Dafny, while remaining, unlike Dafny, fully foundational.

Together, these results show that the big, complex parts of a program verifier can be built once, foundationally, and shared: a step towards a universal verification platform in which push-button automation and foundational guarantees coexist.

Acknowledgements

(To be written.)

Contents

List of Figures

1

Introduction

With the rise of large language models (LLMs), the world of software has changed dramatically in just a few years — and the change has the shape of an escalation. At first, AI was a companion that merely made us faster: in controlled experiments, developers assisted by AI completed programming tasks much quicker than those without [102]. Then it began gaining autonomy: by 2025, a quarter to a third of all new code at Google and Microsoft was machine-generated [92, 105]. Soon it crossed into performing complex engineering tasks entirely on its own: a team of AI agents recently produced a working 100,000-line C compiler — one that boots Linux and compiles SQLite and PostgreSQL — in two weeks, for under $20,000 [4]. And so we find ourselves at the last stage of the escalation: AI produces so much code that we can no longer follow it. As Andrej Karpathy — a founding member of OpenAI and former Director of AI at Tesla — put it:

> *I "Accept All" always, I don't read the diffs anymore.* [62]

Unsurprisingly, AI-generated code, much like human-written code, cannot be blindly trusted. Despite the recency of industry-scale adoption of autonomous AI code generation, news articles report AI agents ignoring security constraints [125], abusing API keys [123], and even exposing sensitive data [93].

The situation is exacerbated by the fact that the rising power of AI can be utilised not only to produce large amounts of code, but also to find exploits in large codebases. AI-powered systems such as Google's Big Sleep, the AI-augmented OSS-Fuzz, and Anthropic's Project Glasswing have already discovered tens of thousands of vulnerabilities in critical open-source software, including OpenSSL, wolfSSL, and SQLite [5, 14, 21, 134].

This therefore calls for the re-evaluation of our code quality standards. Over the past decades, software bugs were mostly perceived as a minor problem in everyday software. Code review and unit testing were considered sufficient to ensure that the code was "good enough" [58]. Putting more effort into building trust in codebases was not considered worthwhile because — to rephrase the famous quote from Tony Hoare [58] — the world just did not suffer significantly from the problem of software unreliability. He himself framed this as an empirical observation about the software of his day. Now, the situation is changing. First, the amount of code produced is so large that we cannot manually guarantee that it adheres to even minimal safety and security requirements. Code review does not scale any more. Instead, we need automated tools that can provide us with certainty about programs' behaviour without a human ever reading the source. Sec-

ond, as we saw above, AI-generated code is indeed buggy and the cost of discovering critical bugs in large systems has dropped significantly. Thus the distinction between "almost" correct and trivially broken programs is now blurring [118]. With this, blurs the value of software testing tools, because, as Dijkstra famously put it, "testing can be used to show the *presence* of bugs, but never to show their *absence*" [35].

Luckily, the tools designed to verify the absence of program bugs are not a new idea: they have already been studied quite extensively and are known as *program verifiers*.

1.1 Program Verifiers and Proof Assistants

A *program verifier* is a software system used to mathematically prove that a program adheres to its *specification*: a high-level set of requirements, stating what the program has to do, without stating how it does it. Unlike testing, which samples a program's behaviours, a verifier covers all of them at once: every input, every execution path, every corner case.

Historically, however, program verification has proven hard to put into practice. In a famously provocative essay, Millo, Lipton, and Perlis argued that proofs of programs are doomed: unlike mathematical theorems, whose proofs are read, re-derived, and absorbed by a community, program proofs are long, tedious, and read by no one — and a proof that lacks this social process, they claimed, is not a proof [83]. The follow-up debate [36, 122] identified the real obstacle: hand-written proofs about programs are too laborious to produce and too unreliable to trust. The modern solution to this problem is the *proof assistant*: a system in which proofs are not social artefacts but formal objects, checked mechanically by a small, independently auditable kernel. The social process is replaced by a machine — one that, unlike a human reviewer, never tires and never skips a step.

The new era of AI-generated software, described above, places three specific demands on this technology. First, *strong foundations with a small trusted computing base*: AI systems already can discover critical bugs which were undetected for decades [21, 30] in hours! There is no space for probability anymore: any loophole can be exploited. Second, *generality and flexibility*: AI generates large amounts of code across wildly different domains, so verification tools must be general enough to handle all of them. And finally, *scalability*: the tools must be scalable enough to keep up with the pace of code production.

A recent tendency in the formal methods community suggests where such tools should live: embedded in Lean [109] proof assistant:

- Lean has a small trusted kernel which checks the proofs [33]. Moreover, this kernel has multiple independent implementations [7, 20].
- Lean is based on dependent type theory [19], which makes it a highly expressive tool to embed computation models from the entirety of Computer Science.
- Lean is an efficient programming language [129] — with great support for metaprogramming [128], which makes it easy to build efficient automation on top of it.

Indeed, a wave of successful verifiers has been built on top of Lean in the past few

years: Veil, for distributed protocols [108]; Aeneas, for Rust programs [56]; and Cedar, Amazon's authorization-policy language, whose model and analyses are formalised in Lean [31].

Yet each of these verifiers is an immense standalone effort: each builds its own semantic foundations, its own proof infrastructure, and its own automation, from scratch. This raises the question this thesis answers: *can we do better — can the big, complex parts of a verifier be built once, foundationally, and shared?* To see what those parts are, let us consider how one builds a program verifier.

1.2 How to build a program verifier?

The first — and defining — challenge to solve when building a program verifier is embedding the source computational model into the proof assistant's logic. The base computational model of prevailing proof assistants [16, 60, 96, 126] is pure total functional programming. Whereas we are usually interested in building verifiers for programming languages with effects, such as state, partiality, exceptions, non-determinism, and IO. There are two main approaches here: deep and shallow embedding.

Deep embedding axiomatises the source language's syntax and operational semantics in the logic of a proof assistant. While this approach is flexible, as it allows one to manipulate the source language's programs as any other data type, it comes with a significant overhead of re-modelling what the host proof assistant already has — binding, recursion, type checking and even editor/LSP support.

Shallow embedding reuses the proof assistant's own programs — benefiting from language features implemented in it — to model the source language's programs, thus neglecting the deep-embedding overhead. To shoehorn the impure semantics of the source language into the total functional programming of the proof assistant, most of the notable shallow embedding approaches model effects of the source with monads [22, 85, 133].

This paradigm naturally accommodates verifiers, where the source is a high-level abstraction language, co-designed with the verifier, such as Veil. Veil models transitions between protocol states as non-deterministic programs modifying and asserting facts about the protocol state. This can naturally be expressed as a computations in a monad with non-determinism, state and exceptions. Reusing Lean's extensible `do`-notation syntax for monadic programs [130], Veil enjoys syntax highlighting and type checking for free.

Somewhat surprisingly, shallow embedding is also shown to be useful to reason about real-world programming languages, whose semantics is full of low-level details [22, 56, 110]. AutoCorres framework [50] — used to verify the seL4 microkernel — models C semantics shallowly in Isabelle by hiding the low-level memory manipulations behind monadic non-determinism [28].

In this thesis, we will focus on the shallow embedding approach. This clearly identifies

the common part in the program verifier's development process: building a framework for reasoning about monadic programs. Of course, there were attempts to build such shared foundation [64, 121, 131] but none of them keeps up with the demands of today's reality: some approaches, like F*, develop a practical system, scalable to industrial applications [13, 111, 139] but sacrifice foundational guarantees, while others, like dynamic logics and algebraic-effect reasoning [64, 89], while being foundational and general, lack a coherent story for a scalable automation.

1.3 Contributions and Outline

Foundational guarantees, generality, and scalability have long been at odds in program verification: existing frameworks pick at most two. The thesis defended in this work is that

> **program verifiers built as monadic shallow embeddings in the Lean proof assistant can deliver all three at once: Lean's kernel provides foundational guarantees with a minimal trusted base; the theory of weakest preconditions for monads distills the metatheory that all such verifiers share, making them general and reusable across effects and domains; and Lean's metaprogramming facilities make their automation scale to match industrial needs.**

To support this thesis, we develop Loom — a unified foundation for shallow embedding of program verifiers in the Lean proof assistant. Loom consists of three main components:

- **Strong foundations derived from first principles.** The core of Loom is based on a single monadic interface `WPMonad`, with provided instances for the Lean native monadic effects. Neither the interface nor the derived laws are axiomatic or extend Lean's type system. `WPMonad` is implemented in the Lean standard library and is shipped with the proof assistant by default.
- **Reasoning techniques establishing the generality and flexibility of the foundations.** Loom provides a set of reasoning techniques for monadic programs, including support for modularity, proving partial and total correctness, reasoning about non-deterministic programs, simplified construction of `WPMonad` instances and more. This is implemented in a separate Loom library, which has been published as a part of our Principles of Programming Languages (POPL) 2026 paper *Foundational Multi-Modal Program Verifiers* [46].[1]
- **Highly efficient automation for monadic program verification.** Loom provides `vcgen`: a highly efficient and extensible tactic for monadic program verification. This tactic is based on the `WPMonad` interface and is implemented in the Lean standard library as well.

On top of that, **we show case the practicality of the foundation by building Velvet**: an auto-active verifier in Lean, based on Loom. While, being a fully foundational verifier,

[1] The accompanying software artifact is archived on Zenodo at `https://doi.org/10.5281/zenodo.17347734`; the working repository is available at `https://github.com/verse-lab/loom`.

the performance of Velvet is comparable to state-of-the-art auto-active verifiers, such as Dafny. Velvet is published in our Computer Aided Verification (CAV) 2026 paper *Velvet: A Foundational Multi-Modal Verifier for Imperative Programs in Lean* [45].[2]

1.4 Other Work Done

Besides the contributions described in this thesis, the author has contributed to a number of other research results:

- Developed *Infinitary Logic*, a Separation Logic extension to reason about infinitary relational properties [47]
- Contributed to developing an agentic framework to automatically generate verified Velvet programs [37]
- Formalised a verification-condition generator for the intermediate verification language B3 [41].
- Contributed to Yolo: a technique for Separation Logic lazy proof automation [82].
- Contributed to developing a formal semantics of Veil — a framework for automated and interactive verification of transition systems, which has since become one of the first external adopters of Loom [107, 108].
- Developed a framework for reasoning about computations over sparse and structured data [43, 44].
- Developed LeanSSR: small-scale reflection proof methodology and tactic language for Lean [42].

The remainder of this thesis is structured as follows: Chapter 2 provides the background on monads, program verifiers and Lean proof assistant. Chapter 3 demonstrates the process of building a simple verifier for a toy imperative language with Loom. Chapter 4 describes the foundations and verification techniques for monadic programs. Chapter 5 explains how the automation for monadic program verification is constructed, showcasing Lean metaprogramming capabilities specific for scalable symbolic execution engines. Chapter 6 demonstrates all Loom ecosystem in action on Velvet. Chapter 7 concludes the thesis and outlines future directions.

[2]The accompanying software artifact is archived on Zenodo at `https://doi.org/10.5281/zenodo.19801401`.

2

Background

In this chapter we survey the literature surrounding this thesis. We begin in Sec. 2.1 by contrasting deep and shallow embeddings of programs in a proof assistant, identifying the *semantic gap* as the central difficulty of shallow embedding and monads as the standard way to bridge it. Sec. 2.2 then gives a background on monads and shows how a range of computational effects are encoded in the monadic interface in Lean. Equipped with this, Sec. 2.3 surveys the most notable verification projects built on shallow monadic embeddings. Finally, Sec. 2.4 takes a deep dive into frameworks for reasoning about monadic code, highlighting the main shortcoming of existing approaches that this thesis sets out to address.

2.1 How Deep Shall We Embed?

In the early 1990s, while embedding hardware description languages in HOL, Boulton et al. observed that there are two approaches to design a verifier in a proof assistant: deep and shallow embeddings. The former represents the abstract syntax of programs as data types inside the proof assistant, and then gives the a meaning via a semantic mapping to logical formulas. The latter defines semantic operators in the logic of the proof assistant and maps the source program syntax directly onto those operators. In particular, Boulton et al. used different embedding styles for different hardware languages: shallow embeddings for ELLA [87] and Silage [55], and a deep embedding for VHDL [59]. Comparing the two approaches, they identified an important disadvantage of deep embedding over the shallow one: deep embedding comes with the overhead of rebuilding the syntactic infrastructure for the source language inside the proof assistant:

> *Shallow embedding is simpler than deep for large languages since it avoids building infrastructure for the syntax of the language within the logic. [15]*

That said, since shallow embedding requires a mapping from source programs to the proof assistant's programs, and HOL only supports pure functional code, the embedding of ELLA was restricted to its functional fragment. In contrast, embedding VHDL deeply allowed Boulton et al. to define its semantics *operationally*, *i.e.* by defining a simulating interpreter with a notion of state, thus supporting the whole language.

This restriction is not an accident of HOL: it is the defining tension of shallow embedding. Because a shallow embedding maps source programs onto the host logic's own programs, it inherits the logic's semantics which is usually *pure* and *total*. Real lan-

```
def StateM (σ α : Type) := σ → α × σ

def StateM.pure a : StateM σ α := fun s => (a, s)

def StateM.bind x f : StateM σ β := fun s => let (a, s') := x s; f a s'

def get : StateM σ σ := fun s => (s, s)
def set s : StateM σ Unit := fun _ => ((), s)

def tick : StateM Nat Nat := do
  let n ← get
  set (n + 1)
  pure n
```

Figure 2.1: The state monad `StateM`.

guages, however, are effectful: they mutate state, raise exceptions, diverge, and behave non-deterministically. The standard way to bridge this semantic gap is to map the source not into bare pure functions but into a *monad* [84, 132], which recovers effectful behaviour inside a pure setting — letting a shallow embedding model state, exceptions, divergence, and non-determinism without re-modelling syntax. Before delving into a survey of notable shallow-embedding projects, we introduce the formal notion of a monad and give a few simple examples.

2.2 Crash Course in Monads

Initially a concept from category theory [67], *monads* were introduced to computer science courtesy of Moggi. Translating from the heavy category-theoretic language used in [84], a *monad* is a functor `m : Type → Type` [1] together with two operations: `pure : α → m α`, which turns a plain value into a computation that does nothing but return it, and `bind : m α → (α → m β) → m β` (written `>>=`), which runs one computation and feeds its result to the next. Intuitively, a value of type `m α` is a *computation* that yields a result of type `α` while possibly performing some *effect along the way*; `pure` is the trivial effect-free computation, and `bind` is sequential composition that threads the effect through. These two operations are required to satisfy three *monadic laws* — `pure` is a left and right unit for `bind`, and `bind` is associative — which together guarantee that sequencing behaves predictably, regardless of the underlying effect. We now consider a number of simple examples.

2.2.1 Simple Effects

The power of the abstraction is that the *same* interface describes a wide range of effects: the choice of `m` fixes what "effect along the way" means.

Example 2.1 (State)**.** For a fixed state type σ, the *state monad* `StateM σ` represents a computation as a function from an initial state to a result paired with a final state, `StateM σ α = σ → α × σ` (Fig. 2.1). `StateM.pure` returns its value and leaves the state untouched, while `StateM.bind` runs the first computation and threads the resulting state into the sec-

[1] In other words, `m` is a mapping `Type → Type` that preserves identities and function composition.

```
1  def ExceptM (ε α : Type) := ε ⊕ α
2
3  def ExceptM.pure a : ExceptM ε α := .inr a
4
5  def ExceptM.throw e : ExceptM ε α := .inl e
6
7  def ExceptM.bind x f : ExceptM ε β :=
8    match x with
9    | .inl e => .inl e
10   | .inr a => f a
11
12 def div x y : ExceptM String Nat :=
13   if y = 0 then throw "division by zero"
14   else pure (x / y)
```

Figure 2.2: The exception monad `ExceptM`.

```
1 def StateT (σ : Type) (m : Type → Type) (α : Type) := σ → m (α × σ)
2
3 def ExceptT (ε : Type) (m : Type → Type) (α : Type) := m (ε ⊕ α)
4
5 def div x y : ExceptT String (StateM Nat) Unit :=
6   if y = 0 then throw "division by zero"
7   else set (x / y)
```

Figure 2.3: The `StateT` and `ExceptT` transformers.

ond; the effect "along the way" is the implicit reading and writing of σ. Two primitive operations expose the state: `get` (line 7) reads the current state, and `set` (line 8) overwrites it. Using these primitives, one can implement the `tick` function (line 10), which increments the current counter and returns it; it is written in Lean's `do`-notation, which we detail in Sec. 2.2.3.

Example 2.2 (Exceptions)**.** For an error type ε, the *exception monad* `ExceptM ε` represents a computation as either a successful result or an error, `ExceptM ε α = ε ⊕ α` (Fig. 2.2). `ExceptM.pure` (line 3) injects a successful value, and a dedicated operation `ExceptM.throw` (line 5) aborts with an error; `ExceptT ε m` (line 7) short-circuits, propagating the first error and otherwise passing the result on. An example of a computation in `ExceptM` is the division operator `div` (line 12), which throws an exception if the divisor is zero.

Example 2.3 (Non-determinism)**.** Non-deterministic computations are captured by powersets: `NonDetM α = Set α` (Fig. 2.4), where a computation denotes the *set* of all results it may produce. `NonDetM.pure` is the singleton, `NonDetM.bind` takes the union of its continuation over every possible result, and the operation `pick` (line 7) takes a predicate `p` and non-deterministically yields every value satisfying it. Here the effect is the multiplicity of outcomes.

2.2.2 *Monad Transformers*

Each monad above captures a *single* effect, yet realistic programs combine several at once — a computation may read and write state *and* signal failure. The abstraction to compose effects from different monads is called a *monad transformer*. A transformer takes an under-

```
1 def NonDetM (α : Type) := Set α
2
3 def NonDetM.pure a : NonDetM α := {a}
4
5 def NonDetM.bind x f : NonDetM β := { b | ∃ a ∈ x, b ∈ f a }
6
7 def pick {α} (p : α → Prop) : NonDetM α := { x | p x }
```

Figure 2.4: Non-determinism via sets.

lying monad `m` and returns a richer monad that adds one more effect on top of it. The state transformer `StateT σ m` adds mutable state to `m`, and the exception transformer `ExceptT ε m` adds exceptions (Fig. 2.3). Stacking them yields a monad with *both* effects: for instance, `div` (Fig. 2.3, line 5) lives in `ExceptT String (StateM Nat)` — it `throws` on division by zero and otherwise writes the quotient into the underlying state with `set`.

2.2.3 *Monads Today*

Moggi's work was itself purely theoretical: he explained how to encode state, exceptions, non-determinism, and I/O in a monadic interface, and introduced the monadic laws as a mechanism to reason about monadic computations — in particular, to prove equivalence. These ideas were later picked up by Wadler and implemented in Haskell [104, 133]. Since then, expressing effectful computations with monads has been adopted in many functional programming languages — among them OCaml, through binding operators [124]; Scala, via `for`-comprehensions [127]; and F#, through computation expressions [103] — and Lean is no exception. The monadic interface in Lean is encoded via the `Monad` type class, which takes a functor `m : Type → Type` as a parameter and requires corresponding definitions of `pure` and `bind`. Once user instantiates the `Monad` class for their custom monad `m`, they can implement code in `m` using Lean's `do`-notation [130]. An example of Lean `do`-notation is shown in Fig. 2.1, line 10. Such code looks very much like imperative code: it consists of a sequence of commands, each of which is a monadic computation. If a monadic computation, such as `get`, returns a non-trivial result, that result can be captured with `let`. For instance, the `do` block `let n ← get; set (n + 1)` desugars into `bind get (fun n => set (n + 1))`.

Unlike other programming languages, such as Haskell, Lean's `do`-notation is not a primitive — it is defined via Lean's macro system. In particular, this makes it dynamically extensible with custom syntax.

2.3 Shallow Embeddings in the Wild

With the first implementations of monadic interfaces in pure functional programming languages, we took a major step towards bridging the limitation of shallow embedding that we discussed in Sec. 2.1 — we can now embed effectful computations in the *logic* of the proof assistant. The next step is to develop a general mechanism for reasoning about monadic computations. Moggi attempted this via equational reasoning based on the monad laws, and also sketched a monadic rendering of Hoare Triples for the state monad [85]; this was later generalised to Hoare Triples for arbitrary monads by Schröder and Mossakowski. That line of work continued, defining and proving sound monadic

Hoare logics and even extending the approach to dynamic logics [2] [89, 113, 114].

Unfortunately, while quite general, Schröder and Mossakowski's approach could not be used to reason about shallow embeddings of realistic programs at the time. The framework's generality outran the tooling of its day. Reasoning uniformly over an *arbitrary* monad requires higher-kinded abstraction — a variable $\texttt{m} : \texttt{Type} \to \texttt{Type}$ — which only very expressive assistants like Coq can state, yet those offered little automation beyond interactive tactics at the time; the more automated, simply-typed systems such as Isabelle/HOL cannot abstract over the monad constructor at all, forcing each monad to be instantiated concretely and forfeiting genericity.

It is no surprise, then, that the first verification efforts to succeed at scale abandoned monad-independence: each fixed a single, concrete monad and engineered a weakest-precondition reading with a verification-condition generator tailored to it. One of them was built on Isabelle/HOL.

2.3.1 *Verification of seL4 with AutoCorres*

The first demonstration that a realistic system could be verified via a shallow monadic embedding was the seL4 microkernel [65]. Its proof is a refinement chain of three layers, all written in Isabelle/HOL: an abstract specification, an executable *design* specification (automatically translated from a Haskell prototype of the kernel), and the C implementation, with each layer proved to refine the one above by forward simulation.

The first two layers are monadic shallow embeddings of a Haskell kernel prototype. The second, the executable specification, is quite low-level and models the Haskell prototype verbatim: it uses data types such as 32-bit machine words and models the program's heap memory explicitly as part of its state, operating on raw pointers. The reason this approach scaled is that it does not reason about the C code or executable specification directly, but about their abstracted counterpart (the first layer), which does the same thing at a higher level of abstraction. For instance, it uses nondeterminism to hide low-level details (such as pointer allocation) and abstract functions in place of explicit pointer representations. The precise monad they used was $\sigma \to \texttt{Set}\ (\alpha \times \sigma) \times \texttt{Bool}$, a nondeterministic monad tracking state σ together with a failure flag (the $\times$ Bool part). The Hoare Triple for such a monad simply says that every result reachable from a P-state satisfies Q:

$$\{P\}\ m\ \{Q\} \triangleq \forall s.\ P\,s \Rightarrow \forall (r, s') \in \mathsf{fst}\,(m\,s).\ Q\,r\,s'$$

Finally, they implemented a verification-condition generator (VCG) that discharges such triples by applying structural rules phrased in backward-reasoning form, leaving only the residual logical conditions to the prover. Because the abstract specification hides the low-level memory details, proof engineers discharging the residual VCs could focus on the "essence" of the obligation, without worrying about low-level details such as pointer arithmetic and buffer overflows.

To relate correctness proofs about the abstract specification to the actual C implementa-

[2] Dynamic logic extends Hoare logic with modal operators $[p]\varphi$ and $\langle p\rangle\varphi$ — "φ holds after every (respectively, some) terminating execution of program p" — so that one can express not only partial correctness but also termination and total correctness; a Hoare Triple $\{\varphi\}\ p\ \{\psi\}$ becomes the special case $\varphi \to [p]\psi$ [52].

tion, Cock et al. first developed a refinement between the abstract specification and the Haskell executable model. A few years later, Klein et al. proved a similar refinement from the executable model down to the deep embedding of the C implementation, completing the whole effort.

The original effort stops at the C level and assumes a correct compiler; later work closed that gap by translation validation down to the binary [115], and lifted the guarantees to high-level security properties such as integrity and information-flow confidentiality [91]. The manual C-to-monad embedding was itself subsequently automated by AutoCorres [50], which mechanically abstracts C into the same nondeterministic state monad — the tool we associate with this line of work.

2.3.2 *OCaml Verification with CFML*

A more lightweight alternative to the multi-layer refinement of seL4 is CFML [22]. CFML verifies OCaml programs by translating them *directly* into logical formulas — *characteristic formulas* — in Coq, with no intermediate executable model or deep embedding. In its weakest-precondition presentation [23], the characteristic formula of a program is a *predicate transformer over heap predicates*. Writing $\mathsf{hprop} \triangleq \mathsf{Heap} \to \mathsf{Prop}$ for heap assertions, the formula of a computation returning a value of type α has type

$$\mathsf{Formula}\,\alpha \;\triangleq\; (\alpha \to \mathsf{hprop}) \to \mathsf{hprop},$$

mapping a postcondition to the weakest precondition that guarantees it.

This type is exactly the *continuation monad* over hprop, and its monad operations are precisely the weakest-precondition rules,

$$\mathsf{pure}\,x \;\triangleq\; \lambda Q.\, Q\,x, \qquad \mathsf{bind}\,m\,f \;\triangleq\; \lambda Q.\, m\,(\lambda x.\, f\,x\,Q),$$

so $\mathsf{pure}\,x$ requires the postcondition to already hold of x, and bind threads the intermediate postcondition through a sequence. [3] A value of type Formula α is therefore, in effect, the *specification* of a heap-manipulating computation, and the primitive heap operations are just particular predicate transformers. Modelling the heap as a finite map, with $h[p \mapsto v]$ denoting h with p updated to v, a pointer update is

$$\mathsf{set}\,p\,v \;=\; \lambda Q.\, \lambda h.\, Q\,()\,h[p \mapsto v], \tag{2.1}$$

read as: to satisfy the postcondition Q after the write, it must hold of the heap in which p has been updated to v. [4]

As with AutoCorres, this lets CFML hide the low-level complexity of OCaml's memory behind nondeterminism — here in *demonic* form, as quantifiers in the predicate transformer rather than a powerset in a state monad. Most strikingly, allocation involves no address

[3] Charguéraud restricts to *local* transformers — those closed under the frame and consequence rules, hence monotone — which form the sub-monad corresponding to genuine weakest preconditions.

[4] CFML is in fact a *separation logic* [97]: heap predicates are built from the points-to assertion $p \mapsto v$ and the separating conjunction $*$, and the transformers are originally represented over these connectives in a small-footprint style. For brevity, we use the simpler whole-heap form here.

arithmetic at all: the fresh location is simply universally quantified,

$$\mathsf{ref}\, v \;=\; \lambda Q.\, \lambda h.\, \forall l.\, l \notin \mathsf{dom}\,(h) \implies Q\, l\, (h \cup \{l \mapsto v\}), \tag{2.2}$$

so the obligation becomes "Q must hold *for every* fresh location l the allocator might return," abstracting its choice away entirely.

Like seL4, CFML supports reasoning as a library of Coq tactics for interactive proofs about these characteristic formulas. Because CFML is a separation logic, the tactics automate *frame inference* — working out which part of the heap each operation touches — but do not provide push-button automation in general.

2.3.3 *Specifications in Types: Hoare Type Theory*

In both seL4 and CFML, the specification of a computation is a *separate* logical object — a Hoare Triple attached to a monadic program, or a predicate transformer — standing apart from the program's type. *Hoare Type Theory* (HTT) [94] takes a different route: it folds the specification into the *type* of the computation. HTT introduces a *Hoare type* $\{P\}\, x{:}A\, \{Q\}$, inhabited by effectful computations that, run from a heap satisfying P, return a value $x : A$ and a heap satisfying Q. This Hoare type is itself a monad — the *Hoare monad* — whose pure and bind compose the pre- and postconditions of their arguments, so a well-typed program carries its own separation-logic specification, checked by the type checker.

Two features set HTT apart. First, it is specialised to a *single* effect: it is developed for mutable heap state — in our terms, the state monad `StateM` — with P and Q ranging over heap predicates, and is not monad-generic. Second, because the type $\{P\}\, x{:}A\, \{Q\}$ *depends* on the specification predicates, the Hoare monad is a *dependently typed* construct: unlike the predicate-transformer approaches, which keep the specification as an ordinary predicate and live happily in simply-typed logics (seL4 and AutoCorres in Isabelle/HOL), HTT can only be expressed in a dependent type theory, and was realised in the Ynot library [95]. Reasoning in HTT is, like CFML, interactive: its Coq tactics automate frame inference using canonical structures [49], but stop short of push-button automation.

2.3.4 *F* and Project Everest*

The largest-scale evidence that shallow embedding scales to real, performance-critical software comes from F* [121], a dependently-typed programming language and proof assistant built for verification. F* combines refinement types, an effect system, and SMT-based automation: a program is checked by computing a weakest-precondition predicate transformer and discharging the resulting verification conditions with the Z3 solver. This mechanism is realised by *Dijkstra monads* [119], a practical reworking of Hoare Type Theory closely related to CFML. It introduces a new type DST $\alpha\, wp$ which, like HTT, indexes the computation by its specification *wp*, but does so in the form of a predicate transformer — *wp*'s type is exactly Formula α from Sec. 2.3.2. A value of type DST $\alpha\, wp$ is a heap-manipulating computation returning α such that, for any postcondition *post*, if it starts in a state satisfying *wp post* it ends in one satisfying *post*.

Swamy et al. argue that indexing by the predicate transformer, rather than by pre- and postconditions, improves on HTT in two practical respects. First, it yields *verification-condition generation by construction*: since `pure` and `bind` compose weakest preconditions, the type checker computes a program's VC directly, whereas Hoare Triple specifications "[do] not lead directly to a VC generation algorithm" and required bespoke machinery to extract one. Second, it is *friendlier to automation*: HTT's computed verification conditions proliferate existential quantifiers over intermediate states, which SMT solvers handle poorly, whereas composing transformers avoids them.

The original construction targets stateful computation; later works derive Dijkstra monads canonically from a wide class of effect monads [3, 78].

This combination has been pushed to industrial scale by *Project Everest* [13], whose goal is a verified, drop-in replacement for the HTTPS stack, with its performance-critical code written in Low*. Everest comprises several verified components: HACL*, a library of cryptographic primitives [139]; Vale, verified performance-critical assembly; EverCrypt, an agile provider that multiplexes the best HACL* or Vale implementation behind a single API [111]; EverParse, a generator of verified zero-copy parsers for binary formats; and miTLS, a verified implementation of TLS. The result is not a laboratory curiosity: verified routines from HACL* and EverCrypt are deployed in production systems including Firefox, the Linux kernel, mbedTLS, the WireGuard VPN, and the Tezos blockchain.

For all its practical success, however, F* is *not foundational*: its guarantees rest on a large trusted computing base rather than a small proof kernel. The SMT solver sits *inside* the typechecking loop — every verification condition is discharged by Z3, which must therefore be trusted — and the goals handed to it go through a heavy, intricate encoding into first-order logic [2]. Moreover, F's effect system is organised around a fixed lattice of *primitive* effects that are *built into* the language rather than derived: `Tot` for pure total code, `Div` for possibly-diverging computations, `ST` for state, `Exn` for exceptions, and `ML` for their combination, together with `Ghost` for computationally irrelevant (erased) code [121]. In short, F* buys its automation by enlarging what one must trust — a deliberate trade-off we return to in Sec. 2.4.

2.3.5 *Rust Verification with Aeneas*

Rust occupies a sweet spot for shallow embedding. Its ownership-and-borrowing discipline rules out shared mutable aliasing, so *safe* Rust programs (those avoiding `unsafe` and interior mutability) behave like pure functional programs — there is no heap to model. Aeneas [56] exploits exactly this: it gives a pure functional semantics to a large subset of Rust and translates safe Rust into a functional model in a proof assistant (with backends including Lean), turning mutable borrows into ordinary value passing via a device called *backward functions*. The only effect that survives is partiality — a computation may `panic` or fail to terminate — which Aeneas captures in a `Result` monad with three outcomes — success, failure, or divergence:

```
inductive Result (α : Type) where
  | ok   : α → Result α
  | fail : Error → Result α
  | div  : Result α
```

Failure and divergence are propagated through `bind`, so each translated function returns a value in `Result` while its core stays pure. A shallow monadic embedding is thus a particularly natural fit for Rust.

Unlike the interactive CFML and HTT, Aeneas supplies custom reasoning principles for its `Result` monad together with *bespoke push-button automation* — but this machinery is built from scratch and specific to that one monad.

2.3.6 *Verification of Distributed Protocols in Veil*

The utility of shallow embedding goes beyond just building program verifiers. Veil is a verifier for state transition systems, built in Lean [108]. It lets users conveniently describe a transition system by providing syntax for specifying the system's state, its transitions, and the desired properties. To verify that the claimed properties indeed hold, it provides a comprehensive suite of techniques, from explicit-state model checking to SMT-based verification of safety properties.

At the same time, Veil is completely shallow: all Veil declarations expand to Lean declarations, all Veil UI is Lean UI, all Veil code is Lean code [109]. The main application of Veil is verification of distributed protocols, which often can be expressed in terms of state transition systems. When specifying a protocol in Veil, to match the user's mental model, it lets one write transitions as possibly nondeterministic imperative programs that modify and make assertions about the protocol state. Although such programs look like simple imperative code, thanks to Lean's extensible `do`-notation they macro-expand into monadic computations in the `VeilM` monad, which is defined as

$$\texttt{VeilM}\ \alpha \quad \triangleq \quad \texttt{NonDetT (ReaderT}\ \rho\ \texttt{(StateT}\ \sigma\ \texttt{(ExceptT ExId DivM))) }\ \alpha$$

Where ρ is an immutable part of the protocol state (wrapped in to a *reader* monad transformer — read only state transformer), σ is a mutable part state of protocol state, `ExId` — exception identifier and `DivM` is a monad for diverge (which we discuss in Sec. 2.4). Like Aeneas, Veil provides custom reasoning principles and bespoke push-button automation for `VeilM`, again built specifically for that monad.

2.4 Reasoning about Monads, Deep Dive

At this point, we hope to convince the reader that formal methods are abound in shallow embedding verifiers. All of them are based on specific monads modeling object computations and, inevitably, require to develop techniques to specify corresponding monadic code as well as prove such specifications automatically. The question is: Is there a framework to do so without compromising on the demands we have stated in Sec. 1.3? Such a framework should be general, scale to large applications and provide foundational guarantees.

Generality As we have discussed in Sec. 2.3 many frameworks sacrify generality for the sake of automation. AutoCorres, CFML, Aeneas and Veil develop specification and automated reasoning principles tailored to specific monads. Each such frameworks focuses on its own monad, and redesigns everything from scratch — including even the defini-

tion of Hoare Triple.

Scalability Unifying foundations of reasoning about monadic programs in *some* way was almost never a problem. Moggi sketched the techniques to do that, shortly after he introduced monads to Computer Science as a concept. Later his ideas were polished [89, 113, 114] and even formalised in the Agda proof assistant [64]. The problem, however, those foundations are impractical. Practical verifications systems, should implement a VCG algorithm, and, as shown by Swamy et al. (see Sec. 2.3.4), weakest-precondition calculus formalism is a better fit for that in the scope of monadic verification.

Foundational guarantees Finally, F* team address both generality and scalability concerns, developing Dijkstra monads formalism — specification and reasoning techniques applicable to derive weakest-precondition calculus for virtually any reachable monad. This falls one step short from developing a framework we want — F* itself is not foundational — as we have seen in Sec. 2.3.4 it trusts SMT outputs, its own heavy first-order encodings and implementation of certain basic monadic effects.

In the rest of this section we will take a deep dive into the Dijkstra monads formalism. In particular, we argue why, in its original shape, it bakes quantification over postconditions into the very definition of a Hoare triple — an encoding overhead that no amount of user care can avoid.

2.4.1 *Specification Monads*

Following the discussion in Sec. 2.3.4, early F* works are powered by DST α *wp* type of heap-manipulating x : StateM Heap α computations respecting *wp* transformer of type $(\alpha \to \mathsf{Heap} \to \mathsf{Prop}) \to (\mathsf{Heap} \to \mathsf{Prop})$ as follows:

$$\forall (post : \alpha \to \mathsf{Heap} \to \mathsf{Prop})\ (h : \mathsf{Heap}),\ wp\ post\ h \implies post\ (\mathsf{fst}(x\ h))(\mathsf{snd}(x\ h)) \tag{2.3}$$

In general, the monad M of computations we are verifying is referred as *computation monad* and the monad of corresponding predicate transformers as *specification monad*. Specification monads are assumed to be equipped with a partial order relation $\leq$ capturing the specification refinement. In case of $(\alpha \to \mathsf{Heap} \to \mathsf{Prop}) \to (\mathsf{Heap} \to \mathsf{Prop})$, $\leq$ is defined pointwise:

$$wp_1 \leq wp_2 \triangleq \forall post\ h,\ wp_1\ post\ h \implies wp_2\ post\ h$$

Note that this order quantifies over postconditions — an innocuous-looking fact that will become significant at the end of this section. Clearly, different computation monads correspond to different specification monads with different ways to establishing (2.3) connection. In fact, as [78] shows such correspondence is not one-to-one: one computation monad might correspond to multiple specification ones and vice versa. In general, to connect a computation monad M to a specification monad W, one has to define an weakest precondition morphism

$$\mathbf{wp}_M : \forall \alpha,\ M\ \alpha \to W\ \alpha$$

```
example : wp tick =
  let n → wp get
  wp (set (n + 1))
  wp (pure n)

example : wp get = fun post s => post s s

example : wp (set s') = fun post s => post () s'
```

Figure 2.5: Computing **wp** for simple programs

such that, it respects `pure` and `bind` operations. Then, a corresponding Dijkstra type would consists of all computations $x : M\ \alpha$ such that $wp \leq \mathbf{wp}_M\ x$ — in other words, weakest precondition of x, refines its specification wp. Once, such correspondence has been established, one can reason about an underlying computation x, in terms of its specification wp. For StateM Heap α, we can define **wp** as

$$\mathbf{wp}_{\mathsf{StateM\ Heap}}\ x \triangleq \lambda\ post\ (h : \mathsf{Heap}).post\ (\mathsf{fst}(x\ h))(\mathsf{snd}(x\ h)) \tag{2.4}$$

Unfolding the $wp \leq \mathbf{wp}_M\ x$ refinement for StateM Heap, we will exactly get (2.3).

2.4.2 *Adding More Effects*

Ahman et al. have proposed an approach to derive for a certain class of computation monads M, so called, *canonical specification* monad [3]. First step, would be to fix the **wp** definition for the simplest *identity monad* IdM.[5] The specification monad in this case would just be a continuation monad $\mathsf{ContM}\ \alpha \triangleq (\alpha \to \mathsf{Prop}) \to \mathsf{Prop}$, and

$$\mathbf{wp}_{\mathsf{IdM}}(x : \mathsf{IdM}\ \alpha) \triangleq \lambda post.\ post\ x$$

Next, we observe that, a lot of standard monads can be represented as $T\ \mathsf{IdM}$ for some monad transformer T, say $\mathsf{StateM} = \mathsf{StateT\ IdM}$ and $\mathsf{exceptM} = \mathsf{ExceptT}$. With this, for monad M, represented as $T\ \mathsf{IdM}$, we can define:

$$W \triangleq T\ \mathsf{ContM} \qquad \mathbf{wp}_{T\ \mathsf{ContM}}(x : T\ \mathsf{IdM}) \triangleq \mathbf{wp}_{\mathsf{IdM}}\ \langle\$\rangle_T\ x,$$

where $\langle\$\rangle_T$: $(M\ \alpha \to N\ \alpha) \to (T\ M\ \alpha \to T\ N\ \alpha)$ is a functor mapping coming with a monad transformer interface. Such method usually gives us plausible definitions of W. Say, for StateM Heap, correspondent W would look like

$$\mathsf{StateT\ ContM} = \mathsf{Heap} \to (\alpha \times \mathsf{Heap} \to \mathsf{Prop}) \to \mathit{Prop}$$

Which is equivalent to the definition we have presented above.

Later, Maillard et al. generalised the method to capture other base computation monads rather than IdM [78].

2.4.3 *Specification Monads and Hoare Triples*

[5] Identity monad IdM is defined simply as $\mathsf{IdM}\ \alpha \triangleq \alpha$, where `pure` is an identity function and `bind` is an application

The clear advantage of Dijkstra monad formalism is that it provides a way to generate weakest-precondition for a given monadic computation out of the box. Imagine, we want to compute a weakest precondition for `tick` function from Fig. 2.1. Since we know that **wp** distributes over `bind`, the first step would be to nest it as deep as we can in the program, resolving into Fig. 2.5, lines 1-4. Now we are working inside specification monad, which, in general, provides nice formulas for primitive operations. For example, `bind` corresponds to nesting, and `pure` to simply computing a post condition on the returning result (see Sec. 2.3.2). **wp** for `set` and `get` is also quite simple. For `get`, it just computes post condition on a current state (line 6) and for `set`, on a state being set (line 8). Finally, substituting **wp** for primitive functions and composing them with **wp** for bind, we get the formula `fun post n => post n (n + 1)` for `tick`'s weakest precondition.

The problems with this approach show up when we want to prove high level specifications. Imagine we want to prove a Hoare Triple $\{\lambda n,\ n = n_0\}$ tick $\{\lambda res\ n',\ n' - n_0 = 1\}$. That is, the difference between state values before and after running `tick` is 1. For this particular example we can easily define Hoare Triple $\{pre\}x\{post\}$ as

$$\{pre\}x\{post\} \triangleq \forall s,\ pre\ s \Longrightarrow \mathbf{wp}\ x\ post\ s \tag{2.5}$$

This works, we know the exact shape of our specification monad $W = (\alpha \to \mathsf{Nat} \to \mathsf{Prop}) \to (\mathsf{Nat} \to \mathsf{Prop})$. However, in general, if we follow Sec. 2.4.2 approach for deriving specification monad for given computational monad $M = T\ \mathsf{IdM}$, we would only know that W looks like $T\ \mathsf{ContM}$ for an arbitrary monad transformer T and is equipped with some partial order $\leq$. This does not give us enough structure to generalise definition (2.5) and provide a way to define Hoare Triples for arbitrary computational monads. The approach, taken in [3, 78] is to develop a technique deriving spec *pre post* operation on specification monad W which encodes the predicate transformer based on its pre- and postconditions, and then define

$$\{pre\}x\{post\} \triangleq \mathsf{spec}\ pre\ post\ \leq \mathbf{wp}\ x \tag{2.6}$$

Let's see, what does it unfolds to in case of StateM Nat. spec would be

$$\mathsf{spec}\ pre\ post = \lambda post'\ s,\ pre\ s\ \wedge\ (\forall res\ s',\ post\ res\ s' \Longrightarrow post'\ res\ s')$$

And hence 2.6 would be

$$\forall\ post'\ s,\ pre\ s\ \wedge\ (\forall res\ s',\ post\ res\ s' \Longrightarrow post'\ res\ s') \Longrightarrow \mathbf{wp}\ x\ post'\ s \tag{2.7}$$

The problem with the formula above is its leading quantifier: it ranges over all postconditions $post'$, which are *predicates* of type $\alpha \to \sigma \to \mathsf{Prop}$. That is, definition (2.6) introduces quantification over predicates — *second-order* quantification — into every Hoare triple, regardless of what the user wrote in *pre* and *post*. To see why this is problematic, recall from Sec. 2.3.4 that F* — the flagship implementation of the Dijkstra-monad formalism — relies on an SMT solver as its automation backend: every verification condition it generates must ultimately be expressed in SMT-LIB, a first-order language. On its own, the quantification over postconditions is unpleasant but survivable. Since the quantifier sits at the very top of the proof goal, $post'$ can be *lifted into the context*: replaced by a fresh

uninterpreted predicate symbol, which SMT-LIB readily accepts. As long as the state σ is first-order, the resulting formula is first-order again. This lifting breaks down, however, as soon as σ itself contains higher-order data, such as functions or relations.[6] The lifted *post*′ would then be a predicate *over functions* — a symbol that cannot even be declared in SMT-LIB, whose function symbols only accept first-order sorts as arguments; nor can the quantifier be kept in place, since SMT-LIB does not quantify over predicates. For such states, there is virtually no way to embed the obligation into SMT-LIB shallowly. This reveals the real problem with the framework: definition (2.6) inseparably couples the program state with quantified predicates over it. Even if the user makes an effort to keep *pre* and *post* in a well-behaved first-order fragment, a single higher-order component in the state renders the generated verification conditions higher-order, with no shallow way out.

This is precisely the situation F* finds itself in. Since its VCs cannot be shallowly mapped into SMT-LIB (*e.g.* by mapping F* function types onto SMT-LIB function types), F* develops a "deep embedding" of its own calculus into SMT-LIB. In particular, with this encoding, function types from F* are translated into special axiomatised uninterpreted types. This F*-to-SMT-LIB encoding is ad-hoc and must be trusted, hence significantly extending TCB and enlarging the attack surface.

2.5 Summary

In this chapter, we have extensively studied the story of shallow embeddings, all the way from Moggi's seminal work introducing the concept of a monad to Computer Science. We have concluded the chapter with a detailed study of F* — one of the most prominent shallow embedding frameworks. Our study has shown that none of the existing approaches keeps up with the demands stated in Sec. 1.3.

In later chapters of this thesis we will: propose a formalism which introduces no encoding overhead — its Hoare triples never quantify over postconditions, so first-order specifications yield first-order proof obligations — while remaining convenient for building efficient VC generators (Chapter 4); show how the absence of this overhead makes it possible to extend the automation with backends such as SMT without enlarging the trusted code base (Chapter 5); and show how to define and work with advanced computational effects fully foundationally (Chapter 4).

[6] For example, Veil computations operate with protocol states which very often contain functions or relations describing the correspondence between other state components.

3

Building a Simple Verifier

We start by building the intuition for Loom design and outlining the experience of using it. To do so, we will go through stages of developing a verifier for Cashmere a toy imperative WHILE-style language for implementing simple monetary operations, embedded into Lean.

3.1 Embedding Stateful Computations into Lean

Programs in Cashmere manipulate a simple state that contains a single mutable component representing the balance on a user's account (for demonstration purposes, let us forgo realism and assume the language only supports one user). We begin by implementing a function that withdraws a desired amount from a current balance. Its code is shown in Fig. 3.1, and it is defined in a Lean monad `StateM`, which represents a stateful computation, first reading the current value of the user account's balance (line 4) and then updating it with the new amount (line 5).

Lean's meta-programming facilities make it easy to disguise the code in Fig. 3.1, to make it look closer to familiar imperative code by adding syntax (`balance := ...`) for updating mutable state components, a `returns` clause to specify the return value, as well as `require`/`ensures` statements to specify its execution contract in terms of pre- and post-conditions. The code in Fig. 3.2 macro-expands to the original code in Fig. 3.1, which can be executed in Lean natively using the `StateM.run` function. Based on the user-provided `ensures` annotation, the definition in Fig. 3.2 also generates the corresponding correctness theorem for `withdraw`, which we will discuss next.

3.2 Specifying Stateful Computations with Loom

The correctness theorem generated by Loom for `withdraw` from its specification looks as follows:

$$\forall\ \mathsf{amount}\ b_{\mathrm{old}},\ \{\lambda b.\ b = b_{\mathrm{old}}\}\ \mathsf{withdraw}\ \mathsf{amount}\ \{\lambda res\ b.\ b + \mathsf{amount} = b_{\mathrm{old}}\} \tag{3.1}$$

The Hoare triple (3.1) is quantified over the input amount and the pre-defined *ghost* logical variable b_{old} capturing the initial balance. Its precondition is a predicate on the program state b, stating that its initial state is b_{old}. The postcondition is a predicate on the program result *res* (of type `Unit`) and the final state, also denoted as b. It states that the final balance state plus the input amount is equal to the initial balance (for now, we

```
abbrev Balance := Int

def withdraw (amount : Nat) : StateM Balance Unit := do
  balance ← get
  set (balance - amount)
```

Figure 3.1: Changing balance of the account

```
bdef withdraw (amount : Nat) returns Unit
ensures balance + amount = balanceOld do
  balance := balance - amount
```

Figure 3.2: Changing balance with macros

allow the balance to be negative). The program with `require`/`ensures` clauses is macro-expanded to a term `triple P c Q`, where `triple` is Loom's definition that can be used for computations `c` in any monad M that is equipped with an instance of `WPMonad` type class (explained in Sec. 4.1). To instantiate this type class, one has to provide: (1) A *complete lattice Pred*, to serve as the assertion language for computations in M, used to state pre- and postconditions on normally terminated (without throwing an exception) program paths. One can think of it as a type of propositions over the respective state. (2) A complete lattice *EPred*, to serve as a collection of assertion languages, used to state post-conditions on program paths terminating with exceptions. If considered monad does not have non-local control flow effects, such collection would be just empty, and denoted as `EPost⟨⟩` in the code. `EPost⟨⟩` is equivalent to a unit type. We will how does this collection changes when if monad can throw an exception in Sec. 3.5 (3) A *weakest-precondition transformer* wpTrans : $M\ \alpha \to$ PredTrans *Pred EPred* α, where the latter is just a wrapper over $(\alpha \to \mathit{Pred}) \to \mathit{EPred} \to \mathit{Pred}$. Specifically, wpTrans, computes a weakest-precondition for a given computation in M. It takes a postcondition on normally terminated paths and paths terminated with exception, and returns a precondition, which might hold to satisfy the postconditions after the program terminates. (4) A proof that wpTrans is monotone and respects `pure` and `bind` in M[1]

Changing the shape of M, *Pred*, or *EPred* would require a different instance of `WPMonad`. Fortunately, Loom minimises this overhead by providing a methodology for assembling computations modularly, as a combination of monads and monad transformers such as `StateT`, `ReaderT`, and `ExceptT`. If the semantics of a computation can be expressed as such a composition [76], Loom derives the corresponding `WPMonad` instance *automatically*.

Coming back to our example, the assertion language *Pred* for `StateM Balance` computations is simply the type of predicates on the balance, `Balance → Prop`; and, since the state monad has no exceptions, its exceptional assertion language is trivial (`EPost⟨⟩`). It follows that wpTrans has type `StateM Balance α → PredTrans (Balance → Prop) EPost⟨⟩`. Its implementation is shown in Fig. 3.3: given a computation `x`, a postcondition `post`, and an initial state `balance`, it runs `x balance` to obtain a result `ret` and a final state `balance'`, and returns `DivM` — the weakest precondition for a fixed postcondition is just that postcondition applied to the computation's result and output state.

[1]Since PredTrans *Pred EPred* is also a monad — its `pure` and `bind` can be defined similarly to Formula ones from Sec. 2.3.2 — "preserves `pure` and `bind`" means here that, it mapping M's `pure` and `bind` into PredTrans's ones.

```
instance : WPMonad (StateM Balance) (Balance → Prop) EPost⟨⟩ where
  wpTrans x := λ post balance =>
    let (ret, balance') := x balance
    post ret balance'
  ... -- Proofs about wpTrans
```

Figure 3.3: Embedding stateful computations

```
bdef withdraw (amounts : List Nat) returns Unit
ensures balance + amounts.sum = balanceOld do
  let mut tmp := amounts
  while tmp.nonEmpty
  invariant balance + amounts.sum = balanceOld + tmp.sum do
    let amount := tmp.head
    balance := balance - amount
    tmp := tmp.tail
```

Figure 3.4: Session-based withdrawal loop

Having provided wpTrans for `StateM Balance`'s primitives, Loom computes the weakest precondition [35] of a whole computation by composing wpTrans over its structure, so the statement (3.1) becomes

$$\forall\ \mathsf{amount}\ b_{\mathrm{old}}\ b,\ b = b_{\mathrm{old}} \implies \mathsf{wp}\ (\mathsf{withdraw\ amount})\ (\lambda res\ b'.\ b' + \mathsf{amount} = b_{\mathrm{old}})\ \langle\rangle\ b. \tag{3.2}$$

Where wp is just wpTrans which directly unfolds PredTrans *Pred EPred* into a predicate transformer type. For this example, wp (withdraw amount) *post* () returns $\lambda\ b.\ post\ ()\ (b - \mathsf{amount})$, meaning that the postcondition *post* holds on a final state b exactly when it holds of the result () : Unit and the state $b - \mathsf{amount}$. Note that, we still have to account for post-conditions on paths terminated with exceptions, but since such type is an empty collection of types, we provide an empty sequence of such postconditions $\langle\rangle$: EPost$\langle\rangle$.

After substituting this into (3.2), the resulting verification condition (VC) can be discharged either by translation to SMT-LIB via one of the existing Lean–SMT toolkits, or via Lean's own tactics. To improve both automation and the human-assisted proving experience, for more complicated VCs Loom provides a tactic that splits them into separate goals, one per `ensures` or `invariant` annotation. For this example, the proof of the VC is obtained by running Lean's standard `grind` tactic, delivering an independently checkable certificate of correctness as a Lean proof term.

3.3 Adding Non-Terminating Loops

Next, let us extend Cashmere with loops. This makes it possible to implement a procedure for handling multiple withdrawals in a single session by passing a list of withdrawal amounts to `withdraw` (*cf.* Fig. 3.5), decrementing the balance by the value of each list element in a loop.

An observant reader might notice that we do not supply an explicit termination measure for the loop — the mechanism of potentially non-terminating computations is enabled by Lean's machinery of partial fixpoints, which we touch upon in more detail in Chapter 4. To support reasoning about such partial loops, Loom implements proof principles for

```
bdef withdraw (inAmounts : Queue Nat) returns Unit
ensures balance + amounts.sum = balanceOld do
  let mut amounts := inAmounts
  while amounts.nonEmpty
  invariant balance + amounts.sum = balanceOld + inAmounts.sum do
    let amount := amounts.dequeue
    balance := balance - amount
    amounts := amounts.tail
```

Figure 3.5: Balance mutation in the loop

```
instance : WPMonad (StateT Balance DivM) (Balance → Prop) EPost⟨⟩ where
  wpTrans x := λ post balance => ⟨
    match x balance with
    | some (x, balance') => post x balance'
    | none => True⟩
  ...
```

Figure 3.6: Semantic embedding of divergence

partial correctness, wherein any postcondition holds for a loop that does not terminate. By virtue of the shallow embedding into Lean, Loom supports the execution of divergent computations natively (*i.e.*, not by extraction and linking with external code, as is done in Coq). In addition, Loom lets one reason about non-terminating computations both intrinsically (via loop invariants) and extrinsically (via explicit theorems), disentangling proofs of partial and total correctness (*cf.* Sec. 3.4).

A popular way to encode the semantics of non-terminating computations is via coinductive definitions, such as interaction trees [136], which, at the time of writing, are missing from Lean. Instead of coinduction, to encode divergence we change the underlying monad of our examples to `StateT Balance DivM`, where `StateT` is a state monad transformer [76] and `DivM` is an alias for `Option`; the semantic value of a divergent computation is `none`.

Although the assertion language *Pred* for `StateT Balance DivM` stays the same, we must change the definition of `wpTrans` and re-prove its properties, as shown in Fig. 3.6: now `x balance` can return `none`, in which case `wpTrans` simply returns `True`. The choice of value for the `none` case is dictated by the notion of partial correctness; in Chapter 4 we explain this choice, present an alternative, and show how to avoid the monotonicity proofs altogether.

The property we want to verify is that the initial balance equals the final one plus the sum of the withdrawn amounts. To support automated VC generation for the new `withdraw`, we add an `invariant` annotation to the loop; Loom then generates the VCs from the `ensures` and `invariant` annotations and dispatches them automatically with the help of `grind`.

3.4 Proving Total Correctness

The obvious problem with partial-correctness semantics is that any postcondition trivially holds for divergent computations. For instance, dropping the tail update from the loop body in Fig. 3.5 would make `withdraw` run forever, yet its VC would still be dis-

```
1 bdef withdraw (amounts : List Nat) returns Unit
2 ensures balance + amounts.sum = balanceOld
3 signals err => amounts.sum > balanceOld do
4   let mut tmp := amounts
5   while tmp.nonEmpty
6   invariant balance + amounts.sum = balanceOld + tmp.sum
7   decreasing tmp.length do
8     let amount := tmp.head
9     if amount > balance then
10      throw "Insufficient funds"
11    else balance := balance - amount
12    tmp := tmp.tail
```

Figure 3.7: Withdrawal with exceptions

charged successfully.

Loom lets one control the semantics of divergent computations by changing the `none` case in Fig. 3.6 from `True` to `False`: the weakest precondition of a divergent computation then becomes `False`, so no postcondition can be proved for it. To enable an intrinsic proof of `withdraw`, one then adds a decreasing measure after the `invariant` clause, such as `decreasing amounts.length`.

To accommodate both partial- and total-correctness proofs, we define `withdraw` in the `Option` monad, where it *might* diverge. A correctness proof of `withdraw` in the total semantics then also ensures that it terminates, so Loom derives for it the following correctness statement:

$$\forall\ \mathsf{amounts}\ b_{\mathrm{old}},\ \{\lambda b.\ b = b_{\mathrm{old}}\}\ \mathsf{withdraw}\ \mathsf{amounts}\ \{\lambda res\ b.\ b + \mathsf{amounts.sum} = b_{\mathrm{old}}\}\,.$$

It implies that, for any `amounts`, the outcome of `withdraw` is always `some` (res, b), so that we can extract its result and state for the postcondition.

We show in Chapter 6 that, to obtain a total-correctness proof for a program, one can first prove its partial correctness with respect to a desired property and then, separately, prove that `True` holds as a postcondition in the total semantics; Loom automatically combines these two proofs into a proof of total correctness with respect to the desired property.

3.5 Reasoning about Programs with Exceptions

Until this point, the type of Cashmere state was `Int`, and the programs did not enforce a clearly desirable property that the value of the balance always remains positive. To eliminate such scenarios in our running examples, we will change the implementation of `withdraw` to throw an exception if one is attempting to withdraw a larger value than the one of the current balance. The amended code is shown in Fig. 3.7 (lines 9-11).

To support exceptions in the executable semantics, we will have to adjust our computational monad once again, making it into `ExceptT String (StateT Balance DivM) Balance` where `ExceptT` is the exception monad transformer and `String` is the type of exceptions. Luckily, due to the similarity in the working of the state and exception monad transformers, this time, we do not need to fully redefine the semantic embedding. Instead,

```
wpTrans (x : ExceptT α BankM α) :
    PredTrans (Balance → Prop) EPost⟨String → Balance → Prop⟩ := ⟨
  fun post epost => wp (m := BankM) x.run
    (fun eret s' => match eret with
      | .ok ret => post ret s'
      | .error err => epost err s') ()⟩
```

Figure 3.8: Semantic embedding of exceptions

```
bdef withdraw returns (history : List Nat)
require balance ≥ 0
ensures balance + history.sum = balanceOld do
  let amounts :| amounts.sum ≤ balance
  let mut tmp := amounts
  while tmp.nonEmpty
  invariant balance + amounts.sum = balanceOld + tmp.sum
  invariant balance ≥ tmp.sum
  decreasing tmp.length do
    let amount := tmp.head
    if amount > balance then throw "Insufficient funds"
    else balance := balance - amount
    tmp := tmp.tail
  return amounts
```

Figure 3.9: Withdrawal with non-determinism

we can define the new version of `WPMonad` instance by reusing the old one. The first and defining difference is a change in the *EPred* type of the new instance. Since our programs can now throw exceptions, we want to reason about the states in which such exceptions arise. This changes our *EPred* type to `EPost⟨String → Balance → Prop⟩`, which is essentially a wrapper over `String → Balance → Prop` — a type capturing predicates over the thrown exception and the resulting state. Fig. 3.8 shows a definition of `wpTrans` for `ExceptT String (StateT Balance DivM)`, where `StateT Balance DivM` is abbreviated as `BankM`.

This definition immediately delegates to the inner `wpTrans` for `BankM`, which expects an element of type `BankM (Balance → Prop)`. To construct it, we first cast `x` to `(λ _ => False)` by unfolding the definition of `ExceptT` with `ExceptT.run`, and then collapse the `Except` into a single assertion: on a normal result we keep the supplied postcondition, while on an exception `err` we fall back to the *exceptional* postcondition drawn from *EPred*, applied to `err`. Taking that exceptional postcondition to be `List Nat` recovers the limiting case in which no exception may be thrown at all. In general, Loom can automatically derive a `WPMonad` instance for any monad enhanced with exceptions from an instance for the inner monad together with the instance for `ExceptT String`.

To let a program instead *specify* its exceptional behaviour, Cashmere adds a new `signals` clause (line 3 of Fig. 3.7), dual to `ensures`: whereas `ensures` states a postcondition that must hold on normally terminating paths, `signals err => P` states that the predicate `P` must hold of the resulting state whenever the program terminates by throwing the exception `err`. A `signals` clause is thus an element of the exceptional assertion language (in this case it does not directly depend on the exact exception value), and Loom feeds it to `wpTrans` as the exceptional postcondition; the earlier, exception-free reading is recovered by leaving `signals` unspecified (equivalently, taking it to be `False`).

Crucially, adding exceptions does not force us to revisit `withdraw`'s original contract: the normal-path `ensures` specification proved in Sec. 3.2 is reused verbatim, and only the new exceptional behaviour is described through `signals`. This is a first instance of Loom's "build the verifier as you go" methodology — extending a computation with a new effect reuses the existing semantic embedding and specifications rather than redoing them.

3.6 Modelling Non-Determinism in Program Semantics

As another effect, Loom supports non-deterministic computations, which are useful, for instance, for modelling interactions with input/output. Continuing with our example, instead of passing the list of amounts to deduct from the balance to `withdraw` as an argument, we can model it as a non-deterministic choice provided interactively by the user. Line 4 of Fig. 3.9 shows how this is implemented in Cashmere, using the Dafny-style *let-such-that* operator `:|` (also known as Hilbert's choice operator). The semantics of such a choice in our example can be summarised as "picking an arbitrary sequence of amounts to withdraw such that their sum does not exceed the current balance." The non-deterministically picked `amounts` also serves as the result of the whole procedure, which we name `history`, as it records the sequence of withdrawals performed. Note that, since now we constrain amounts to be below the current balance, exception will not be thrown and we can omit the `signals` clause (which as the same as setting it to just False).

Loom derives the VC to prove the new version of `withdraw` correct by assuming *demonic* semantics for the non-deterministic choice: the correctness theorem ensures that the post-condition holds for *all* values of `amounts` satisfying the constraint on line 4. As with exceptions, this new effect is accommodated by extending the underlying monad, while the semantic embedding and reasoning principles carry over unchanged. Furthermore, given a generator for values of type `List Nat`, Loom provides a *sound* execution of `withdraw`, repeatedly generating `amounts` candidates until it finds one whose sum does not exceed the balance; Sec. 3.7 describes this feature in detail.

3.7 Symbolic Execution with Angelic Non-Determinism

In addition to modelling demonic choices for the sake of safety reasoning, Loom also supports *angelic* non-determinism, which enables symbolic execution of programs. As a last variation on our example, imagine that the `withdraw` implementation from Fig. 3.9 imposes no constraint on the choice on line 4. In that case, it would be useful to *prove* — in the sense of program logics for under-approximate reasoning [99] — that the "Insufficient funds" exception can in fact be thrown.

To do so, we instruct Loom to use angelic non-determinism and to read the `signals` clause as the exceptional outcome we wish to *witness* rather than rule out. Under this semantics, we take the normal postcondition to be `False` and the `signals` clause to be `err => err = "Insufficient funds"`. Intuitively, such a goal can succeed only if an execution throws the "Insufficient funds" exception: a normal termination can never satisfy `False`, whereas an exceptional path succeeds exactly when it raises that exception.

In the interest of space, we omit the code of this example here; it can be found in our Lean development, together with the Lean proof of `withdraw`'s incorrectness.

3.8 Putting It All Together

We have now walked through a series of examples, building a Lean-embedded verifier for a small effectful language using Loom together with a little Lean meta-programming. Importantly, none of this machinery is bespoke: the `WPMonad` type class and its instances are part of the Lean standard library and facilities to model and reason about partiality and nondeterminism are shipped with Loom library, so building such a verifier requires no external framework.

Loom ships with an extensible library of monads and monad transformers for modelling a variety of computational effects, including `StateM`/`StateT`, `ReaderT`, `ExceptT`, `EStateM`, `NonDetT`, `DivM`, `Option`/`OptionT`, `StateRefT`, the probability monad `PMF`, and `Gen` — Lean's monad for property-based random sampling in the style of QuickCheck.

Several of these effects expose parameters that fix their execution semantics and the shape of the generated VCs. For `ExceptT` with exceptions of type ε, the exceptional post-condition — the `signals` clause of Sec. 3.5 — determines how exceptions are treated: taking it to be `False` forbids them, taking it to be `True` treats them as successes (the angelic, witness-style reasoning of Sec. 3.7), and a general predicate $\varepsilon \to$ `Prop` specifies their intended behaviour. For `DivM`, one controls the symbolic treatment of divergence — partial or total: in the partial case no decreasing measure is required, while in the total case correctness proofs additionally guarantee termination. Finally, for `NonDetT` one chooses between demonic and angelic non-determinism: the former accounts for all possible choices and is used to prove safety, whereas the latter underlies symbolic model checking and is used to prove the presence of a bug or to reason about reachability.

The remainder of this thesis develops the foundations behind this experience. Chapter 4 introduces `WPMonad` in full, together with the techniques to reason about these effects, and Chapter 5 shows how to make VC generation efficient.

4

Monadic Program Verification

This chapter presents the main contribution of the thesis: a general yet practical interface for monadic program verification and a collection of techniques built on top of it.

Sec. 4.1 develops the foundations of our monadic verification framework, Sec. 4.2 extends them to monad transformers, and Sec. 4.3 explains how we encode them in Lean; this implementation has been adopted by the Lean standard library. The remaining sections describe the techniques assembled in the Loom library on top of these foundations: Sec. 4.4 provides a simplified interface to our foundations for a common class of monads; Sec. 4.5 shows how to reason about total and partial correctness foundationally; Sec. 4.6 formalises a non-determinism effect; and Sec. 4.7 shows how to define and reason about erasable ghost code.

4.1 Weakest Precondition for Monads

In this section, we develop the foundations. First, we derive the most general form that still gives us enough structure to overcome the central shortcoming of existing formalisms identified in Sec. 2.4 — Hoare triples that quantify over postconditions; we then argue why we need this form in full generality.

4.1.1 The Shape of Specifications

Building on Sec. 2.4, we can postulate two main requirements for our foundations. First, they should be based on a weakest-precondition calculus, to allow for efficient VC generation. Second, the Hoare-triple definition must not introduce quantification over postconditions: as we saw in Sec. 2.4, such quantification imposes an encoding overhead on every generated verification condition, so unfolding a triple should only ever quantify over program *states*, akin to the definition we had for simple stateful computations (2.5). Given a weakest-precondition transformer $\mathbf{wp} : \forall \alpha,\ M\ \alpha \to W\ \alpha$ for some computational monad M and specification monad W, the Hoare triple must therefore be defined as

$$\{pre\}\ x\ \{post\} \triangleq pre \leq \mathbf{wp}\ x\ post \tag{4.1}$$

In this case, $\leq$ should be an order not on the type of specifications — whose order, as in (2.6), quantifies over postconditions — but on the type of *pre* terms, which, as we will see below, quantify over states. This is exactly the distinction drawn in Sec. 2.4: a quantified state, even a higher-order one, can be lifted into an automation backend's

context as a collection of uninterpreted symbols, whereas a quantified predicate over such states cannot.

This gives us two clues. First, we need to actually fix the types of pre- and postconditions for our monadic triples, say *PrePred* and *PostPred*. Second, there should be a way to "apply" $\mathbf{wp}\ x\ :\ W\ \alpha$ to postconditions *post*. If we want to relate the result of such an application to preconditions via some order $\leq$, its type must also be *PrePred*. This gives us a clue on the shape of $W\ \alpha$: it should be $\mathit{PostPred} \to \mathit{PrePred}$. Finally, if we want to reason about programs modularly, in particular, reason about sequential composition in terms of its composites, we need to establish the so-called *cut* rule for Hoare triples [57]: to prove the Hoare triple $\{\mathit{pre}\}\ x \gg\!= f\ \{\mathit{post}\}$, it should be sufficient to provide some postcondition $\mathit{post}' : \mathit{PostPred}$ such that we can validate $\{\mathit{pre}\}\ x\ \{\mathit{post}'\}$ and, for any output $v\ :\ \alpha$ of x, $\{\mathit{post}'\ v\}\ f\ v\ \{\mathit{post}\}$. The latter term, however, does not type check: post' is of type *PostPred*, but should be of type $\alpha \to \mathit{PrePred}$. Thus, we must have a projection from *PostPred* into $\alpha \to \mathit{PrePred}$, or in other words $\mathit{PostPred} = (\alpha \to \mathit{PrePred}) \times \mathit{PostPred}'$ for some $\mathit{PostPred}'$. To this end, we focus on specification monads $W\ \alpha$ of the shape $(\alpha \to \tau) \times \tau' \to \tau$, for some types τ' and τ, or, in the curried version, $(\alpha \to \tau) \to \tau' \to \tau$. We call the latter a *type of predicate transformers* and denote it as $\mathsf{PredTrans}\ \tau\ \tau'\ \alpha$.

4.1.2 Predicate Transformer Monad

It turns out that $\mathsf{PredTrans}\ \tau\ \tau'$ (the same as $\lambda\alpha.\ \mathsf{PredTrans}\ \tau\ \tau'\ \alpha$) is in fact a monad, similar to the *continuation monad* [133] $\mathsf{ContM}\ \tau\ \alpha \triangleq (\alpha \to \tau) \to \tau$, which is the special case of $\mathsf{PredTrans}\ \tau\ \tau'\ \alpha$ with τ' a trivial Unit type. Before giving a formal definition of `pure` and `bind`, let us first discuss the intuition behind PredTrans.

We have already seen how instantiations of $\mathsf{ContM}\ \tau\ \alpha$ serve as a useful abstraction for certain specification monads. Both Sec. 2.3.2 and Sec. 2.3.4 demonstrate the utility of $\mathsf{ContM}\ \mathsf{hprop}\ \alpha$ as a specification monad for heap-manipulating programs, turning postconditions of computations into their preconditions. We have also seen that this monad can incorporate specifications for non-deterministic computations (2.2). In general, $\mathsf{ContM}\ (\sigma \to \mathsf{Prop})\ \alpha$ can model the weakest preconditions of computations operating on a state σ and possibly featuring nondeterminism and divergence (see the `DivM` instance from Sec. 3.4). This monad, however, does not require an additional τ' type, so the observant reader might wonder whether such an extension is even relevant in practice.

Cast in monadic terms, the plain continuation monad falls short for languages with non-local control flow. Leino [71] showed that, for such languages, the weakest precondition of a program x must receive two arguments: a postcondition *post* on program paths terminating normally and a postcondition *epost* on program paths terminating with an exception. With this, we can reason about non-local control flow operators `throw` and `try-catch` in the same way as we reason about `pure` and `bind`.

Just as `pure` has a single path that terminates normally, `throw` has a single path that terminates with an exception. Hence, their weakest preconditions should just immediately return *post* and *epost*, respectively:

$$\mathbf{wp}\ (\mathsf{pure}\ x)\ \mathit{post}\ \mathit{epost} = \mathit{post}\ x \qquad \mathbf{wp}\ (\mathsf{throw}\ e)\ \mathit{post}\ \mathit{epost} = \mathit{epost}\ e \tag{4.2}$$

For `bind`, the `let` $v \leftarrow x$ `in` f statement (notation for $x \gg\!= f$) runs x and passes control to f upon normal termination of x. Similarly, `try` x `catch` $e \Rightarrow f$ runs x and passes control to f upon termination of x with an exception. Hence, the **wp** rules simply thread the postconditions into the appropriate components:

$$\mathbf{wp}\ (\texttt{let}\ v \leftarrow x\ \texttt{in}\ f)\ post\ epost = \mathbf{wp}\ x\ (\lambda v,\ \mathbf{wp}(f\ v)\ post\ epost)\ epost \tag{4.3}$$

$$\mathbf{wp}\ (\texttt{try}\ x\ \texttt{catch}\ e \Rightarrow f)\ post\ epost = \mathbf{wp}\ x\ post\ (\lambda e,\ \mathbf{wp}(f\ e)\ post\ epost) \tag{4.4}$$

This finally answers the question raised above: the exceptional postcondition *epost* is precisely the τ' argument of PredTrans $\tau\ \tau'\ \alpha$ — the extra structure that the plain continuation monad lacks. For a single exception type ε, taking $\tau' = \varepsilon \to \tau$ recovers exactly the two-argument **wp** above, and richer non-local control flow corresponds to richer choices of τ'. Following this intuition, from now on we write *Pred* instead of τ and *EPred* instead of τ'.

These ideas have been further generalised to languages with multiple types of exceptions, where *EPred* becomes a collection of postconditions, one for each kind of exception [25, 71], and even to model the semantics of languages with arbitrary forward jumps, where *EPred* is a finite mapping from jump labels to postconditions on the corresponding jump points [41].

4.1.3 *Weakest Precondition Transformer*

Now we are ready to define a weakest-precondition transformer for monads:

Definition 4.1. We say that a monad M is equipped with a weakest-precondition transformer if there are

- *Complete lattices Pred* and *EPred*
- A function $\mathbf{wp}_M : \forall\ \alpha,\ M\ \alpha \to \mathsf{PredTrans}\ Pred\ EPred\ \alpha$

such that the following laws hold:

1. for any $v : \alpha$, $post : \alpha \to Pred$ and $epost : EPred$, $post\ v \leq \mathbf{wp}_M\ (\mathsf{pure}\ v)\ post\ epost$
2. for any $x : M\ \alpha$, $f : \alpha \to M\ \beta$, $post : \alpha \to Pred$ and $epost : EPred$
   $\mathbf{wp}_M\ x\ (\lambda v,\ \mathbf{wp}_M\ (f\ v)\ post\ epost)\ epost \leq \mathbf{wp}_M(x \gg\!= f)\ post\ epost$
3. for any $x : M\ \alpha$, $post$, $post' : \alpha \to Pred$ and $epost$, $epost' : EPred$,
   if $post \leq post'$ and $epost \leq epost'$, then $\mathbf{wp}_M\ x\ post\ epost \leq \mathbf{wp}_M\ x\ post'\ epost'$

The first two laws guarantee that $\mathbf{wp}_M$ respects `pure` and `bind`, and the last one ensures that $\mathbf{wp}_M$ is monotone. For brevity, we will omit $_M$ suffix in $\mathbf{wp}_M$ and simply write **wp**, when the underlying monad is clear for the context. Note that we require *Pred* and *EPred* to be complete lattices rather than regular orders and state `bind` and `pure` laws as an inequality rather than equality and also require. We will justify these choices by the needs of concrete instances below. However, before we see examples of **wp** definitions for concrete monads, note that, once a monad M is equipped with a weakest-precondition transformer, we can define a Hoare triple according to (4.1):

Definition 4.2 (Hoare Triple)**.** Consider a monad M which is equipped with a weakest-precondition transformer $\mathbf{wp}_M$ over complete lattices *Pred* and *EPred*. For $x : M\ \alpha$, $pre : Pred$, $post : \alpha \to Pred$ and $epost : EPred$, we define the Hoare triple statement $\{pre\}\ x\ \{post; epost\}$ as

$$pre \leq \mathbf{wp}\ x\ post\ epost$$

In contrast to the Dijkstra-monad triple (2.6), which compares two elements of the specification monad and thereby quantifies over postconditions, Definition 4.2 compares two *preconditions*. As we will see with concrete instances below, unfolding it only ever quantifies over program states, so the definition introduces no encoding overhead: for first-order specifications the formula remains first-order. Better yet, it degrades gracefully for *higher-order* states, such as the function- and relation-valued protocol states of Veil: since the state quantifier is the outermost one, the state can be lifted into the context as a collection of uninterpreted symbols — an option unavailable for the quantified predicates over such states produced by (2.6). Keeping proof obligations free of this overhead is what makes it possible to extend the automation with backends such as SMT solvers without enlarging the trusted code base; we explain this in detail in Chapter 5. Note also where the quantification over postconditions went: it did not disappear, but moved into the laws of Definition 4.1, which do quantify over *post* and *epost*. Those laws, however, are proved once per monad, in Lean, by the library author — no quantifier over postconditions ever reaches a proof obligation about a user's program.

Example 4.1 (State)**.** For $\mathsf{StateM}\ \sigma$ we take the lattice of normal postconditions to be the predicates on the state, $Pred = \sigma \to \mathsf{Prop}$, ordered pointwise by implication; since the monad raises no exceptions, the lattice of exceptional postconditions is trivial, $EPred = \mathsf{Unit}$. A computation $x : \mathsf{StateM}\ \sigma\ \alpha$ run from a state s produces a single pair $x\ s = (a, s')$, so its weakest precondition simply asserts *post* of that result:

$$\mathbf{wp}_{\mathsf{StateM}}\ x\ post\ epost \triangleq \lambda s,\ post\ (\mathsf{fst}\ (x\ s))\ (\mathsf{snd}\ (x\ s)).$$

The exceptional postcondition *epost* is ignored, as expected for a monad with no exceptional paths. Note that the Hoare triple of Definition 4.2 unfolds exactly to (2.5), *i.e.*

$$\forall s,\ pre\ s \Longrightarrow post\ (\mathsf{fst}\ (x\ s))\ (\mathsf{snd}\ (x\ s)),$$

whose only quantifier ranges over states, not over postconditions.

Example 4.2 (State with exceptions)**.** For $\mathsf{EStateM}\ \varepsilon\ \sigma = \mathsf{ExceptT}\ \varepsilon\ (\mathsf{StateM}\ \sigma)$ the normal-postcondition lattice is unchanged, $Pred = \sigma \to \mathsf{Prop}$, but now the exceptional lattice is inhabited: following Sec. 4.1, with a single exception type ε we take $EPred = \varepsilon \to \sigma \to \mathsf{Prop}$, a postcondition relating the raised error to the state at the point of failure. Running $x : \mathsf{EStateM}\ \varepsilon\ \sigma\ \alpha$ from a state s produces a result together with a final state — either $\mathsf{ok}\ a\ s'$ or $\mathsf{error}\ e\ s'$ — on which the weakest precondition dispatches:

$$\mathbf{wp}_{\mathsf{EStateM}}\ x\ post\ epost \triangleq \lambda s,\ \begin{cases} post\ a\ s' & \text{if } x\ s = \mathsf{ok}\ a\ s', \\ epost\ e\ s' & \text{if } x\ s = \mathsf{error}\ e\ s'. \end{cases}$$

Now both continuations are used: `ok` feeds the value and state to *post*, while `error` feeds the error and state to *epost*.

Stacking a second exception layer, $\mathsf{ExceptT}\ \varepsilon_1\ (\mathsf{ExceptT}\ \varepsilon_2\ (\mathsf{StateM}\ \sigma))$, leaves *Pred* untouched but enlarges the exceptional lattice. A computation may now finish normally or fail with an ε_1- or an ε_2-error, so $\mathit{EPred} = (\varepsilon_1 \to \sigma \to \mathsf{Prop}) \times (\varepsilon_2 \to \sigma \to \mathsf{Prop})$ carries one postcondition per exception type, and the weakest precondition dispatches into three cases, routing each error to the matching component of *epost*. This is precisely the "collection of postconditions" generalisation of Sec. 4.1.

Example 4.3 (Randomised Generation)**.** To model randomised generation for property-based testing applications, Lean uses the `Gen` monad [69]. In a very simplified approximation, this monad can be defined as StateM Seed — stateful computations over the seed type Seed. To enable random generation inside this monad for a particular type τ, one has to provide a function $\mathsf{next}_\tau\ :\ \mathsf{Gen}\ \tau$, which uses some pseudo-random function to obtain an element of τ from the seed and updates the current seed, so that the next call to next_τ produces a different value. If we used the approach from Example 4.1 to define a weakest precondition for this monad, our pre- and postconditions would be predicates over the seed. However, the seed itself is an implementation detail, which we would not want to expose to the user. Instead, we would want our pre- and postconditions to quantify over an arbitrary seed implicitly, so that the *Pred* type would be just Prop, not $\mathsf{Seed} \to \mathsf{Prop}$. In particular, the weakest precondition for next should simply follow from predicate *post* being satisfied on all $t : \tau$:

$$(\forall t\ :\ \tau,\ \mathit{post}\ t) \Longrightarrow \mathbf{wp}_{\mathsf{Gen}}\ (\mathsf{next}_\tau)\ \mathit{post}\ \mathit{epost}$$

Hence, we must define $\mathbf{wp}_{\mathsf{Gen}}$ as

$$\mathbf{wp}_{\mathsf{Gen}}\ x\ \mathit{post}\ \mathit{epost} \triangleq \forall s,\ \mathit{post}\ (\mathsf{fst}(x\ s))$$

That is, we want the postcondition to hold on the result of $x\ s$ for any initial seed s.

Note that, while this definition leads to a sound weakest-precondition calculus, unlike the other monads we have seen, the **wp** of a sequential composition is *not* equal to the nesting of the **wp**'s. A concrete instance makes this visible. Take $\mathsf{Seed} = \mathbb{N}$ with $\mathsf{next}\ s = (s,\ s+1)$, which returns the current seed and advances it, and draw twice:

$$p \triangleq \texttt{let}\ a \leftarrow \mathsf{next}\ \texttt{in}\ \texttt{let}\ b \leftarrow \mathsf{next}\ \texttt{in}\ \mathsf{pure}\ (a + 1 == b).$$

Running p from any seed s draws the consecutive pair $a = s$, $b = s+1$, so p always returns true. Let the postcondition be $\mathit{post}\ r \triangleq (r = \mathsf{true})$, asserting that the returned boolean is true. If we reasoning about p as a whole, its weakest precondition is $\forall s,\ (s + 1 = s + 1)$, which holds. However, if we reason *compositionally*, the second draw is analysed under its own, independently quantified seed, giving $\forall s,\ \forall s',\ (s + 1 = s')$, which is false (take $s = s' = 0$). The compositional form implies the monolithic one but not the converse, so the `bind` law holds only as the inequality $\leq$, never as an equality. This is exactly why Definition 4.1 demands $\leq$ rather than $=$: hiding the seed, by replacing $\mathsf{Seed} \to \mathsf{Prop}$ with Prop, is precisely what costs us exactness.

This example also explains the completeness requirement of Definition 4.1. The quantifier $\forall\ s$ in the definition of $\mathbf{wp}_{\mathsf{Gen}}$ is an infimum over the entire type Seed; for such universally quantified definitions to denote a genuine element of the assertion lattice in general, that lattice must be closed under arbitrary meets — that is, it must be a *complete* lattice.

4.2 Weakest Precondition for Monad Transformers

In the previous section, we discussed a recipe for deriving weakest-precondition calculi with specification monads of the form PredTrans *Pred EPred* α. In this section, we address the challenge of deriving weakest-precondition transformers for composite monads.

For example, assume we have defined a weakest-precondition transformer instance for a monad M, and now we want to add a state component to it by making use of the StateT monad transformer. Do we have to define an instance of weakest-precondition transformer for StateT $\sigma\ M\ \alpha$ from scratch, proving all the required properties, or is there a compositional way to do so, allowing for proof reuse? Below, we provide an approach to achieve the latter, by introducing a weakest-precondition transformer analogue for monad transformers.

The main idea is simple: for a given monad transformer T, which "adds" a particular effect, let us define a weakest-precondition transformer instance for this transformer as if it were applied to an arbitrary monad M, which itself already comes with a weakest-precondition transformer instance over some *Pred*, *EPred* and $\mathbf{wp}_M$.

4.2.1 State Transformer

We illustrate how this can be done for StateT transformer. First, assuming that the assertion languages for M are *Pred* and *EPred*, we have to define assertion languages for StateT $\sigma\ M$. In assertions about the normally terminating paths of stateful computations in StateT $\sigma\ M$, we want to be able to state everything we could have stated for M, and additionally reason about the state. This suggests defining the new assertion language as $\sigma \to \mathit{Pred}$. At the same time, since StateT $\sigma\ M$ cannot throw more exceptions than M originally could, *EPred* stays the same. Now, we need to define the weakest-precondition transformer $\mathbf{wp}_{\mathsf{StateT}\ \sigma\ M}$ of type StateT $\sigma\ M\ \alpha \to$ PredTrans $(\sigma \to \mathit{Pred})\ \mathit{EPred}\ \alpha$ by making use of $\mathbf{wp}_M$. Before going through its definition, remember that StateT $\sigma\ M\ \alpha$ is defined as $\sigma \to M(\alpha \times \sigma)$: given an initial state of type σ, computations in StateT $\sigma\ M\ \alpha$ yield monadic executions in M carrying a result of type α and an updated state. Therefore, for x of type StateT $\sigma\ M\ \alpha$, a normal postcondition $\mathit{post} : \alpha \to \sigma \to \mathit{Pred}$, and an exceptional postcondition $\mathit{epost} : \mathit{EPred}$, we define

$$\mathbf{wp}_{\mathsf{StateT}\ \sigma\ M}\ x\ \mathit{post}\ \mathit{epost} \triangleq \lambda(s : \sigma).\ \mathbf{wp}_M\ (x\ s)\ (\lambda(a, s'),\ \mathit{post}\ a\ s')\ \mathit{epost} \tag{4.5}$$

Running x from an initial state s produces a computation $x\ s : M(\alpha \times \sigma)$ in the underlying monad. We hand its weakest precondition $\mathbf{wp}_M$ the postcondition that, given the returned value a and final state s', demands $\mathit{post}\ a\ s'$; the exceptional postcondition *epost* passes through unchanged, since StateT $\sigma\ M$ adds no exceptional behaviour of its

own.

As we prove in our Lean development, by assuming that $\mathbf{wp}_M$ satisfies the laws of Definition 4.1, we can derive the same laws for $\mathbf{wp}_{\mathsf{StateT}\ \sigma\ M}$. This means that, when composing StateT with other monads, one does not have to redefine and reprove everything from scratch, obtaining a correct-by-construction weakest-precondition transformer.

4.2.2 *Exception Transformer*

The construction for ExceptT follows the same recipe, this time enlarging the exceptional assertion language rather than the normal one. Recall that $\mathsf{ExceptT}\ \varepsilon\ M\ \alpha$ is defined as $M\ (\mathsf{Except}\ \varepsilon\ \alpha)$: a computation returns, inside the underlying monad M, either a value $\mathsf{ok}\ a$ or an error $\mathsf{error}\ e$ with $e : \varepsilon$. Since $\mathsf{ExceptT}\ \varepsilon\ M$ does not change how normal results are reported, the normal assertion language stays Pred. Its exceptional behaviour, however, now has two sources: the new ε-exceptions introduced by the transformer, and whatever exceptions M could already throw. Accordingly, the exceptional assertion language becomes the product $(\varepsilon \to \mathit{Pred}) \times \mathit{EPred}$.

We then define $\mathbf{wp}_{\mathsf{ExceptT}\ \varepsilon\ M}$ of type $\mathsf{ExceptT}\ \varepsilon\ M\ \alpha \to \mathsf{PredTrans}\ \mathit{Pred}\ ((\varepsilon \to \mathit{Pred}) \times \mathit{EPred})\ \alpha$ from $\mathbf{wp}_M$. For $x : \mathsf{ExceptT}\ \varepsilon\ M\ \alpha$, a normal postcondition $\mathit{post} : \alpha \to \mathit{Pred}$, and an exceptional postcondition $(\mathit{epost}_\varepsilon, \mathit{epost}_M) : (\varepsilon \to \mathit{Pred}) \times \mathit{EPred}$,

$$\mathbf{wp}_{\mathsf{ExceptT}\ \varepsilon\ M}\ x\ \mathit{post}\ (\mathit{epost}_\varepsilon, \mathit{epost}_M) \triangleq \mathbf{wp}_M\ x\ \left(\lambda r,\ \begin{cases} \mathit{post}\ a & \text{if } r = \mathsf{ok}\ a, \\ \mathit{epost}_\varepsilon\ e & \text{if } r = \mathsf{error}\ e \end{cases}\right)\ \mathit{epost}_M \tag{4.6}$$

Viewing x as the M-computation it is, we run its weakest precondition $\mathbf{wp}_M$ with a postcondition that dispatches on the returned Except value — routing a success $\mathsf{ok}\ a$ to post and a new ε-error $\mathsf{error}\ e$ to $\mathit{epost}_\varepsilon$ — while the exceptions of M itself are left to epost_M. As with StateT, assuming $\mathbf{wp}_M$ satisfies the laws of Definition 4.1 lets us derive them for $\mathbf{wp}_{\mathsf{ExceptT}\ \varepsilon\ M}$, so the transformer composes without redoing any proofs.

4.2.3 *General Transformer*

Now we are ready to define an analogue of the weakest-precondition transformer for monad transformers. A weakest-precondition transformer for a monad M is built over its assertion lattices Pred and EPred, which are themselves Lean types. For a monad transformer, which turns one monad into another, the corresponding objects should be type constructors on such lattices, or more specifically, two *endofunctors* on complete lattices (*i.e.*, mappings $\mathsf{Type} \to \mathsf{Type}$ that preserve identity, function composition and complete lattice structure): one, F, acting on the normal lattice Pred, and one, EF, on the exceptional lattice EPred. For instance, $\mathsf{StateT}\ \sigma\ M$ turned the normal lattice Pred into $\sigma \to \mathit{Pred}$ while leaving EPred untouched, so it acts by $F_{\mathsf{StateT}\ \sigma\ M}\ \mathit{Pred} \triangleq \sigma \to \mathit{Pred}$ and $EF_{\mathsf{StateT}\ \sigma\ M}\ \mathit{EPred} \triangleq \mathit{EPred}$. Dually, $\mathsf{ExceptT}\ \varepsilon\ M$ left Pred untouched and enlarged the exceptional lattice via $EF_{\mathsf{ExceptT}\ \varepsilon\ M}\ \mathit{EPred} \triangleq (\varepsilon \to \mathit{Pred}) \times \mathit{EPred}$. This suggests defining a weakest-precondition transformer *for a monad transformer* T as a pair of endofunctors F and EF such that, for any monad M equipped with a weakest-precondition transformer over Pred and EPred, the composite $T\ M$ is again equipped with one over $F\ \mathit{Pred}$ and $EF\ \mathit{EPred}$, satisfying the laws of Definition 4.1.

```
class WPMonad (m : Type → Type) (Pred EPred : outParam Type)
    [Monad m] [CompleteLattice Pred] [CompleteLattice EPred]
    extends LawfulMonad m where
  wpTrans : m α → PredTrans Pred EPred α
  wp_trans_pure (x : α) : pure x ⊑ wpTrans (pure x)
  wp_trans_bind (x : m α) (f : α → m β) :
    wpTrans x >>= (wpTrans ∘ f) ⊑ wpTrans (x >>= f)
  wp_trans_monotone (x : m α) : (wpTrans x).monotone
```

Figure 4.1: The `WPMonad` type class.

```
instance : WPMonad (EStateM ε σ) (σ → Prop) EPost⟨ε → σ → Prop⟩ where
  wpTrans x := ⟨fun post epost s =>
    match x s with
    | .ok a s'    => post a s'
    | .error e s' => epost.head e s'⟩
  -- laws: cases on `x s`
```

Figure 4.2: An instance for `EStateM`

Definition 4.3. Assume T is a monad transformer and F and EF are endofunctors on complete lattices. We say that T is equipped with a weakest-precondition transformer if, for any monad M equipped with a weakest-precondition transformer $\mathbf{wp}_M$ over *Pred* and *EPred*, there exists a function $\mathbf{wp}_{T\ M} : \forall \alpha,\ T\ M\ \alpha \to \mathsf{PredTrans}\ (F\ Pred)\ (EF\ EPred)\ \alpha$ satisfying the laws from Definition 4.1.

4.3 Implementing Foundations in Lean Standard Library

In this section, we discuss elements of the weakest-precondition implementation, which lives in the Lean standard library.

4.3.1 *Inferring Assertion Types for Monads*

Fig. 4.1 shows the encoding of the weakest-precondition transformer as a type class `WPMonad`, paraphrasing Definition 4.1 in Lean. This class requires the assertion languages `Pred` and `EPred` to be complete lattices, and the type `m` to be a lawful monad, *i.e.*, it should have `pure` and `bind` operations satisfying the monadic laws. The only unusual bit in our encoding is the annotation `outParam` of the type class arguments `Pred` and `EPred`, which means that, even if the types of the assertion languages are unspecified, Lean will try to infer them automatically based on the type class instances available in the context. To explain the true utility of `outParam`, assume we have a computation of type `EStateM String Int Unit` and want to get its weakest precondition by passing it to the respective `wp` function. Remember that the `wp` function has the following type for a computation monad `m` and assertion languages `Pred` and `EPred` (Sec. 4.1):

```
∀ {m} {α Pred EPred} [CompleteLattice Pred] [CompleteLattice EPred]
    [WPMonad m Pred EPred], m α → (α → Pred) → EPred → Pred
```

When we apply `wp` to `x : EStateM String Int Unit`, Lean is able to automatically infer its arguments `m` and `α` from the type of `x`, but the assertion languages `Pred` and `EPred` are not determined by `x`. However, since both `Pred` and `EPred` are marked as `outParam`, they are *synthesised* from the `WPMonad` instance for `EStateM` automatically. In this case, according to

```
instance [Monad m] [CompleteLattice Pred] [CompleteLattice EPred]
    [WPMonad m Pred EPred] : WPMonad (StateT σ m) (σ → Pred) EPred where
  wpTrans x := ⟨fun post epost s =>
    wp (x.run s) (fun (a, s') => post a s') epost⟩
  -- laws from those of m
```

Figure 4.3: State transformer

```
instance [Monad m] [CompleteLattice Pred] [CompleteLattice EPred]
    [WPMonad m Pred EPred] :
    WPMonad (ExceptT ε m) Pred (EPost.cons (ε → Pred) EPred) where
  wpTrans x := ⟨fun post epost =>
    wp x.run (fun r => match r with
      | .ok a    => post a
      | .error e => epost.head e) epost⟩
  -- laws from those of m
```

Figure 4.4: Exception transformer

the instance in Fig. 4.2, that would be `Int` → Prop and EPost⟨`String` → `Int` → Prop⟩. In the next subsection we will explain what EPost⟨...⟩ is and where it comes from.

4.3.2 *Deriving Instances for Transformers*

While instances for base monads such as `EStateM` are written by hand, the instances for monad transformers are defined once and reused for every stack built from them. Fig. 4.3 and Fig. 4.4 show the two principal cases, mirroring the constructions of (4.5) and (4.6). Each takes a `WPMonad` instance for an underlying monad `m` over `Pred` and `EPred` and produces one for the transformed monad over suitably adjusted assertion languages. `StateT` leaves the exceptional lattice `EPred` untouched and turns the normal lattice `Pred` into $\sigma \to$ `Pred`, so a normal postcondition may now refer to the state. Its `wpTrans` runs the underlying computation from the current state `s` and feeds `wp` a postcondition that, given the returned value and the updated state, applies the original postcondition to them. Dually, `ExceptT` leaves `Pred` untouched and grows the exceptional lattice. We encode the exceptional postcondition as a *stack of layers*, one per exception type in scope: EPost.nil is the empty stack, for a monad that throws nothing, and EPost.cons h t adds a layer of postcondition type h on top of a tail t — the Lean encoding of the product $(\varepsilon \to \mathit{Pred}) \times \mathit{EPred}$ from (4.6). A fully built stack EPost.cons e_1 $(\cdots$ EPost.nil$)$ is displayed with the shorthand EPost$\langle e_1, \ldots \rangle$; a base monad with a single exception type, such as `EStateM` (Fig. 4.2), thus has exceptional lattice EPost$\langle \varepsilon \to \sigma \to \text{Prop} \rangle$ = EPost.cons $(\varepsilon \to \sigma \to \text{Prop})$ EPost.nil. Accordingly, `ExceptT` prepends a single layer, EPost.cons $(\varepsilon \to$ `Pred`$)$ `EPred`, so a computation may now fail with a fresh ε-error on top of whatever `m` could already throw; its `wpTrans` dispatches on the `Except` result, routing a success to `post` and a new error to the freshly added layer, `epost.head`.

Because these instances are generic in `m`, they compose along an arbitrary monad stack. For `ExceptT String (StateT Balance m)`, for example, Lean assembles the assertion languages from the inside out: the inner `StateT` turns `m`'s normal lattice `Pred` into `Balance` → `Pred` while keeping its `EPred`, and the outer `ExceptT` keeps that normal lattice but extends the exceptional one to EPost.cons (`String` → `Balance` → `Pred`) `EPred`. The `outParam` mechanism from the previous subsection drives this synthesis automatically, so the as-

sertion languages of a composite monad are never spelled out by hand.

4.4 Ordered Monad Algebras

The weakest-precondition transformer definition we gave in Sec. 4.1 is quite general, as it accounts for "non-exact" definitions of **wp** and for a general type of exceptional postconditions *EPred*. However, for a large class of common effects, such as state, divergence and non-determinism, this generality is excessive. A suitable specification monad for these effects would be just a continuation monad $\mathsf{ContM}\ \mathit{Pred}\ \beta$ for some type *Pred* of pre- and postconditions. Moreover, the first two laws from Definition 4.1 would be exact for the corresponding **wp**s. Of course, if we have an exact $\mathbf{wp} : \forall \beta,\ M\ \beta \to \mathsf{ContM}\ \mathit{Pred}\ \beta$, we can simply cast it to $\mathbf{wp} : \forall \beta,\ M\ \beta \to \mathsf{PredTrans}\ \mathit{Pred}\ \mathsf{EPost}\langle\rangle\ \beta$ and still plug it into our framework. However, deriving such **wp**s can be simplified, and in this section we discuss how.

4.4.1 Monad Algebras

For a given monad M, there is a one-to-one correspondence between weakest precondition morphisms $\mathbf{wp} : M\ \beta \to \mathsf{Cont}\ \mathit{Pred}\ \beta$ respecting `pure` and `bind` (but not necessarily monotone) and structures called *monad algebras* that relate M with an assertion language *Pred*:[1]

Definition 4.4 (Monad Algebra)**.** For a given computational monad M we say that a type *Pred* and a morphism $\alpha : M\ \mathit{Pred} \to \mathit{Pred}$ form a *monad algebra* if the following two laws hold:

1. for any $p : \mathit{Pred}$, $\alpha\ (\mathsf{pure}\ p) = p$
2. for any $m : M\ \beta$, $f, g : \beta \to M\ \mathit{Pred}$, if $\forall x, \alpha(f\ x) = \alpha(g\ x)$ then $\alpha(m \gg\!= f) = \alpha(m \gg\!= g)$

In this context, the type *Pred* is referred to as the *monad algebra object*.

Before providing an intuition for this definition, we enhance the notion of a monad algebra to extend this one-to-one correspondence to weakest precondition morphisms that are monotone (3):

Definition 4.5 (Ordered Monad Algebra)**.** A complete lattice *Pred* with a partial order $\leq$ and a morphism $\alpha : M\ \mathit{Pred} \to \mathit{Pred}$ form an *ordered monad algebra* for a monad M if the following laws hold:

1. for any $p : \mathit{Pred}$, $\alpha\ (\mathsf{pure}\ p) = p$
2. for any $m : M\ \beta$, $f, g : \beta \to M\ \mathit{Pred}$, if $\forall x, \alpha(f\ x) \leq \alpha(g\ x)$ then $\alpha(m \gg\!= f) \leq \alpha(m \gg\!= g)$

The only difference with Definition 4.4 is that we replace "=" with "$\leq$" in the second

[1] This observation is discussed in Sec. 4.4 of the paper by Maillard et al. [78], who derive Dijkstra monads using a more general mechanism of *effect observations* [63].

law. From here on, we will only consider ordered monad algebras, referring to them as just monad algebras. Intuitively, the function $\alpha : M\ Pred \to Pred$ is a *symbolic runner*, which interprets M-computations ending with a postcondition in *Pred* as assertions in *Pred*. Specifically, α "treats" assertions as values: it "runs" the computation, returning an assertion that must hold in order for all its "resulting" assertions to hold true. One can think of α as a weakest precondition predicate transformer *for a fixed postcondition*. The first law of Definition 4.5 states that the "symbolic run" of a computation simply returning an assertion p is just p, while the second law captures the monotonicity property of α. That is, strengthening some part of the symbolic run of a program (by, *e.g.*, imposing a stronger assertion at the end) should only strengthen the overall outcome of a symbolic run.

4.4.2 *Deriving Weakest-Precondition Transformers from Monad Algebras*

Let us show how to derive a morphism **wp** from a given α of a monad algebra over monad M. Given $x : M\ \beta$,

$$\mathbf{wp}\ x : \mathsf{PredTrans}\ Pred\ \mathsf{EPost}\langle\rangle\ \beta \triangleq \lambda(post : \beta \to Pred)\ epost.\ \alpha\ (post \mathbin{<\$>} x) \tag{4.7}$$

The $<\$>$ operator above is a monadic mapping of type $(\beta \to \gamma) \to M\ \beta \to M\ \gamma$. To understand the intuition behind (4.7), note that $post \mathbin{<\$>} x$ is essentially the same monadic computation as x but returning an assertion $post\ a$ instead of an "ordinary" value a. Applying α to $(post \mathbin{<\$>} x)$ results in a precondition that must hold before running $post \mathbin{<\$>} x$ for it to return a true assertion.

For **wp** defined this way, we can state the following theorem (which we proved in Lean):

Theorem 4.4.1 (Properties of **wp** derived from Ordered Monad Algebra)**.** *If an assertion language Pred and a morphism $\alpha : M\ Pred \to Pred$ form an ordered monad algebra for a monad M, then **wp** derived using (4.7) satisfies the properties (1), (2), and (3).*

As mentioned in Chapter 2, a **wp** with properties (1), (2), and (3) immediately provides rules to compose the weakest preconditions of basic operations into specifications of composite programs. That said, we have not yet discussed how to derive **wp**s for basic operations for a given monad (*e.g.*, get/set of StateM). While we do not automate this aspect in this work, these derivations can usually be done mostly mechanically by unfolding the respective definitions of α and **wp**.

As an example, consider the derivation of **wp** for StateM's get operation, *i.e.*, a computation of type $\mathsf{StateM}\ \sigma\ \sigma$ which returns the underlying state. By definition, we unfold get into $\lambda s.\ (s, s)$: for an initial state s, get's outcome is this exact state s, hence the following derivation:

$$\begin{aligned}
\mathbf{wp}_{\mathsf{StateM}\ \sigma}\ \mathsf{get}\ post\ s &= \alpha_{\mathsf{StateM}\ \sigma}(post \mathbin{<\$>} \mathsf{get})\ s && (4.8)\\
&= \alpha_{\mathsf{StateM}\ \sigma}(post \mathbin{<\$>} \lambda s.\ (s, s))\ s && (4.9)\\
&= \alpha_{\mathsf{StateM}\ \sigma}(\lambda s.\ (post\ s, s))\ s = post\ s\ s && (4.10)
\end{aligned}$$

The last line (4.10) of the derivation above can be validated by unfolding the definition

of $\alpha_{\mathsf{StateM}\ \sigma}$.

Following the outlined methodology, obtaining a Hoare-style verifier and a respective VC generator for a computation monad M with ContM-shaped specification monad boils down to (a) selecting an assertion language *Pred* and a morphism α (both are usually relatively straightforward, as shown in Fig. 3.3), (b) proving the laws stated in Definition 4.5, and (c) deriving the definitions of **wp** for all operations specific to the monad.

4.5 Reasoning about Divergent Computations

So far our computations have always terminated. In this section we add general recursion, and show how the weakest-precondition framework reasons about loops that may diverge, both in the partial- and the total-correctness sense.

4.5.1 *Divergent Computations in Lean*

Lean allows one to define generally-recursive functions inside a class of monads that come with a *chain-complete partial order* (CCPO) on the type of their computations.

Definition 4.6 (Chain-Complete Partial Order)**.** A partial order $\leq$ on a type A is called *chain-complete* if for any chain $a_1 \leq a_2 \leq \ldots$ in A there exists the least upper bound $\bigsqcup_i a_i$ in A.

Definition 4.7 (Chain-Complete Monad)**.** A monad M is called *chain-complete* if it provides a chain-complete partial order on $M\ \alpha$ for any result type α.

For general recursion, it is additionally required that `bind` is monotone with respect to this CCPO:

Definition 4.8 (Monotonicity of `bind`)**.** A chain-complete monad M is called *monotone* if for any $m_1, m_2 : M\ \alpha$ and $f_1, f_2 : \alpha \to M\ \beta$, if $m_1 \leq m_2$ and $\forall x, f_1\ x \leq f_2\ x$, then $m_1 \gg\!= f_1 \leq m_2 \gg\!= f_2$.

With Definition 4.7 and Definition 4.8 holding for a monad M, Lean defines generally-recursive monadic computations in M following a Knaster–Tarski-style construction [1, 12]. A trivial instance of a chain-complete monotone monad is Option: its CCPO only asserts none $\leq$ some x for any x, modelling a divergent computation as none. Moreover, the Lean standard library establishes that if M is a chain-complete monotone monad and is transformed with one of the standard transformers, such as StateT, ExceptT, ReaderT, *etc*, the resulting monad remains chain-complete and monotone.

Given such a monad and a function $f : M\ \alpha \to M\ \alpha$ — one that takes the recursive call as an argument — Lean defines its fixpoint as $\bigsqcup_{x \in f^{\uparrow *}} x$, where $f^{\uparrow *} : M\ \alpha \to \mathsf{Prop}$ (pronounced "f iterated") is the smallest inductively defined set such that (1) for the bottom element $\bot$ of the corresponding CCPO, $f\ \bot \in f^{\uparrow *}$, (2) for each $x \in f^{\uparrow *}$, $f\ x \in f^{\uparrow *}$, and (3) for each chain $c \subseteq f^{\uparrow *}$, the least upper bound of c is also in $f^{\uparrow *}$.

With these amenities, Lean can define general loops (including the `while`-loop) for a chain-

$$\frac{\forall a,\ \{\mathsf{inv}(\mathsf{inl}\ a)\}\ m\ \{ab,\ \mathsf{inv}(ab)\,;\ epost\}}{\{\mathsf{inv}(\mathsf{inl}\ a_0)\}\ \mathsf{iter}\ a_0\ m\ \{b,\ \mathsf{inv}(\mathsf{inr}\ b)\,;\ epost\}}$$

(a) Partial iteration rule

$$\frac{\forall a,\ \{\mathsf{inv}(\mathsf{inl}\ a)\}\ m\ \{ab,\ \mathsf{inv}(ab) \wedge \ulcorner ab < a \urcorner\,;\ epost\} \qquad < \text{ is well-founded on } \alpha}{\{\mathsf{inv}(\mathsf{inl}\ a_0)\}\ \mathsf{iter}\ a_0\ m\ \{b,\ \mathsf{inv}(\mathsf{inr}\ b)\,;\ epost\}}$$

(b) Total iteration rule

Figure 4.5: Partial and total iteration rules

complete monotone monad M without having to provide a termination measure. Loom exploits this feature, providing a general iteration operator iter [136, Sec. 3.2], which can express many different kinds of loops, including `while`, `for`, `do-while`, *etc.* The operator is parametric in the underlying monad M and has type $\alpha \to (\alpha \to M\ (\alpha + \beta)) \to M\ \beta$. Intuitively, the output of the loop body is either an α or a β, modelled by the tagged union $+$. The type α is ascribed to the value computed by each iteration and passed to the next; the type β corresponds to a value indicating termination: if the body returns $b : \beta$, the loop terminates. Since nothing guarantees that the body will ever return a β, such a loop may diverge. Although Lean can execute this operator natively, it does not by itself provide an invariant-based mechanism for reasoning about it. We define one next.

4.5.2 *Reasoning about Partial and Total Correctness with Loops*

Loom's general Hoare-style rules for iter are shown in Fig. 4.5. Both rely on a traditional loop invariant inv. To match the type of iter, the invariant inv has type $\alpha + \beta \to \mathit{Pred}$, where Pred is the normal assertion language associated with the monad M; the exceptional postcondition $\mathit{epost} : \mathit{EPred}$ is threaded through both rules unchanged, since the loop does not handle the exceptions of M. Intuitively, $\mathsf{inv} : \alpha + \beta \to \mathit{Pred}$ bundles two functions, $\mathsf{inv}_{\mathsf{inl}} : \alpha \to \mathit{Pred}$ and $\mathsf{inv}_{\mathsf{inr}} : \beta \to \mathit{Pred}$: the former expresses an invariant that must hold on the result computed by each iteration, the latter the condition that must hold upon termination.

The partial-correctness rule (Fig. 4.5a) reads as follows: if the invariant holds on the initial value $a_0 : \alpha$, injected into $\alpha + \beta$ via inl, then after executing the iter-loop the invariant holds on the final result $b : \beta$, injected via inr. The rule carries no condition enforcing termination, matching the partial semantics of iter. The total-correctness rule (Fig. 4.5b) is similar but adds a termination condition, expressed via the assertion $\ulcorner ab < a \urcorner$, where $\ulcorner \cdot \urcorner$ injects a pure (effect-agnostic) proposition into Pred: it requires the loop result $ab : \alpha + \beta$ to be smaller than a when it has the form $\mathsf{inl}\ a'$, and is vacuously true otherwise. In addition, the rule requires the order $<$ on α to be *well-founded*, so that the termination measure cannot decrease indefinitely.

The total-correctness rule is valid for any monad on which iter can be defined. The partial-

correctness rule, however, is sound only when the weakest precondition interacts well with the CCPO of M. We capture this requirement by strengthening Definition 4.1:

Definition 4.9 (Partial Weakest-Precondition Transformer)**.** A chain-complete monad M equipped with a weakest-precondition transformer (Definition 4.1) is said to be equipped with a *partial* weakest-precondition transformer if, for any chain $c \subseteq M\ \alpha$, any $post : \alpha \to Pred$ and $epost : EPred$,

$$\bigwedge_{m \in c} \mathbf{wp}_M\ m\ post\ epost \ \leq\ \mathbf{wp}_M \Big(\bigsqcup_{m \in c} m \Big)\ post\ epost,$$

where $\bigwedge$ is the meet on $Pred$ and $\bigsqcup$ is the least upper bound in the CCPO on $M\ \alpha$.

Reading c as the chain of iterations of f, the law says that if a precondition implies $\mathbf{wp}_M\ m$ for every iteration m, then it implies the weakest precondition of the least upper bound of the chain — which coincides with the least fixpoint of f. In other words, to establish a property of the fixpoint it suffices to establish it for every finite iteration. The partial-correctness rule is sound for any monad equipped with a partial weakest-precondition transformer.

This condition is what pins down the symbolic treatment of divergence. Consider Option, modelling divergence as none, and a body $m \triangleq \lambda a.\ \mathsf{pure}\ (\mathsf{inl}\ a)$ that never returns a β, so that $\mathsf{iter}\ a_0\ m\ =\ \mathsf{none}$ diverges. If we set $\mathbf{wp}_{\mathsf{Option}}\ \mathsf{none}\ post\ epost\ =\ \mathsf{False}$, the partial rule becomes unsound: its premise holds vacuously for a trivial invariant, yet its conclusion would require False. Taking $\mathbf{wp}_{\mathsf{Option}}\ \mathsf{none}\ post\ epost\ =\ \mathsf{True}$ instead — so that a divergent computation vacuously satisfies any normal postcondition — makes Option a partial weakest-precondition transformer in the sense of Definition 4.9, validating the partial rule. The two readings correspond exactly to partial and total correctness: under the True interpretation divergence is benign, while a total-correctness specification rules it out via the termination measure of the total rule.

4.6 Reasoning about Non-Deterministic Computations

This section describes Loom's monad transformer for non-deterministic computations, NonDetT. We first equip it with a weakest-precondition transformer, supporting both *demonic* and *angelic* styles of reasoning, and then discuss how such computations are executed.

4.6.1 A Monad Transformer for Non-Determinism

We model non-determinism with a variant of a *program monad* [75], whose simplified definition is shown in Fig. 4.6. The first two constructors are analogous to the *Interaction Tree* structure [136] and correspond to the `pure` and `bind` operations: `pure` returns a value, while `vis x k` performs an effect `x` of the underlying monad `M` and continues with `k`. The third constructor encodes a *Hilbert epsilon operator* [40], whose operational meaning is to non-deterministically pick an element of type τ satisfying the predicate `p`.

Since NonDetT M adds neither state nor exceptions, its assertion languages are exactly

```
inductive NonDetT M α where
  | pure : α → NonDetT M α
  | vis {β}   : M β → (β → NonDetT M α) → NonDetT M α
  | pick (τ) : (τ → Prop) → (τ → NonDetT M α) → NonDetT M α
```

Figure 4.6: A monad transformer for non-determinism

those of M, namely *Pred* and *EPred*. The only question is how a non-deterministic computation should be evaluated against a normal postcondition: we may demand that the postcondition hold for *all* possible outcomes or for *at least one* of them, corresponding to *demonic* and *angelic* non-determinism [18]. The weakest precondition therefore recurses over the structure of the computation, expressing the two outcomes as a *meet* (resp. *join*) over the choices. For the demonic reading,

$$\mathbf{wp}\ x\ post\ epost \triangleq \begin{cases} post\ a & \text{if } x = \mathsf{pure}\ a, \\ \mathbf{wp}_M\ y\ (\lambda b,\ \mathbf{wp}\ (k\ b)\ post\ epost)\ epost & \text{if } x = \mathsf{vis}\ y\ k, \\ \bigwedge_{a\,:\,\tau,\ p\ a}\ \mathbf{wp}\ (k\ a)\ post\ epost & \text{if } x = \mathsf{pick}\ \tau\ p\ k. \end{cases}$$

A `pure` computation simply applies *post*; a `vis` runs the underlying effect through $\mathbf{wp}_M$ and continues; and a `pick` takes the meet $\bigwedge$ of the weakest preconditions of all admissible choices, which requires *Pred* to be a complete lattice. The *angelic* instance is obtained by replacing the meet with the join $\bigvee$; in the interest of space we discuss only the demonic one. The exceptional postcondition *epost* is threaded unchanged, as NonDetT introduces no new exceptions.

The Hilbert-epsilon operator itself, written `x :| p` in the surface syntax of Cashmere (after Dafny [73]), is the special case of pick that returns the chosen element:

$$\mathsf{pickSuchThat}\ \tau\ p \triangleq \mathsf{NonDetT.pick}\ \tau\ p\ \mathsf{NonDetT.pure}.$$

4.6.2 *Executing Non-Deterministic Computations*

We now show how to soundly execute computations in NonDetT under the demonic, partial-correctness reading; our formalisation provides the remaining combinations. The goal is a function $\mathsf{NonDetT.run} : \mathsf{NonDetT}\ M\ \alpha \to M\ \alpha$ whose result *refines* the original computation, *i.e.*, produces one of its possible outcomes.

The first two constructors execute as in Interaction Trees [136]: pure becomes the underlying monad's `pure`, and vis becomes a `bind`. To execute a non-deterministic choice we need extra structure on τ and `p`. Following the treatment of Hilbert's choice in Dafny [73], we parametrise NonDetT.run by a *witness*: an inductive type Extract (Fig. 4.7) that recurses over vis and, at each pick, requires a $\mathsf{Findable}\ \tau\ p$ instance. The essential component of this class is $\mathsf{find} : \mathsf{Option}\ \tau$ together with its specification, ensuring that any value it returns satisfies `p`:

$$\mathsf{find\_spec} : \forall x : \tau,\ \mathsf{find}\ \tau\ p = \mathsf{some}\ x \implies p\ x.$$

Loom infers a Findable instance automatically whenever τ is countable and `p` is decidable, by enumerating τ until a witness is found. Given such an instance, NonDetT.run (Fig. 4.8) recurses over vis and, at each pick, calls find: if it returns a value the run continues, other-

```
inductive Extract : NonDetT M α → Type where
  | pure : ∀ x, Extract (.pure x)
  | vis (x cont) : (∀ y, Extract (cont y)) → Extract (.vis x cont)
  | pick (τ p cont) [Findable τ p] :
      (∀ t, Extract (cont t)) → Extract (.pick τ p cont)
```

Figure 4.7: Choices for a computation

```
def NonDetT.run (x : NonDetT M α) : Extract x → M α
  | pure x => pure x
  | vis x k kEx => x >>= fun a => NonDetT.run (k a) (kEx a)
  | pick τ p k kEx =>
      match find τ p with
      | some x => NonDetT.run (k x) (kEx x)
      | none   => ⊥
```

Figure 4.8: Running a computation

wise it yields the divergence value $\bot$.

We have proven the following soundness theorem in Lean. It requires M to be equipped with a *partial* weakest-precondition transformer (Definition 4.9), so that divergence is treated as partial correctness:

Theorem 4.6.1 (Soundness of NonDetT.run)**.** *If monad M is equipped with a partial weakest-precondition transformer, then for any c : NonDetT M α, witness ex : Extract c, and pre-/postconditions,*

$$\{pre\}\, c\, \{post; epost\} \implies \{pre\}\, \mathit{NonDetT.run}\, c\, ex\, \{post; epost\}\,.$$

If M is not equipped with a partial weakest-precondition transformer, soundness of NonDetT.run additionally requires that every reachable non-deterministic choice be *realisable*: whenever a pick with predicate p is reachable, p must hold for some value. Formalising reachability calls for a *weakest liberal precondition* calculus, whose details can be found in our Lean development.

4.7 Erasable Ghost Code

In the previous sections, we have discussed how to foundationally reason about basic effects. We have shown how to define them in the Lean proof assistant, derive a weakest-precondition transformer and prove basic rules — all from first principles. In other systems, like F*, such effects are built-in and axiomatised. In particular, F* axiomatises effects for divergence, state, exceptions, some of their combinations, and computationally irrelevant ghost code. While we have discussed state, exceptions and divergence quite extensively, we have not considered ghost code so far. In this section we will show how to define and reason about it from first principles completely within Lean ecosystem.

4.7.1 Ghost Type

Ghost code — also known as *auxiliary* or *specification-only* code — is program text written purely for the sake of a proof [39, 100]. A typical example is an auxiliary variable

recording information that the argument needs but the program does not: the history of operations performed so far, the number of loop iterations executed, or a snapshot of an earlier state. The idea goes back to the auxiliary variables of Owicki and Gries [100], and dedicated `ghost` declarations are a staple of modern deductive verifiers such as Dafny [72] and Why3 [38]; F* exposes the same idea as the primitive, axiomatised `Ghost/GTot` effect (Sec. 2.3.4). What makes code "ghost" is the guarantee of *non-interference*, articulated by Filliâtre et al. [39]: ghost data may guide a proof but must never influence the computation. It therefore has to be *erasable* — the compiled program must behave exactly as if the ghost code were not there — and if computationally relevant data ever leaks from a ghost value into ordinary code, the verifier should reject the program.

Lean has no built-in ghost effect, so we must build one. The lever we use is that Lean's compiler *erases* every term of type `Prop`: this is sound precisely because `Prop` enjoys proof irrelevance and admits no large elimination into `Type`, so propositional content can never be observed by a running program.

A first, naive attempt is to define Ghost as Nonempty α. Since Nonempty α : Prop, it will be erased by Lean compiler. The trouble is that Nonempty α is itself a proposition: by proof irrelevance all its inhabitants are equal, so no proposition can tell two ghost values apart. We could neither state nor prove anything about *which* value a ghost holds — the data is lost. We want erasure *without* collapsing distinctness.

The mechanism that achieves this is Lean's `noncomputable` marker. Lean requires this annotation on all computationally irrelevant definitions by construction — most notably anything built with the classical axiom of choice (`Classical.choice`). Lean cannot compile such terms: the axiom of choice has no implementation, it might appear in proofs but can never be executed. The marker therefore propagates: any definition that could ever invoke a noncomputable one must itself be marked `noncomputable` and is excluded from compilation. This is exactly the guard we are after — if a ghost value can be projected back to its underlying α only *noncomputably*, then it can never feed a real computation.

Fig. 4.9 sketches the construction. We represent a ghost value as a *singleton subset* of α: the structure on lines 3–5 packages a predicate toSet : $\alpha \to$ Prop (line 4) together with a proof exists_unique that this predicate holds of exactly one element (line 5). The constructor Ghost.mk x (line 7) takes the singleton predicate $(\cdot = x)$, so distinct values stay distinguishable — Ghost.mk x = Ghost.mk $y \iff x = y$, and a proposition over Ghost α can refer to the represented element. This is the crucial difference from Nonempty α. Recovering the element, Ghost.reveal : Ghost $\alpha \to \alpha$ (line 9), must *choose* the unique inhabitant from the exists_unique proof via `Classical.choice` and is therefore `noncomputable`. In contrast, Ghost.modify (line 12) transforms the represented element by a function `f` *without* revealing it: it maps the underlying singleton $\{y\}$ to the singleton $\{z \mid \exists y,\ \text{toSet}\ y \wedge z = f\ y\} = \{f\ y\}$, operating on the predicate directly, and hence stays computable.

The only data field of Ghost α is toSet : $\alpha \to$ Prop (line 4), a function whose results are propositions, and the accompanying exists_unique witness (line 5) is itself a `Prop`. The compiler therefore erases both: a ghost value carries no run-time data and compiles to a

```
1  def Set (α : Type u) := α → Prop
2
3  structure Ghost (α : Type u) where
4    toSet         : Set α
5    exists_unique : ∃! x, toSet x   -- holds of exactly one element
6
7  def Ghost.mk (x : α) : Ghost α := ⟨fun y => y = x, ···⟩
8
9  noncomputable def Ghost.reveal (s : Ghost α) : α :=
10   s.exists_unique.choose
11
12 def Ghost.modify (s : Ghost α) (f : α → α) : Ghost α :=
13   ⟨fun z => ∃ y, s.toSet y ∧ z = f y, ···⟩
14
15 instance : Monad Ghost where
16   pure x   := Ghost.mk x
17   bind s f := ⟨fun z => ∃ y, s.toSet y ∧ (f y).toSet z, ···⟩
18
19 noncomputable instance : WPMonad Ghost Prop EPost⟨⟩ where
20   wpTrans s := ⟨fun post epost => post s.reveal⟩
21   -- laws: via reveal (mk x) = x
```

Figure 4.9: The `Ghost` type, its operations, its monad instance, and its `WPMonad` instance. Proofs are elided with $\cdots$.

trivial representation. The one operation that could produce an actual α, reveal (line 9), is `noncomputable`, so ghost data provably cannot leak into a computation.

`Ghost` is, moreover, a monad (lines 15–17). The maps mk and reveal witness an isomorphism Ghost $\alpha \cong \alpha$, so Ghost is the identity monad in disguise. Its unit is pure $x \triangleq$ Ghost.mk x (line 16), and its bind is the singleton of the composite, bind $s\ f \triangleq \{z \mid \exists y,\ s\ y \wedge (f\ y)\ z\}$ (line 17). Crucially, both pure and bind are defined directly on the underlying singleton predicate — like Ghost.modify — rather than through the noncomputable reveal, so they stay *computable* and the enclosing code remains compilable (with the ghost parts erased).

Finally, `Ghost` comes with a `WPMonad` instance of its own (lines 19–20). Since Ghost $\alpha \cong \alpha$, a ghost computation is really just a pure value, and its weakest precondition says exactly that: the normal lattice is $Pred = \mathsf{Prop}$, the exceptional one is trivial ($EPred = \mathsf{EPost}\langle\rangle$, as Ghost raises nothing), and

$$\mathbf{wp}_{\mathsf{Ghost}}\ s\ post\ epost \triangleq post\ (\mathsf{reveal}\ s).$$

To get its hands on the underlying value, the instance has to call reveal (line 20), so it is `noncomputable` itself — which is harmless, as weakest preconditions live only inside specifications and proofs, never in compiled code. The lemmas reveal (mk x) $= x$ and mk (reveal s) $= s$ then make $\mathbf{wp}_{\mathsf{Ghost}}$ coincide with the weakest precondition of the identity monad: reasoning about a ghost computation is just reasoning about the plain value it represents.

With all of this in place, ghost code can be dropped into ordinary programs wherever it is useful. Here is the simplest possible host — a pure computation in the identity monad IdM (`Id` in Lean) that increments its input — carrying two ghost values that mirror what it computes:

```
1  structure GhostStateT (σ : Type) (m : Type → Type) (α : Type) where
2    run : StateT (Ghost σ) m α
3
4  def GhostStateT.modify [Monad m] (f : σ → σ) : GhostStateT σ m Unit :=
5    ⟨fun s => pure ((), s.modify f)⟩
6
7  noncomputable instance [Monad m] [CompleteLattice Pred] [CompleteLattice EPred]
8      [WPMonad m Pred EPred] : WPMonad (GhostStateT σ m) (σ → Pred) EPred where
9    wpTrans x := ⟨fun post epost s =>
10     wp (x.run.run (Ghost.mk s)) (fun (a, s') => post a s'.reveal) epost⟩
11   -- laws from those of m
```

Figure 4.10: The `GhostStateT` transformer: its definition, ghost-state modification, and its `WPMonad` instance (laws elided).

```
def incr (n : Nat) : Id Nat := do
  let x : Ghost Nat := Ghost.mk n                  -- ghost: record the input
  let y : Ghost Nat := x.modify (fun v => v + 1)   -- ghost: mirror the increment
  pure (n + 1)
```

The ghost values are manipulated either through modify, as above, or through the `Monad Ghost` instance — we could just as well have written `let y : Ghost Nat := do let v ← x; pure (v + 1)`. Both forms are computable, so `incr` compiles as usual, and the ghost values remain available for reasoning: with $\mathbf{wp}_{\mathsf{Ghost}}$ one can prove, for instance, that reveal $y = n + 1$. The compiler, meanwhile, simply drops both ghost lines — the emitted code is exactly `pure (n + 1)`, as if only the non-ghost part had ever been written.

4.7.2 *Ghost State Monad*

A particularly useful application of ghost data is *ghost state*: a piece of state that a program may *update* but never *observe*. This is exactly what is needed to instrument a program with quantities that matter only for reasoning and not for execution — a step counter when reasoning about *complexity* and *cost*, or a security label when reasoning about *information-flow security*, as in Storm [70]. Since the state is ghost, it is erased at run time, so the instrumented program executes exactly as fast as the uninstrumented one while still admitting Hoare-style reasoning about the instrumented quantity.

We package this as a thin wrapper around StateT over a ghost state; Fig. 4.10 shows the construction. GhostStateT σ m α is a single-field structure (lines 1–2) whose `run` field has type StateT (Ghost σ) m α. Wrapping it in a fresh structure lets us decide which operations to expose — and the whole point lies in what we choose to leave out. The one operation we do expose is state modification, GhostStateT.modify (line 4), which applies a function $\sigma \to \sigma$ to the state. Built directly on top of Ghost.modify, it stays *computable* — matching the intent precisely: a program may *change* the ghost state, but it has no way to *read* it back into the computation. There is deliberately no get: reading the state would require reveal, which is noncomputable and would make the whole program non-runnable.

In proofs, however, we do want to talk about the state directly. In order to achieve that, we give GhostStateT σ m a `WPMonad` instance (lines 7–10) whose normal assertion language is $\sigma \to Pred$ — predicates over the *bare* state σ, not over Ghost σ — so that pre- and postconditions read exactly like those of an ordinary state monad. Under the hood, the

```
def linearSearchIdx? (p : α → Bool) (xs : Array α) :
    GhostStateT Nat Id (Option Nat) :=
  let rec loop (i : Nat) : GhostStateT Nat Id (Option Nat) := do
    if h : i < xs.size then
      modify (fun c => c + 1)        -- bump the ghost step counter
      if p xs[i] then pure (some i) else loop (i + 1)
    else pure none
  loop 0
```

Figure 4.11: Linear search instrumented with a ghost step counter

instance wraps the initial bare state with Ghost.mk and reads the final one back with reveal (line 10). As before, the use of reveal makes the instance `noncomputable` (line 7) — and, as before, that is harmless, since it lives only in specifications.

Fig. 4.11 shows a representative use: a linear search over an array that counts the number of comparisons it performs in a ghost step counter via modify. The counter is available for complexity reasoning — one can prove that the final count is bounded by `xs.size` — yet it is erased from the compiled program.

The run-time benefit is real. On an array of five million elements, the ghost-state version of Fig. 4.11 runs in ≈315 ms — indistinguishable from a version carrying no counter at all — whereas replacing GhostStateT with an ordinary StateT raises the time to ≈570 ms. Ghost state therefore matters in practice: it lets us thread reasoning-only state through executable Lean programs at essentially no run-time cost.

4.8 Summary

This chapter developed the foundations of Loom. We identified predicate transformers PredTrans *Pred EPred* as the shape of specification monads that supports both efficient VC generation and a natural notion of Hoare triples, and defined what it means for a monad — or a monad transformer — to be equipped with a weakest-precondition transformer over complete lattices of assertions. The defining property of this design is that a Hoare triple $pre \leq \mathbf{wp}\ x\ post\ epost$ compares *preconditions* rather than specifications: unlike the Dijkstra-monad triples of Sec. 2.4, unfolding it quantifies over program states only, never over postconditions. First-order specifications therefore yield first-order proof obligations, and even higher-order states — such as the function- and relation-valued protocol states of Veil — are handled without encoding overhead: a property that, as Chapter 5 explains in detail, allows automation backends such as SMT to be supported without enlarging the trusted code base. This design is realised in the Lean standard library as the `WPMonad` type class, whose instances compose along arbitrary transformer stacks, with the assertion languages of composite monads inferred automatically. On top of these foundations, we introduced ordered monad algebras as a simplified interface for effects whose specifications are continuation-shaped, and gave foundational accounts of three effects that other systems axiomatise: divergence, supporting both partial- and total-correctness reasoning; non-determinism, in demonic and angelic flavours, with provably sound execution of choice operators; and erasable ghost code, enabling reasoning-only state at no run-time cost. The next chapter turns to making verification on top of these foundations efficient and automated.

5

Efficient Automation for Monadic Program Verification

Auto-active program verifiers [74] such as Dafny [72], F* [121] and Viper [90] have demonstrated that formal verification scales to problems of industrial size [13, 53, 54]. The key to their success is autonomy: human involvement is minimised by efficient SMT-based proof automation [34], and is further eased by fast, high-quality feedback whenever the automation fails — the user's job is reduced to guiding the proof search with additional assertions [74, 109]. Unfortunately, a high degree of autonomy comes with a high degree of trust. The faithfulness of such verifiers is based on two components. The first is the correctness of an ad-hoc VC generation procedure [9], whose implementation, optimised in the interest of performance, is usually far from trivial. The second is the correctness of the particular SMT solver implementation [8, 34], together with that of the translation from the verifier's internal representation of VCs into SMT-LIB [10] — a translation that is itself ad-hoc and unverified [2].

Despite the large trusted computing base (TCB), most industry-scale verification projects are built with auto-active verifiers. That is because proof-assistant-based program verification is traditionally associated with a huge manual proof overhead: famously, the functional-correctness proof of the seL4 microkernel took about 20 person-years of effort and 200,000 lines of Isabelle/HOL proof script for roughly 8,700 lines of C code [65]. Even an attempt to match the level of automation and performance of an auto-active verifier like Dafny within a proof assistant sounds like an extremely hard job: on top of performing the proof search, one would have to construct a proof justifying each step of the verifier and, in the end, have this proof certified by the kernel.

This brings us to the question we answer in this chapter: *can we match the performance of a non-foundational auto-active verifier in a proof assistant?* We answer affirmatively! The first step towards this goal has already been taken: Chapter 4 developed Loom, general foundations designed with future automation in mind. Building on Loom, we have developed `vcgen`: a high-performance tactic for reasoning about monadic Lean code. In the next chapter we will see that verification frameworks built on top of `vcgen` match the performance of non-foundational verifier such as Dafny. `vcgen` itself is implemented on top of SymM — Lean's framework for building efficient symbolic execution engines — and is integrated into the Lean standard library [33].

In this chapter, we explain how `vcgen` works by rebuilding it in stages. We start from the central definition of the weakest-precondition transformer and develop a VC gener-

ation procedure in the most naive way possible (Sec. 5.1). Each following section then identifies the main bottleneck of the implementation built so far and presents the mechanism `vcgen` uses to address it. Sec. 5.2 explains how to scale VC generation to programs with hundreds of commands; Sec. 5.3 scales it to programs with sophisticated semantics; Sec. 5.4 extends the tactic to support custom monads; Sec. 5.5 handles control flow; Sec. 5.6 explains how to efficiently dispatch the generated VCs with Lean's `grind` solver [88]; and Sec. 5.7 discusses the limitations of the current implementation, further `vcgen` features not covered in this chapter, and possible alternative designs.

5.1 Building a Simple VCGen for `Cashmere`

Chapter 4 gave us the vocabulary for specifying program correctness: a Hoare triple $\{pre\}\ x\ \{post; epost\}$ unfolds into an entailment $pre \leq \mathbf{wp}\ x\ post\ epost$ between assertions, and specification lemmas relate the weakest precondition of each primitive operation to those of its parts. What we do not have yet is automation. Given a triple about a concrete program, someone still has to symbolically execute the program by chaining those specification lemmas together, and to collect everything that does not follow from them into *verification conditions* (VCs). Such VCs should be just ordinary propositions (not mentioning **wp**), that the user or an automated solver can discharge. In this section we build such a procedure in the most naive way possible. The result is pleasantly small — the complete generator fits on a single page — and its architecture, a ladder of *strategies* run by a *worklist*, is already the architecture of the production `vcgen`: every later section of this chapter refines the code presented here without ever discarding it. Once the generator works, we measure it, watch it fail to scale, and use that failure to understand what a scalable implementation must do differently.

Throughout the section we work in the simplest setting available: the Cashmere state monad of Chapter 3, which we abbreviate

```
abbrev BankM := StateM Balance
```

For this monad, the generic interface of Chapter 4 becomes very concrete. Assertions are plain predicates over the state, $Pred = \mathsf{Balance} \to \mathsf{Prop}$, and since there are no exception layers, *EPred* is the trivial `EPost.Nil`. Better yet, the entailment order $\leq$ is pointwise implication *by definition*, and the weakest preconditions of the two primitive operations compute by `rfl`:

```
example : (P ≤ Q) = ∀ s, P s → Q s := rfl
example : wp (get : BankM Balance) post epost = fun s => post s s := rfl
example : wp (set v : BankM Unit)  post epost = fun _ => post () v := rfl
```

These definitional equalities suggest the representation that our early prototype tactic adopts. As soon as we peel off the triple, we apply everything to a concrete initial state and keep every intermediate goal in *pointwise* form: `wp prog post epost s : Prop` for a concrete state `s`. A goal of this shape is an ordinary proposition, so from this point on there is no lattice structure left to manage.

```
theorem get_spec {post : Balance → Balance → Prop} {epost : EPost.Nil}
    {s : Balance} (h : post s s) :
    wp (get : BankM Balance) post epost s := h

theorem bind_spec {x : BankM α} {f : α → BankM β} {post : β → Balance → Prop}
    {epost : EPost.Nil} {s : Balance}
    (h : wp x (fun a => wp (f a) post epost) epost s) :
    wp (x >>= f) post epost s :=
  (Spec.bind x f).le_wp s h
```

Figure 5.1: The Stage-1 specification lemmas, written directly in applicable *pointwise* form (`set_spec` and the entry rule `triple_intro` are elided).

5.1.1 *Strategies in a Loop*

Before writing any code, we should understand what the generator will actually see — and it is not the pretty `do`-notation, but the term that notation elaborates to. Our running example is `step`, adapted from the `vcgen` benchmark suite of the Lean compiler. It deposits `v` and immediately withdraws it, so it preserves any balance:

```
def step (v : Balance) : BankM Unit := do
  let b ← get
  set (b + v)
  let b ← get
  set (b - v)
```

Elaboration turns this `do`-block into a right-nested chain of applications (instance arguments elided):

```
Bind.bind MonadState.get (fun b =>
  Bind.bind (MonadState.set (b + v)) (fun _ =>
    Bind.bind MonadState.get (fun b => MonadState.set (b - v))))
```

Note the three head constants: `Bind.bind`, `MonadState.get`, and `MonadState.set`. These are the names our generator will recognize and dispatch on.

Next, we need specification lemmas for these three constants, plus an entry rule that peels the `Triple` structure. Fig. 5.1 shows the two representative ones; the lemma for `set` mirrors the one for `get`, and the entry rule `triple_intro` is nothing more than the `Triple` constructor. Instead of stating natural Hoare triples, we write each lemma directly in the form the loop will be able to `apply`: its conclusion is the pointwise weakest precondition of one program shape, and its hypothesis is the subgoal that the application leaves behind. For `get` and `set`, the `rfl`-equations above make the hypothesis and the conclusion the *same* proposition, so the proof is literally the hypothesis. The interesting rule is the one for `bind`, because it is the engine of symbolic execution. Read backwards, it turns the task "verify `x >>= f`" into the task "verify `x`, against a postcondition that carries the obligation to verify `f a`"; every step of a run peels off one `bind` in exactly this way. Its proof goes through the standard library's `Spec.bind` triple, applied pointwise via its `le_wp` field. Writing the lemmas in applicable form by hand is Stage 1's one real simplification. The production tactic lets users state specifications as natural triples and compiles them into applicable rules on demand — a mechanism we reconstruct in Sec. 5.4.

With the lemmas in place, the generator itself fits in Fig. 5.2, and it is worth reading in full: every later stage of this chapter reuses its shape. The heart of it is `solveStep`, a single-step

```
def specTable : Name → Option Name          -- Stage-1 stand-in for the spec database
  | ``Bind.bind        => some ``bind_spec
  | ``MonadState.get   => some ``get_spec
  | ``MonadState.set   => some ``set_spec
  | _                  => none

inductive StepResult where
  | goals (subgoals : List MVarId)          -- the goal decomposed
  | vc    (goal : MVarId)                   -- no strategy applies: residual VC

/-- Strategy 1: unfold a Hoare triple into its pointwise form --
    apply `triple_intro` backwards, then introduce `s` and `hP`. -/
def tripleIntro? (goal : MVarId) (target : Expr) : MetaM (Option (List MVarId)) := ...

/-- Strategy 2: the `WPInfo` view of a pointwise goal `wp prog post epost s...`. -/
structure WPInfo where (head : Expr) (args : Array Expr) where ...

def WPInfo.prog (info : WPInfo) : Expr := info.args[7]!   -- the program sits at index 7

def getWPInfo? (target : Expr) : Option WPInfo := ...

/-- Strategy 3: dispatch on the program's head constant and apply the spec. -/
def applySpec? (goal : MVarId) (info : WPInfo) : MetaM (Option (List MVarId)) := do
  let some fname := info.prog.getAppFn.constName? | return none
  let some spec  := specTable fname | return none
  return some (← goal.apply spec)

/-- One VC-generation step: try each strategy in order. -/
def solveStep (goal : MVarId) : MetaM StepResult := goal.withContext do
  let target ← instantiateMVars (← goal.getType)
  if let some gs ← tripleIntro? goal target then return .goals gs
  let some info := getWPInfo? target | return .vc goal
  if let some gs ← applySpec? goal info then return .goals gs
  return .vc goal

/-- The worklist driver: run `solveStep` to exhaustion, collect the VCs. -/
def work (goal : MVarId) : MetaM (List MVarId) := do
  let mut worklist : Array MVarId := #[goal]
  let mut vcs : Array MVarId := #[]
  while let some g := worklist.back? do
    worklist := worklist.pop
    match ← solveStep g with
    | .goals subgoals => worklist := worklist ++ subgoals.toArray.reverse
    | .vc g           => vcs := vcs.push g
  return vcs.toList

elab "cash_vcgen" : tactic => liftMetaTactic work
```

Figure 5.2: The complete Stage-1 VC generator (the bodies of the first two strategies are elided).

classifier that tries an ordered ladder of *strategies* on the current goal:

- `tripleIntro?` recognizes a `Triple` goal, applies the entry rule backwards, and introduces the initial state `s` together with the precondition hypothesis `hP`. The goal is now pointwise.
- `getWPInfo?` splits a pointwise goal `wp prog post epost s` into the `WPInfo` view: a structure assembling some **wp**-specific parameters, which will come handy for further strategies. All it needs to know is that `wp` takes ten core arguments, that the program sits at a fixed position among them, and that any remaining arguments are the states the goal applies it to — for `BankM` exactly one, the current balance; richer monads will add more (Sec. 5.4).
- `applySpec?` reads off the program's head constant, looks it up in the specification table, and applies the matching lemma. The lemma's hypothesis becomes the next goal on the worklist.

If no strategy recognizes the goal, `solveStep` declares it a residual VC. This is the central contract of the design, and it is worth stating explicitly: the generator never promises to prove anything — it only promises to either decompose a goal or hand it back. Program constructs the generator does not (yet) support therefore degrade gracefully, into proof obligations for the user rather than into errors. Around the classifier, `work` runs a last-in-first-out worklist until no goals remain. For the straight-line programs of this section the worklist never holds more than one goal, but we keep the general shape on purpose: control-flow splits, which return several subgoals at once (Sec. 5.5), and the solver integration of Sec. 5.6 will later slot into it unchanged.

5.1.2 *The Cost of `MetaM`*

Before measuring the generator, let us ask how big a realistic input actually is. Consider how programs reach a VC generator in practice: an auto-active verifier in the style of Velvet (Chapter 6) translates every statement of an imperative function into a monadic bind, so a function of industrial size — a few hundred lines — arrives at the generator as a chain of hundreds of sequenced operations. Handling hundreds of commands is therefore not a stress test; it is the size the tool must handle routinely to be useful at all. Our benchmark family measures exactly this dimension. The program `steps n` chains $n+1$ copies of `step`, and the goal states that the whole chain preserves an arbitrary postcondition:

```
def steps : Nat → BankM Unit
  | 0     => step 0
  | n + 1 => do step (n + 1); steps n
```

$$\mathsf{Goal}\ n \triangleq \forall post,\ \{post\}\ \mathsf{steps}\ n\ \{\lambda res.\ post\}\,.$$

Verifying Goal n takes roughly $8n$ rule applications — about 1,600 at $n = 200$ — since each `step` contributes four state operations plus the binds that sequence them. We deliberately keep the verification content trivial: the program obviously preserves any balance, and each run emits exactly one residual VC, $post\ s \vdash post\ (\ldots(s+n-n)\ldots)$, closed by rewriting with add-sub-cancellation. This way the measurement isolates the generator itself, not the difficulty of the proof. One reporting convention, used throughout the

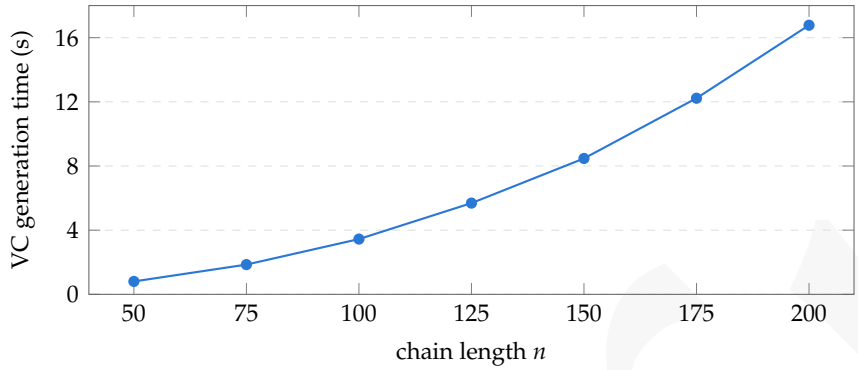

Figure 5.3: VC generation time of the Stage-1 generator on Goal n.

chapter: we always time three phases separately — *generation* (the tactic loop itself), *discharge* (proving the residual VCs), and *kernel* (certifying the final composite proof term) — because, as we will see repeatedly, the three tell different stories.

Fig. 5.3 shows the result, and it is not encouraging. Generation takes 0.8 s at $n = 50$, 3.4 s at 100, and 16.8 s at 200: every doubling of the program multiplies the time by four to five, so the loop is quadratic in the program length. Extrapolated to a thousand-step chain — the size we just argued is routine — it would take over seven minutes. The other two phases are negligible (0.10 s of discharge and 0.04 s of kernel time at $n = 200$), so the generator alone is responsible for over 99% of the cost. To see how much room for improvement there is: the production `vcgen` closes the same Goal 200 in 25 ms, and it grows linearly. Our loop is roughly 700× away. The rest of this chapter is about closing that gap — without changing the architecture we just built.

So where does the time go? The problem is that every iteration of the loop performs work proportional to the size of the whole goal, rather than to the small change the step actually makes — and almost all of that work happens inside one call, the lemma-application primitive `goal.apply` (line 26 of Fig. 5.2): a sampling profile of the $n = 200$ run places 97% of the tactic's time inside it. To find the component at fault, let us walk through what `MVarId.apply` does.

1. **Preprocess the lemma.** `apply` first infers the type of the lemma and inspects its shape. Specification lemmas are small, so this is cheap — although the loop repeats it at every step, for the same three lemmas of our vocabulary.

2. **Preprocess the target.** Next, `apply` must decide how many of the lemma's leading binders to instantiate, and for that it needs to know the shape of the goal. The goal may itself hide ∀-binders behind definitions, so `apply` counts them by reducing *the entire goal* to weak-head normal form with $\delta\beta$-transparency. On a pointwise goal this is very expensive. The reduction does not stop at the head: to normalize it, the walk has to traverse the whole term — including the hundreds of not-yet-processed `step` iterations that make up the rest of the program — allocating fresh nodes along the way, only to produce a single number, which is immediately discarded. Worse, none of this work is ever reused. The goals of consecutive steps agree on almost all of their structure — that same not-yet-processed rest of the program — but each goal is a freshly allocated tree, so the shared structure is never the same object in

memory: `MetaM`'s caches are consulted at every level and never hit, and the cache lookups themselves have to hash and compare large keys structurally.

3. **Unify and finish.** Only now does `apply` do what we asked: it instantiates the lemma's binders with fresh metavariables, unifies its conclusion against the target, synthesizes the typeclass arguments, assigns the goal, and returns the still-unassigned metavariables as subgoals. The unification step deserves a warning of its own. The general-purpose `isDefEq` checks *definitional* equality: it walks two terms structurally, side by side, unfolding definitions whenever it gets stuck. And this is neccessary — nothing guarantees that equal terms are the same object in memory, so pointer identity proves nothing and every comparison is a potential deep traversal with unfoldings.

Thus each step costs $O(n)$ and, since, there are about $8n$ steps, we are observing a quadratic behaviour. This is not specific to our loop: de Moura reports the same shape for a `MetaM`-based symbolic-execution baseline: 88 ms for a ten-step program and 2.1 s for a hundred steps. This is expected: `apply` has to measure the target because it knows nothing about the lemma it applies, and the measurement is expensive because nothing about the goal is shared or remembered between steps. The fix is to compile each rule only once, so that its shape is known and nothing needs to be measured at application time, and to make sharing a property the goals do not lose. Luckily, de Moura has already implemented that in SymM.

5.2 SymM: A Framework for Scalable Symbolic Execution Engines

We have concluded the previous section with three costs, concluding that none of them can be fixed by tuning the loop. This section introduces the remedy that fixes them: SymM (`Lean/Meta/Sym/` in the Lean sources), Lean's framework for building scalable symbolic execution engines, and the framework the production `vcgen` actually runs on. There is no mystery to the monad itself — `SymM` is simply a reader and state layer over `MetaM`, so any `MetaM` action lifts into it. All of the value is in what that state carries: a *sharing table* that keeps every expression in play maximally shared, and a family of caches that this sharing makes sound. We present the mechanisms one by one, then, almost character for character, port our loop and re-run the benchmark.

5.2.1 *Maximal Sharing via Hash-Consing*

The heart of SymM is hash-consing. The session state carries a sharing table into which every expression node is *interned*: when two structurally equal terms enter the table, they leave it as the same object in memory. Interning is cheap — each node hashes its children *by pointer*, so the cost is constant per node — and it is alpha-aware, so terms differing only in bound-variable names are identified as well.

What does this buy? For everything the session has interned, two things become true at once. First, structural equality *is* pointer equality: the deep $O(n)$ walks of Sec. 5.1.2 collapse into single pointer comparisons (`isSameExpr`). Second — and this is the property that changes the complexity class — the parts of the goal a step does *not* touch keep

their pointers from one step to the next. When our loop peels one `bind` off a chain of two hundred, the remaining hundred and ninety-nine re-enter the next goal as the same objects they were before. Per-step cost starts tracking what the step *changed*, not how big the goal is. This is precisely removing the sharing-related cost in the diagnosis.

Entering this world takes a ritual: `preprocessMVar` takes an ordinary goal and interns it wholesale. Preprocessing also unfolds all `reducible` definitions once and folds kernel projections, establishing an invariant that the hot path never needs delta or projection reduction — every comparison the loop makes from then on is purely syntactic.

5.2.2 Pointer-Keyed Caches

Once pointers are canonical, they become excellent cache keys. The SymM state carries pointer-keyed caches for type inference, universe levels, congruence information, and definitional-equality verdicts: facts that `MetaM` re-derives on demand are computed once per *object*, and since unchanged subterms keep their objects across steps, once per object now means once per run rather than once per step.

One of these caches deserves a closer look, because it disposes of the most stubborn fixed cost from the diagnosis. Every metavariable assignment must check that the assigned term makes sense in the metavariable's scope; `MetaM` performs this check by comparing local contexts, which is itself linear. SymM instead caches, per term, the newest free variable occurring in it — so the scope check becomes one lookup and one index comparison. The cache is sound for a reason worth remembering: during symbolic execution the local context only ever *grows*, so a term that was in scope once stays in scope forever.

5.2.3 Compiled Backward Rules

The remaining cost is the per-step lemma elaboration and general unification. SymM's answer is to compile each specification lemma *once*, ahead of time, into a `BackwardRule`: a unification `Pattern` for the lemma's conclusion — preprocessed for matching, with argument roles precomputed and instance and proof arguments marked — together with the positions of the premises that become subgoals. Applying a compiled rule (`BackwardRule.apply`) is then mostly a syntactic affair. Matching proceeds pointer-first, falling back to a cheap structural comparison rather than full `isDefEq`; nothing is unfolded on the hot path (the preprocessing invariant guarantees nothing needs to be); and fresh metavariables are avoided whenever an argument can be read off the goal directly. Stage 1's per-step `apply` is gone entirely.

5.2.4 The Same Loop on *SymM*

Porting our prototype is really mechanical. The strategies and the worklist are metaprograms, but since `SymM` embeds `MetaM`, their code carries over almost line for line — the same definitions, now running in the richer monad. The head-constant dispatch survives too, with its table now mapping to compiled rules instead of lemma names. And `WPInfo` with its destructuring is reused from Stage 1 outright: it is a pure function on expressions, and it could not care less whether the expressions are shared. Fig. 5.4 shows everything that changes: the rules are compiled once at the top of `work` instead of elabo-

```
def mkSpecRules : MetaM SpecRules := do            -- compiled once per run
  return { tripleIntro := ← mkBackwardRuleFromDecl ``triple_intro
           bind        := ← mkBackwardRuleFromDecl ``bind_spec
           ... }

def applySpec? (rules : SpecRules) (goal : MVarId) (info : WPInfo) :
    SymM (Option (List MVarId)) := do
  let some fname := info.prog.getAppFn.constName? | return none
  let some rule  := rules.find? fname | return none
  let .goals gs ← rule.apply goal | return none    -- was: goal.apply spec
  return some gs

def solveStep (rules : SpecRules) (goal : MVarId) : SymM StepResult :=
  goal.withContext do
    let target ← instantiateMVarsS (← goal.getType)  -- was: instantiateMVars
    ...                                              -- the three strategies, unchanged

def work (goal : MVarId) : SymM (List MVarId) := do
  let rules ← mkSpecRules
  let goal  ← preprocessMVar goal                    -- enter the shared world
  ...                                                -- the worklist, unchanged
```

Figure 5.4: The Stage-2 port, showing only what changed relative to Fig. 5.2.

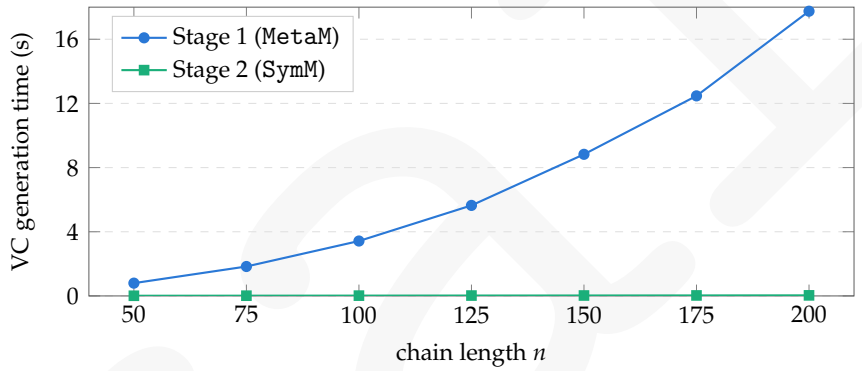

Figure 5.5: The engine swap: Stage-1 vs. Stage-2 VC generation time on the Goal n family.

rated at every step; `preprocessMVar` moves the incoming goal into the sharing table; and `instantiateMVarsS` re-interns whatever it instantiates. Everything else — the strategies, the dispatch, the worklist — carries over verbatim.

Fig. 5.5 re-runs the benchmark of Sec. 5.1.2 on both generators. The quadratic became linear: Stage 2 takes 10 ms at $n = 50$ and 35 ms at $n = 200$, doubling when the program doubles — its series runs nearly flat along the baseline of the chart — and the same goal that took Stage 1 seventeen seconds now takes 35 ms: a factor of roughly 520, and one that keeps growing with n.

5.2.5 *Diving Deep*

A skeptical reader might rightfully note that our example is overly simplistic: the computations we verify manipulate a single piece of state and nothing else. What happens when the number of effects grows? To answer this, we reinstantiated the generator for an artificially complex monad — the same balance, buried under a stack of eight monad transformers:

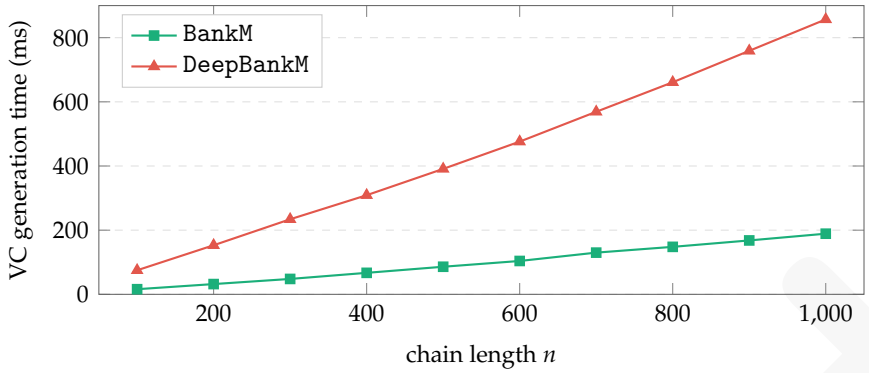

Figure 5.6: Stage-2 VC generation time for the same chain of operations in the plain `BankM` vs. the eight-transformer `DeepBankM`.

```
abbrev DeepBankM :=
  ExceptT String <| ReaderT String <| OptionT <| ExceptT Nat <|
  ReaderT Nat <| OptionT <| ExceptT Unit <| StateT Balance <| Option
```

The results are devastating. As Fig. 5.6 shows, eight layers instead of one cost a stable extra factor of about 4.5, at every program size. Nothing about the programs changed; only their *types* got bigger. Some per-step cost is still proportional to the size of the types involved — which is exactly the kind of cost sharing was supposed to rule out.

5.3 Maximal Sharing Practices

The result in Fig. 5.6 is surprising. The programs behind the two series have the same length, and the loop performs the same number of steps and applies the same four rules; the only thing that differs is the monad the programs are written in. In this section we locate the extra cost by profiling, remove it with a one-line change, and formulate the general lesson behind it: sharing is a property of a *session*, not of a term. We will then meet the same lesson again when we extend the generator with `let`-expressions.

5.3.1 Sharing Is a Session Property

Profiling the Stage-2 loop gives a clear answer: about 97% of the time is spent inside `BackwardRule.apply`. What does rule application spend this time on? On matching the rule's `Pattern` against the goal. The pattern of, say, `bind_spec` ends in a `wp` application whose arguments include the full *instance telescope*: the `WP` and `Assertion` instances and the monad itself. These are closed terms, and structurally identical copies of them appear in every goal the loop will ever see. The problem is that our rules were compiled in `MetaM`, outside the session, so their patterns were never interned into the sharing table — and the table knows nothing about structural equality; it only knows the terms it has interned. As a result, the pattern's copy of the telescope is a different object from the goal's copy: the pointer comparison fails, and every match falls back to the structural walk that SymM was introduced to avoid.

The fix is a single line per rule, applied when the rule is compiled — that is, it wraps each of the `mkBackwardRuleFromDecl` calls on lines 2–4 of Fig. 5.4:

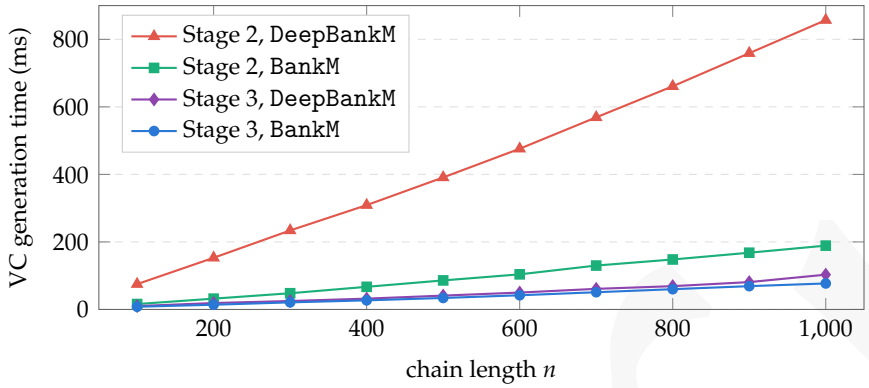

Figure 5.7: VC generation time before (Stage 2) and after (Stage 3) interning the rule patterns.

```
let rule ← mkBackwardRuleFromDecl n
return { rule with pattern := ← rule.pattern.shareCommon }
```

Now the pattern and the goal live in the same table, and the telescope comparison reduces to one pointer check.

Fig. 5.7 shows the effect of this change. The depth penalty drops from the stable 4.5× to about 1.3× (77 vs. 103 ms at $n = 1000$), and the shallow loop itself becomes roughly 2.5× faster: the structural walk grew with the stack, while the pointer comparison does not depend on it. It is useful to summarize the three stages so far. Stage 1 shared nothing and paid per goal size; Stage 2 shared the goals and became linear; Stage 3 shared the last unshared term in the loop, so that each step now costs in proportion to what it changes.

5.3.2 *Handling `let`-Bindings, and the `S`-Builders*

With sharing in order, we can extend the generator with a new feature. So far every program was a plain chain of binds. Real programs, however, introduce local names: a pure `let d := b + v` inside a `do` block survives elaboration as a genuine `let`-expression in the middle of the program. Fig. 5.8 shows the strategy that handles such programs, in two versions that differ only in how they build terms; read the left one first. When the program under `wp` starts with a `let` (line 3), there are two cases. If the bound value is cheap to duplicate — a variable, a constant, or a literal (line 6) — it is substituted into the body (line 7), and the rebuilt program is spliced back into the goal (lines 8–11). Otherwise the `let` is *hoisted*: the goal `wp (let x := v; body) ...` is replaced by the definitionally equal `let x := v; wp body ...` (lines 13–17), and the binder is introduced into the local context (line 18) — the value `v` is now stated only once, no matter how many times `x` is used in the rest of the program.

The left version is correct — and it would quietly ruin the loop. `wpLet?` is the first piece of our own code that constructs new expressions and inserts them into the goal that all later steps will traverse; built with the ordinary constructors, the new nodes are not interned, and every step after a `let` falls off the pointer fast path — the same problem we have just fixed. The version on the right therefore replaces each ordinary builder with its sharing-preserving SymM counterpart, which interns every term it creates: `instantiateRevBetaS`

```
def wpLet? (goal : MVarId) (i : WPInfo) :
    MetaM (Option MVarId) := do
  let .letE x t v body nd := i.prog.getAppFn
    | return none
  let args := i.prog.getAppRevArgs
  if isDuplicable v then
    let body := (body.instantiate1 v).headBeta
    let prog := mkAppRev body args
    let wpArgs := i.args.set! 7 prog
    let tgt  := mkAppN i.head wpArgs
    return some (← goal.replaceTargetDefEq tgt)
  else
    let prog := mkAppRev body args
    let wpArgs := i.args.set! 7 prog
    let wp   := mkAppN i.head wpArgs
    let tgt  := Expr.letE x t v wp nd
    let goal ← goal.replaceTargetDefEq tgt
    let (_, goal) ← goal.intro1
    return some goal
```

(a) With the ordinary `MetaM` builders.

```
def wpLet? (goal : MVarId) (i : WPInfo) :
    SymM (Option MVarId) := do
  let .letE x t v body nd := i.prog.getAppFn
    | return none
  let args := i.prog.getAppRevArgs
  if isDuplicable v then
    let body ← instantiateRevBetaS body #[v]
    let prog ← mkAppRevS body args
    let wpArgs := i.args.set! 7 prog
    let tgt  ← mkAppNS i.head wpArgs
    return some (← goal.replaceTargetDefEq tgt)
  else
    let prog ← mkAppRevS body args
    let wpArgs := i.args.set! 7 prog
    let wp   ← mkAppNS i.head wpArgs
    let tgt  := Expr.letE x t v wp nd
    let goal ← goal.replaceTargetDefEq tgt
    let .goal _ goal ← Sym.intros goal | failure
    return some goal
```

(b) With the sharing-preserving `S`-builders.

Figure 5.8: The `let` strategy, implemented with ordinary `MetaM` builders (left) and with SymM's sharing-preserving builders (right).

for the substitution (line 7), `mkAppRevS` and `mkAppNS` for the applications (lines 8, 10, 13 and 15), and `Sym.intros` for the introduction (line 18). The general rule is simple: if the inputs are shared, the outputs must be kept shared as well.

5.4 A Specification Database, and Rules Constructed on Demand

Our generator is still tied to one monad. The specification lemmas of Sec. 5.1 were written by hand, in pointwise form, specifically for `BankM`; the deep-stack experiments needed their own copies for `DeepBankM`; and the table mapping head constants to rules is hard-coded. The production `vcgen` has none of these restrictions: it works for any monad with a `WPMonad` instance, and users state their specifications as ordinary Hoare triples, registered with the `@[spec]` attribute. In this section we adopt its mechanism.

The mechanism also makes explicit a division of labour that the previous sections only hinted at. Rule construction runs *off* the hot path: it happens once per operation, so it can afford the convenient, expensive `MetaM` machinery. Lookup and application run *on* the hot path, once per step: there everything must stay interned and pointer-keyed. Much of this section is about keeping each piece of code on the right side of this line.

5.4.1 *The `@[spec]` Database*

The database itself is built from a standard piece of Lean metaprogramming infrastructure. A metaprogram can declare an *environment extension*: a named piece of global state attached to the environment, holding data of any type the metaprogram chooses, and persisted across modules like any other declaration. The variant we use, `SimpleScopedEnvExtension`, additionally respects Lean's scoping rules — a specification can be registered globally, only within a namespace, or locally in a section.

On top of one such extension we implement the `@[spec]` attribute. Applying it to a

```
def mkRuleFromSpec (thm : SpecTheorem)
    (info : WPInfo) :
    OptionT MetaM BackwardRule := do
  -- instantiate the stored triple
  let prf ← mkConstWithFreshMVarLevels
              thm.declName
  let ty ← inferType prf
  let (xs, _, ty) ← forallMetaTelescope ty
  -- normalize to the ≤-entailment form
  let (prf, ty) ←
    tripleToWpProof? (mkAppN prf xs) ty
  -- specialize to the goal's monad
  let_expr PartialOrder.rel P' _ pre rhs := ty
    | failure
  guard (← isDefEqGuarded info.Pred P')
  let_expr wp _ _ _ _ _ _
    iWP' _ post' epost' := rhs
    | failure
  guard (← isDefEqGuarded info.instWP iWP')
  guard post'.isMVar
  guard epost'.isMVar
  -- fresh metavariables for the states
  let mut ss := #[]
  for arg in info.excessArgs do
    ss := ss.push
      (← mkFreshExprMVar (← inferType arg))
  -- apply the ≤-proof to them, package up
  let concl ← mkAppM' rhs ss
  let ruleTy ← mkArrow (pre.beta ss) concl
  let prf ← mkExpectedTypeHint
              (← mkAppM' prf ss) ruleTy
  let res ← abstractMVars prf
  mkBackwardRuleFromExpr res.expr
    res.paramNames.toList
```

Figure 5.9: Rule construction: from a stored triple to an applicable `BackwardRule` (continued across the two columns).

theorem whose statement is a Hoare triple,

```
@[spec]
theorem get_spec : Triple (get : BankM Balance) (fun s => post s s) post epost := ...
```

checks that the statement indeed is a triple and stores the theorem in the extension. This is the whole specification database: passive memory, filled at declaration time. How the loop makes use of it — is the subject of the next subsection.

5.4.2 *From Natural Triples to Applicable Rules*

Unfortunately a triple from the database cannot be applied to the goal directly. Recall the shape of our working goals: `wp` applied to a concrete number of internal states — one for `BankM`, three for `DeepBankM`. A stored triple knows nothing about that number. Since VC generation must be monad-agnostic, at the attribute site we do not know which monad the specification will eventually be used with, and therefore how many internal states its proof will have to be applied to. The bridge between the two forms can only be built when the loop meets a concrete goal — the moment the number is finally known.

This is what the rule-construction algorithm in Fig. 5.9 does. We will not go through it line by line; in outline, it takes a stored triple and the `WPInfo` of the goal at hand, brings the triple's statement into the entailment form $pre \leq \mathbf{wp}\ \ldots$, specializes it to the goal's predicate type and `WP` instance, applies the resulting proof to fresh metavariables standing for the goal's internal states, and packages the outcome as a `BackwardRule` — the same compiled-rule format the loop has been applying since Sec. 5.2. In short: a generic `Triple` goes in, a `BackwardRule` specialized to the goal's monad comes out.

Note that this metaprogram seemingly violates the sharing rules postulated in Sec. 5.3. It never leaves `MetaM`, and it makes no attempt to keep the constructed terms hash-consed with the `S`-prefixed constructors of Sec. 5.3.2. More than that, it freely uses costly `MetaM` operations such as `mkAppM'`, which re-checks the application it builds and synthesizes implicit arguments from scratch. All of this is acceptable — because of caching, which we elaborate on in the next subsection.

5.4.3 *Caching: Types and Dictionaries, Never Values*

Note that `mkRuleFromSpec` transforms the general, `Triple`-based specification of `get` into precisely the shape of `get_spec` from Fig. 5.1 — and in that shape the rule can be reused across all `get` calls within the `BankM` monad. The same holds in general: once a specialized rule has been constructed for a function `f` under some monad `m`, it can be reused for every call to `f` within that monad. This suggests caching the results of `mkRuleFromSpec` — most of the time, within a single `vcgen` call, the monad does not change. And once the results are cached, we no longer need to worry about the performance of `mkRuleFromSpec` itself, and can afford the expensive `MetaM` operations like `mkAppM'`, which infer implicit arguments automatically and free us from tedious low-level bookkeeping.[1]

To implement the caching, we first extend the `SymM` monad our tactic runs in with a state holding the specification database and the cached rules:

```
structure State where
  specs     : SpecTheorems
  ruleCache : CachedTheorems

abbrev VCGenM := StateRefT State SymM
```

and wrap `mkRuleFromSpec` into a cache lookup:

```
def mkRuleFromSpecCached (thm : SpecTheorem) (info : WPInfo) :
    VCGenM (Option BackwardRule) := do
  -- a key from the spec's name and the goal's monad data
  let key := mkKey info thm
  -- if a rule for this key is already cached, simply fetch it
  if let some rule := (← get).ruleCache[key]? then return some rule
  -- otherwise construct a new one
  let some rule ← mkRuleFromSpec thm info | return none
  -- intern it
  let rule ← rule.shareCommon
  -- and store it in the cache
  modify fun s => { s with ruleCache := s.ruleCache.insert key rule }
  return some rule
```

The key records the specification's name together with the goal's `WP` instance and its number of internal states — the types and instance dictionaries of the call, never its concrete arguments — so a cached rule is found again exactly when the same operation appears in the same monad. Finally, we integrate the cached construction into the specification-lookup strategy:

```
def applySpec? (goal : MVarId) (info : WPInfo) :
    VCGenM (Option (List MVarId)) := do
  -- the specification database
  let specs := (← get).specs
  -- find a specification theorem for the current program
  let some thm ← specs.findSpecs info.prog | return none
  -- construct (or fetch from the cache) a backward rule for it
  let some rule ← mkRuleFromSpecCached thm info | return none
  -- apply the specification
```

[1] This plays an even bigger role in the production `vcgen`, where rule construction is considerably more sophisticated: it abstracts over pre- and postconditions on demand, and splits off the implications about exception postconditions. Constructing the corresponding proofs with the low-level S-prefixed builders would make the code close to unmaintainable.

```
  let .goals gs ← rule.apply goal | return none
  return gs
```

With all of this in place, the performance of the generator remains the same as in the previous section, so we do not repeat the plot — but its scope has grown considerably: we can now generate verification conditions for arbitrary monadic code featuring `let`-expressions and calls to functions with custom specifications. The next logical step is control flow.

5.5 Splitting Control Flow

In this section we briefly explain how to handle control flow, such as `if-then-else` statements, in monadic programs. Conceptually, on the implementation level, this section adds no new ideas: we will dynamically construct and cache rules for `if-then-else` statements, tailored to specific monads but abstracted in their `then` and `else` branches. The section is meant to showcase the generality of the rule-caching idea from the previous section, and to gently introduce the reader to the problem of VC dispatching.

5.5.1 Split Rules Are Constructed Rules

As a running example for this section, consider a simple monadic function that tries to increment the state, throwing an exception once a certain fixed limit is reached:

```
def step (lim : Nat) : ExceptT String (StateM Nat) Unit := do
  let s ← get
  if s > lim then
    throw "s is too large"
  else set (s + 1)
```

Once `vcgen` reaches the `if` statement in this program, it splits the goal by applying the following rule:

```
∀ (cond : Prop) [Decidable cond]
  (th el : ExceptT String (StateM Nat) Unit) post epost s,
    (cond → wp th post epost s) →
    (¬ cond → wp el post epost s) →
    wp (if cond then th else el) post epost s
```

In general, whenever `vcgen` meets a goal of the shape `wp (if cond then th else el) post epost s1 ... sn`, it should split it into the `wp`s of the two branches, introducing `cond` and its negation `¬ cond` into the respective goal contexts.

In the implementation we handle this in two steps. First,

```
def mkRuleForSplitCached (splitInfo : SplitInfo) (info : WPInfo) :
    VCGenM BackwardRule := ...
```

constructs and caches a new splitting backward rule (or simply fetches it from the cache) based on the shape of the goal (`info`) and the information about the control-flow operator at hand (`splitInfo`: a plain `if`, a dependent `if`, or a `match` statement). Then a new strategy, `wpSplit?`, calls `mkRuleForSplitCached` whenever the program it inspects starts with a control-flow operator.

To measure the new tactic, we iterate `step`:

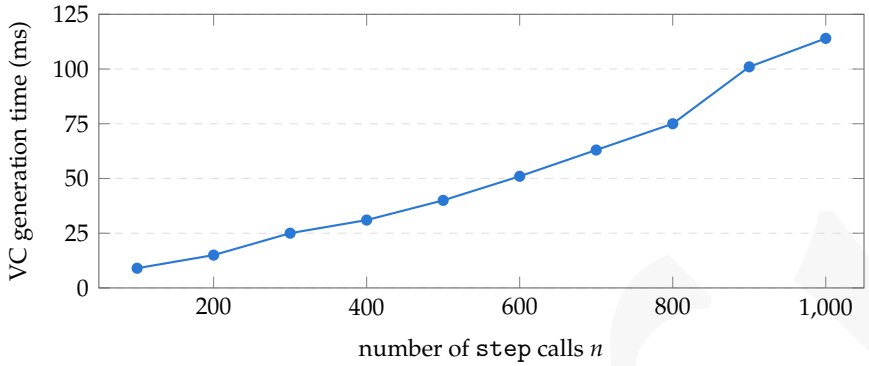

Figure 5.10: VC generation time for chains of n `step` calls.

```
s  : Nat
hP : s = 0
h0 : 0 < s
⊢ False
```

```
s  : Nat
hP : s = 0
h0 : ¬ 0 < s
h1 : 1 < s + 1
⊢ False
```

```
s  : Nat
hP : s = 0
h0 : ¬ 0 < s
h1 : ¬ 1 < s + 1
h2 : 2 < s + 1 + 1
⊢ False
```

```
s  : Nat
hP : s = 0
h0 : ¬ 0 < s
h1 : ¬ 1 < s + 1
h2 : ¬ 2 < s + 1 + 1
⊢ s + 1 + 1 + 1 = 3
```

Figure 5.11: The four verification conditions generated for Goal 3.

```
def loop (n : Nat) : ExceptT String (StateM Nat) Unit := do
  match n with
  | 0 => pure ()
  | n+1 => loop n; step n
```

$$\mathsf{Goal}\ n \triangleq \{\lambda s.\, s = 0\}\ \mathsf{loop}\ n\ \{\lambda res\ s.\, s = n\}\,.$$

Goal n states that running n steps from state 0 ends in state n: starting from zero, no guard ever fires, and every step increments the state by one. Fig. 5.10 records the performance of the new tactic on this family, and the results are very pleasing: generation stays linear, at about a tenth of a millisecond per `if` — mere milliseconds for a program with a thousand of them.

5.5.2 *Generation Is Linear; Naive Discharge Is Not*

In all the sections before, we mostly focused on measuring the VC *generation* time. Our running examples were deliberately chosen to keep every other cost negligible: running `vcgen` would produce one simple VC, easily discharged by a single call to `grind`. In this section the situation changes: the running example produces $n + 1$ verification conditions for Goal n. In particular, we have to prove that none of the `then` branches is feasible (n goals), and that the postcondition holds for the final, successfully terminating state. Fig. 5.11 shows what these goals look like for Goal 3.

Dispatching these goals naively, by calling `grind` on every residual subgoal (`vcgen <;> grind`), quickly balloons the overall execution time: as Fig. 5.12 shows, the climb is quadratic, from 0.2 s at $n = 25$ to 15.5 s at $n = 150$. Luckily, the problem is not lethal. The root cause of the overhead is that `grind` has to redo the same job over and over again. A significant part of every `grind` run is *internalisation*: before any actual reasoning, the solver eliminates the quantifiers in the goal's context and loads the resulting facts into an E-graph; only then does it try to establish that the equivalence class of `True` coincides

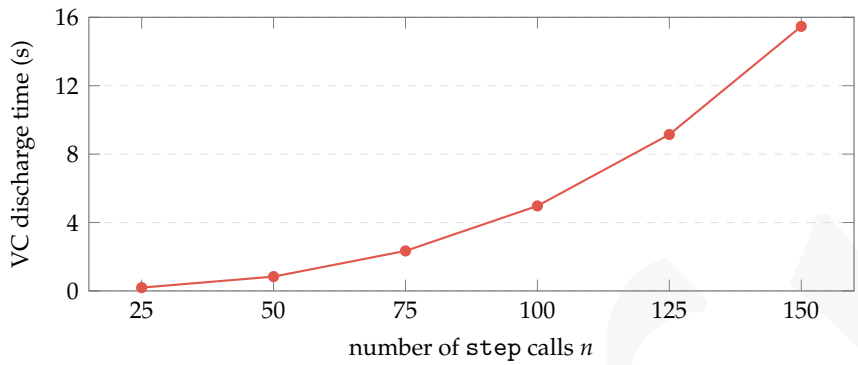

Figure 5.12: Naive discharge, with one `grind` call per generated VC, on the same chains of n `step` calls.

with that of `False`. Now look at the contexts in Fig. 5.11 again: they overlap almost entirely — each goal carries all the hypotheses of the previous ones — so a fresh `grind` call internalises the same facts over and over again. In the next section we explain how to avoid this redundancy.

5.6 Incremental Solving: the Loop Inside `GrindM`

Now that we are happy with the VC generation performance, it is time to think about handling the generated VCs. The main mechanism we exploit here is the `grind` tactic [88]: a highly efficient automation tactic in Lean, based on e-matching. There are two reasons why we settle on `grind` as the default.

First, it is designed specifically for Lean's metatheory, which means we can run it on *any* Lean goal. In contrast, the input to SMT solvers is SMT-LIB, which is effectively first-order logic — a much less expressive theory than Lean's, and one that fails to accommodate a significant share of Lean goals.

Second, it is a white box: `grind` itself is implemented entirely in Lean. As a result, it comes with an extensive and easy-to-use API — which, as we show below, is crucial for resolving the issue we faced in Sec. 5.5.2.

Of course, `grind` is not a silver bullet. For the sake of generality, stability, and performance, it deliberately omits many well-known decision procedures and heuristics: for example, it implements no decision procedure for bit-vector arithmetic and no heuristics for non-linear arithmetic. Nonetheless, in practice most of the generated goals are quite mechanical, and, as we will see in Chapter 6, `grind` dispatches them no worse than an SMT solver would.

Based on these observations, our tactic takes the following strategy: by default, we employ `grind` to discharge as many goals as we can, and hand the residual ones to whatever fits the domain — specialised automation, a manual proof, or an SMT solver. The last option deserves a comment. Goals travel to the solver through the Lean-SMT tool [86], which translates them into SMT-LIB almost verbatim — and simply gives up when a goal does not fit the first-order fragment. The same closeness of the two theories works for us on the way back: Lean-SMT reconstructs the solver's certificates as Lean proofs. What

makes this pipeline workable at all is that, in practice, most of our verification conditions are first-order to begin with — provided nothing inflates them along the way.

This is where a promise made repeatedly in the earlier chapters finally pays off. Chapter 4 went to great lengths to ensure that the foundations add as little encoding overhead to the goals as possible, and both halves of the discharge strategy now profit from that at once: the less noise a goal carries, the more `grind` can dispatch — and the goals that remain stay within the fragment the SMT translation accepts. Fig. 6.1 in the next chapter shows the two effects working together on a concrete program: `grind` discharges most of the generated VCs, and the few that remain are settled by the SMT solver — possible precisely because they stayed first-order.

To put this strategy to work, we will need to implement a highly efficient automation layer around `grind`. But first, let us study the API `grind` offers.

5.6.1 *`GrindM` Is a Stack over `SymM`*

Among Lean users, `grind` is best known as a single terminal tactic.[2] Occasionally, users add lemmas to its hint database with the `@[grind]` attribute or, more rarely, pin down the concrete pattern a lemma should fire on with the `grind_pattern` command. This, however, is only a tiny fraction of what `grind` offers. In fact, users can implement sophisticated `grind`-based automation procedures of their own on top of the `grind` monad, GrindM. The latter is a thin wrapper over SymM, extending it with `grind`-specific metainformation:

```
abbrev GrindM := ReaderT MethodsRef $ ReaderT Context $ StateRefT State SymM
```

At a high level, each layer carries some *global* — reused across multiple goals — `grind`-specific configuration and caches: the callbacks invoked when terms are internalised, the `simp` setup `grind` uses for normalisation, the generated congruence theorems, and various diagnostics. Most importantly for us, the base of the stack is SymM itself: every piece of machinery we have built since Sec. 5.2 runs inside GrindM unchanged, sharing table and all. On top of that, to store what `grind` has learned about each *specific* goal, the framework extends the notion of a Lean goal with `Grind.Goal`:

```
structure Grind.Goal where
  mvarId : MVarId                    -- Lean goals
  nextDeclIdx  : Nat := 0            -- index of next non-internalised hypothesis
  enodeMap     : ENodeMap := default -- E-graph of internalised facts
  ...
```

which is simply a data structure containing the Lean goal itself (`mvarId`), the index of the first local hypothesis that has not yet been internalised (`nextDeclIdx`),[3] the E-graph of the internalised facts (`enodeMap`), and some further bookkeeping.

Out of the whole `grind` API, we will be most interested in the following three functions (the `Grind` namespace prefix omitted):

```
mkGoal              : MVarId → GrindM Goal
```

[2] A *terminal* tactic either closes the goal completely or fails.

[3] `Grind.Goal` maintains the invariant that only a contiguous prefix of the local context is internalised, *i.e.* hypotheses are never skipped or removed.

```
1  def work (goal : MVarId) :
2      VCGenM (List MVarId) := do
3    let goal ← Grind.mkGoal goal
4    let mut worklist : Array Grind.Goal := #[goal]
5    let mut vcs : Array MVarId := #[]
6    while let some goal := worklist.back? do
7      worklist := worklist.pop
8      match ← solveStep goal.mvarId with
9      | .goals subgoals =>
10       let goal ← if subgoals.length > 1
11         then goal.internalizeAll
12         else pure goal
13     let subgoals := subgoals.toArray.map
14       fun mv => { goal with mvarId := mv }
15     worklist := worklist ++ subgoals.reverse
16   | .vc mv =>
17     let goal := { goal with mvarId := mv }
18     let goal ← goal.internalizeAll
19     match ← goal.grind with
20     | .closed => continue
21     | .failed g => vcs := vcs.push g.mvarId
22 return vcs.toList
```

Figure 5.13: The Stage-6 worklist driver, with `grind` integrated into the loop (continued across the two columns).

```
Goal.internalizeAll : Goal → GrindM Goal
Goal.grind          : Goal → GrindM GrindResult
```

The first repeats the job of `preprocessMVar` — unfolding all the abbreviations in the current goal and moving it into SymM's sharing table — and, on top of that, wraps the given metavariable into an idle `Grind.Goal` structure; we will use it to enter the GrindM world. The second internalises all not-yet-internalised hypotheses of the goal's local context into the E-graph; it will let us share the internalisation work between VCs, resolving the issue of Sec. 5.5.2. The third attempts to close the goal using `grind`, returning `.closed` on success and `.failed`, with the first subgoal it could not close, otherwise; it is what we will call to discharge each generated VC. The next subsection explains how exactly the three integrate into `vcgen`.

5.6.2 *Internalize at the Fork*

Integrating `grind` into our `vcgen` pipeline is pleasantly simple. First, we rebase the `VCGenM` monad from SymM onto GrindM:

```
abbrev VCGenM := StateRefT State GrindM
```

Since GrindM embeds SymM, every strategy we have built so far continues to work unchanged. The only other function we need to change is the worklist driver `work`; its new implementation is shown in Fig. 5.13, and three changes deserve a closer look.

First, the solving and internalisation API of `grind` is available only for the `Grind.Goal` structure, not for a bare `MVarId`, so we call `Grind.mkGoal` on the received root metavariable (line 3); as described above, it also takes over the unfolding and sharing duties of `preprocessMVar`.

Next, each time a split happens, the shared context must be internalised once for all the resulting subgoals. This is implemented on lines 10–14: whenever a call to `solveStep` returns more than one subgoal (line 10), we internalise the goal's entire not-yet-internalised context as it stood before the step (line 11), and then share it across all the new subgoals (line 14). Note that, implemented this way, the context sharing fires not only at split rules, but whenever *any* strategy spawns multiple subgoals — a pleasant bonus.

Finally, before emitting a VC, we try to close it with `grind` (lines 18–21): the goal's remaining context is internalised (line 18) and the solver is called on the spot (line 19); a closed VC never leaves the loop, and a failed one is handed back to the user exactly as `grind` left

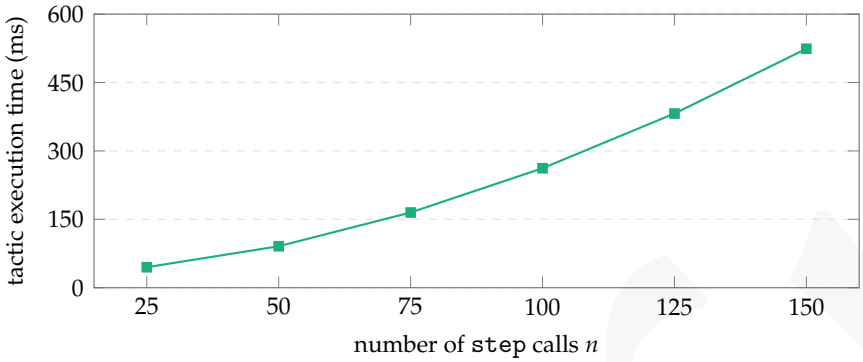

Figure 5.14: The GrindM-based vcgen on chains of n step calls, generating and solving all $n + 1$ VCs inside the loop.

it (line 21).

Fig. 5.14 shows the result of running the GrindM-based vcgen on the running example of Sec. 5.5. The new tactic is tremendously faster than the naive approach of calling grind on each subgoal independently: at Goal 150, where the naive discharge took 15.5 s (Fig. 5.12), generating *and* solving all 151 VCs now takes about half a second — a 30× speedup, at about 3.5 ms of solving per VC where the naive approach paid over 100 ms.

5.7 Production vcgen

The story, however, does not end with incremental solving: the production vcgen implements many further features, on both the performance and the robustness side. In this section we touch upon some of them — without, in the interest of space, zooming into the same level of detail — and discuss the main limitations of the framework.

5.7.1 *Simplifying Assumptions*

An observant reader might have noticed that in Sec. 5.6.2 we shortened our scale, from the usual Goal 100–Goal 1000 down to Goal 25–Goal 150. This is because at Goal 500 this version of vcgen already takes about five seconds to generate and resolve the VCs, and about twenty more seconds in the kernel. The reason is a quadratic growth in goal complexity. As Fig. 5.11 shows, each successive goal grows not only in the *number* of local hypotheses, but also in their *size*: the context of the last VC generated for Goal 500 contains 502 declarations, the last of which compares against a chain of five hundred additions, $\neg\, 499 < s + 1 + \cdots + 1$. Together, the two growths explain the quadratic execution time.

Luckily, there is a remedy. The production vcgen lets the user specify a set of simplification lemmas to be applied at every iteration of the work loop. In our case it suffices to run vcgen with the Nat.add_assoc lemma, which reassociates $s + 1 + \cdots + 1$ into $s + (1 + (\cdots + 1))$; the right-hand chain is now a closed term, which the built-in normalisation folds into a literal, leaving the compact $s + n$. With this simplification hint, dispatching Goal 1000 takes about a second of generation and solving, and around six seconds in the kernel.

5.7.2 Non-Proposition-Based Assertion Types

Recall that at every step of the `work` loop we assumed the current goal to have the form `wp prog post epost s1 ... sn`: the weakest precondition of a program `prog`, applied to its postconditions and to concrete state arguments. For such a term to be of type Prop, the assertion type *Pred* must have the shape $\sigma_1 \to \sigma_2 \to \cdots \to \mathsf{Prop}$ — a predicate over the states the monadic computation operates on. Of course, this is not always the case. For instance, we might want to work with a *polymorphic* monad stack, *i.e.* with the monad StateT σ m for an arbitrary m whose assertion language is some complete lattice *Pred*; the overall assertion type is then $\sigma \to \mathit{Pred}$. Or we might want to work with a probability monad, whose assertions are random distributions, *i.e.* real-valued functions.

In neither case can we apply `wp prog post epost` to enough state arguments to obtain a proposition and put it into a Lean goal. That is why the production `vcgen` keeps its goals in the form of an entailment `pre` $\leq$ `wp prog post epost s1 ... sn`, where both sides are applied to as many state arguments as the assertion type allows, and whatever structure remains is compared by the lattice order. In other words, the tactic introduces as many program states as possible, and keeps the rest as an entailment.

5.7.3 Other Types of Rules

On top of rules for `Triple` specifications and splitting lemmas, the production `vcgen` features many other rules, constructed and cached on demand. First, the `@[spec]` attribute can also be applied to unfoldings and other equational lemmas; this is especially convenient when the user wants to instruct `vcgen` to unfold a certain function, rather than to look up a specification for it. Second, `vcgen` handles the lattice operators, such as meet ($x \sqcap y$), pure-fact embedding ($\ulcorner \varphi \urcorner$), top ($\top$), and bottom ($\bot$). Finally, `vcgen` constructs and caches rules that split `match` statements, in the same way it does for `if`. This last feature, however, comes with a caveat, which we discuss in the next subsection.

5.7.4 Main Limitations

While `vcgen` demonstrates impressive performance on monadic code with thousands of program statements, it has one limitation related to pattern matching. Consider the following code snippet:

```
def matchStep : ExceptT String (StateM Nat) Unit := do
  let s ← get
  match s with
  | 0     => throw "s is zero"
  | n + 1 => set n
```

This function closely resembles the running example of Sec. 5.5–Sec. 5.6, and it is natural to conjecture similar performance, for both tactic execution and kernel certification times. Fig. 5.15, however, disproves this conjecture. Iterating `matchStep` into chains of n calls, exactly as we iterated `step` before (this time running the state down, from n to 0), we find that at Goal 150 the kernel needs 11.9 s to certify the `matchStep` chain — roughly 50× longer than the 0.23 s it spends on the running example (run with the `Nat.add_assoc` simplification hint).

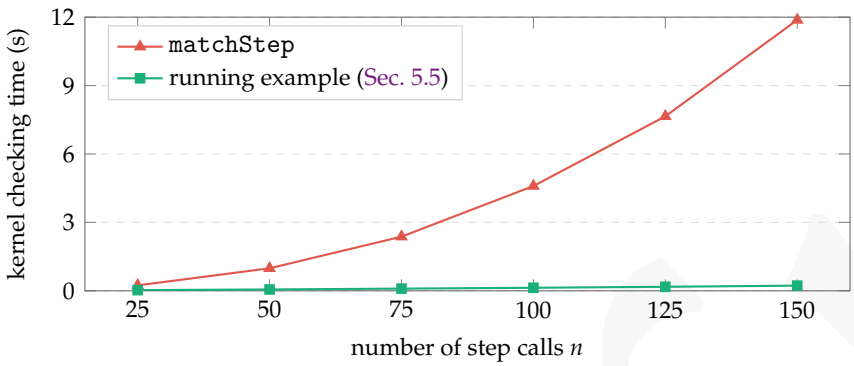

Figure 5.15: Kernel certification time for chains of n `matchStep` calls, with the running example of Sec. 5.5 as a baseline.

This is because, at each step, splitting the `match` introduces a *fresh binder*, and the resulting proof term has the following structure:

```
match.splitter fun n1 ... =>
  match.splitter fun n2 ... =>
    ...
      match.splitter fun n150 =>
        proof n1 n2 ... n150
```

where `match.splitter` is the constructed rule splitting the `match` statement and each `ni` is a new binder carrying the decremented state; since `ni` *is* the program's next state, everything below its introduction genuinely mentions it. This shape is hostile to the kernel, which checks binders by *instantiation*: bodies are stored in de Bruijn form, so to infer the type of `fun ni => body` the kernel first closes `body`, replacing every occurrence of the loose index with a fresh local constant and rebuilding every subterm it passes through. The pass has one mercy, and it is what makes checking large proofs cheap in general: a subterm with no loose indices is returned as-is — same pointer, no copy, no descent. Here the mercy never fires: the innermost body mentions `n1`, so the outermost pass rebuilds essentially the entire certificate below it; and because the binders are separated by the splitter applications, every level runs its own pass over the fresh copy the previous level has just built. The bill is n passes of $O(n)$ work each — quadratic checking time for a certificate of linear size — and, since every pass allocates pointer-distinct copies of structurally equal terms, the kernel's pointer-keyed caches miss at every level too. The running example pays none of this: an `if-then-else` statement does not introduce a binder in its branches. This cost is inherent to the kernel rather than to our procedure, so, unfortunately, we cannot overcome it by tinkering with the `vcgen` implementation. The recommendation we can give to users is to avoid programs with deeply nested `match` statements: either rewrite them with ordinary `if`s, or split such functions into separate sub-functions and prove a high-level specification for each.

5.8 Summary

This chapter gave a tutorial on the `vcgen` implementation. Starting from a naive `MetaM` implementation of a VC generation procedure for stateful monadic computations, we have shown how to refine it into a close prototype of the production `vcgen`. We did it in

five steps. First, we introduced the `SymM` framework — Lean's engine for efficient symbolic execution procedures — and ported our naive implementation to it; this gave us an orders-of-magnitude speedup. Then we spotted a problem with our port — it did not scale to more sophisticated execution models. That is because we had, unconsciously, violated the main invariant of `SymM`: every expression manipulated in the goal must be maximally shared. Once this subtlety was fixed, we extended our tactic to arbitrary monadic computations, and right after that added support for control flow; to add both features, we had to introduce a rule-caching mechanism into our tactic. Finally, we noticed a problem with VC dispatching in the presence of deeply nested control flow — our tactic generates a lot of VCs, and solving all of them independently turns out to be too costly. However, the local contexts of the generated VCs overlap significantly. This brought us to the idea of sharing the `grind` work of internalising those contexts among all the goals, resulting in a tactic that generates and solves hundreds of VCs in a fraction of a second. We concluded the chapter by highlighting other production `vcgen` features and its main limitations.

So far, all our experiments have been on artificial benchmarks, designed to measure each component of `vcgen` in isolation. A reader might therefore still wonder how all the optimisations described in this chapter come together on more realistic case studies. We answer this question in the next chapter, by building Velvet: a foundational auto-active verifier with Dafny-comparable performance.

6

Velvet: A Dafny-Style Verifier in Lean

The previous chapters developed Loom: general and practical foundations for monadic program verification (Chapter 4) together with the automation that makes it practical (Chapter 5). This chapter puts those foundations to work in Velvet — a verifier for imperative programs with effects such as mutable state, non-determinism, and non-termination, built as a Loom instance and addressing shortcomings of existing automated program verifiers. Velvet is also where the question that opened Chapter 5 — *can we match the performance of a non-foundational auto-active verifier in a proof assistant?* — receives its practical answer: evaluated head-to-head against Dafny on a suite of 27 benchmarks (Sec. 6.3), Velvet delivers comparable automation and performance while remaining fully foundational — and is, in fact, faster on 25 of the 27 benchmarks.

Velvet programs are Lean programs, with semantics given by composing a number of computational effects implemented as monad instances. The verifier is designed to support *multi-modal* verification: programs in its language can be compiled, executed, validated using property-based testing, and formally verified within one unified environment. It seamlessly integrates automated reasoning (as in Dafny [72] and Verus [68]) with interactive Lean proofs: when automation fails, the user can complete the proof interactively using standard Lean tactics. By building on a library of monadic effects, Velvet supports mutable variables, arrays, loops (including non-terminating ones), and a random choice operator, in addition to all of Lean's native datatypes, making it well suited for reasoning about textbook algorithms and competitive-programming tasks. As a Lean library, Velvet inherits access to the entire Lean ecosystem, including mathlib [81], the world's largest library of formalised mathematics. Finally, Velvet is *foundational*: like Loom itself, it is formally verified, so any program verified in it provably satisfies its specification at runtime.

6.1 Velvet by Example

In this section, we give a tutorial-style overview of Velvet's key features: `grind` [88] and SMT-based proof automation, multi-modal proofs, runtime verification via property-based testing, and reasoning about program termination and non-determinism.

6.1.1 Auto-Active Program Verification via SMT

Fig. 6.1 displays a Velvet program that computes an integer under-approximation of the square root of a natural number. The program omits a precondition, which is then triv-

```
1  method sqrt (x : ℕ) return (res : ℕ)
2  ensures res * res ≤ x
3  ensures ∀ i ≤ x, i * i ≤ x → i ≤ res do
4    if x = 0 then return 0
5    else
6      let mut i := 0
7      while i * i ≤ x
8      invariant ∀ j, j < i → j * j ≤ x
9      decreasing x + 1 - i do
10       i := i + 1
11     return i - 1
12
13 prove_correct sqrt by velvet_solve <;> velvet_smt [*]
```

Figure 6.1: Discrete square root in Velvet

ially `True`, but features two conjuncts in its postcondition, each given by an individual `ensures`-clause. The first states that the result `res`, squared, is no greater than the argument `x`; the second states that `res` is the largest number to under-approximate $\sqrt{x}$. The body at lines 4–11 computes the result. Line 8 provides an explicit loop invariant for the `while`-loop at lines 7–10, required to prove the postcondition. Line 13 constitutes the proof of the program with respect to the ascribed specification.

The proof at line 13 is achieved by Velvet computing the verification conditions (VCs) for the program and specification via a weakest-precondition calculus, producing a Lean statement whose validity entails the program's correctness. The tactic `velvet_solve` is a thin wrapper over the `vcgen` tactic from Chapter 5, which produces the required VCs and tries to discharge them with the `grind` automation tactic; any remaining goals are then handed to the `velvet_smt` tactic, which calls an external SMT solver — precisely the discharge strategy we laid out in Sec. 5.6. The interaction between Lean and the solver goes through the Lean-SMT library [86], which translates Lean statements into SMT-LIB [10] queries almost verbatim; Velvet uses cvc5 [8] by default but can be configured to use Z3 [34]. Importantly, the solver's results are not trusted: Lean-SMT reconstructs the solver's certificates as Lean proofs, so even the VCs discharged by SMT are certified by the Lean kernel, preserving Velvet's foundational guarantees. In this example, all VCs produced by Velvet are discharged automatically by either `grind` or the SMT solver.

6.1.2 *Combining Automated and Interactive Proofs*

The VCs produced by Velvet are just Lean theorems, and one can inspect the goals remaining after running `velvet_solve`. One of them is shown in Fig. 6.2. As is common in Lean, the proof context — the available variables and assumptions — is shown above the $\vdash$ symbol, while the goal to prove is displayed after it. Velvet automatically assigns names to hypotheses based on their source (`invariant`, `if`-branch, `require`, `ensures`). One can discharge this goal interactively, for instance by introducing `j` and performing case analysis on `j = i`.

This example highlights a crucial difference between Velvet and automated tools such as Dafny and Verus. Whereas those tools place the main automation burden on SMT solvers — reducing the user's role to proving helper lemmas and supplying assertions — in Velvet an SMT solver is just one of several means of automating the generated VCs. If a

```
x : ℕ
if_neg : ¬ x = 0
i : ℕ
invariant_1 : ∀ j < i, j * j ≤ x
if_pos : i * i ≤ x
⊢ ∀ j < i + 1, j * j ≤ x
```

Figure 6.2: A proof goal in Velvet

solver fails to discharge a VC (*e.g.*, because it falls outside its supported theories), the user can prove it interactively or with any Lean automation, such as the `aesop` tactic [77]. One can also simplify an unsolved VC — it is just a Lean goal — until it fits an SMT-supported first-order theory, at which point a solver can be invoked via a dedicated tactic such as `auto` [112]. Finally, Velvet allows a more traditional SMT-centred mode, in which the user adds facts proven as ordinary Lean theorems to the database available to SMT by marking them with the `@[solverHint]` annotation.

6.1.3 *Property-Based Testing against Specifications*

In addition to proof automation, Velvet provides facilities for randomised testing of programs against their specifications. This approach, known as *property-based testing* (PBT) [27], is effective in practice for exposing issues either in the program code or in the specification [48].

The first step in performing PBT for a Velvet program is to extract executable code from its definition. Velvet provides a command that automatically generates a pure function taking the same arguments and returning the same result; if the original program has no non-deterministic choices, the determinised version runs exactly the same code. To enable PBT, pre- and postconditions must be shown to be decidable, so they can be evaluated against generated inputs and resulting outputs. Velvet automates this step and lets users supply a manual proof for non-trivial `Decidable` instances that are not inferred automatically. From these components Velvet assembles an end-to-end tester function: given an input, it returns a `Bool` indicating whether the implication from "the input meets the precondition" to "the post-state of the determinised program satisfies the postcondition" holds. The tester can be used with any PBT library; in our experiments we use Plausible [69].

6.1.4 *Program Proofs and (Non-)Termination*

In its default mode, Velvet verifies *partial* (*a.k.a.* functional) correctness: unlike Dafny, Velvet programs need not terminate to satisfy their specification. In fact, removing or commenting out line 10 in Fig. 6.1 would not prevent the proof at line 13 from succeeding: the `while`-loop would then never terminate, so the program would trivially satisfy any postcondition, even `False`. Fortunately, Velvet allows non-termination to be treated either as success or as failure, enabling *total*-correctness reasoning, and switching to the latter mode is as easy as setting a single option. The foundations supporting both kinds of reasoning are exactly those developed in Sec. 4.5.

For total correctness, the user provides explicit termination measures via `decreasing`-clauses on each `while`-loop, as on line 9 of Fig. 6.1. With these annotations in place

```
1 method Predicate.toArray (mut s : α → Bool) return (res : Array α)
2 ensures ∀ x, sOld x ↔ x ∈ res do
3   let mut res := #[]
4   while ∃ x, s x
5   invariant ∀ x, sOld x ↔ x ∈ res ∨ s x do
6     let x :| s x
7     res := res.push x
8     s := fun y => if y = x then false else s y
9   return res
```

Figure 6.3: A program with a choice operator

and line 10 commented out, the VC generator produces, among others, a Lean goal that cannot be proven — but which conveys a useful insight into the failure: it shows that the termination measure `x + 1 - i` does not, in fact, decrease with each iteration of the loop.

6.1.5 *Reasoning about Non-Determinism*

Velvet supports the Dafny-style *let-such-that* operator `:|` (*a.k.a.* Hilbert's epsilon operator) [73], useful for modelling non-deterministic choice. Fig. 6.3 features a program that takes a predicate `s` and returns an array of all the values satisfying `s`. The postcondition states that any element of the result satisfies the predicate and any value satisfying the predicate occurs in the result. The code is straightforward: it non-deterministically takes a value satisfying `s` (line 6), adds it to the answer (line 7), and removes it from the predicate (line 8), repeating until none are left (the condition on line 4). The operator `:|` models the choice inside the loop.

For the program in Fig. 6.3, the concrete values chosen by `:|` affect neither the outcome nor correctness; yet it is easy to imagine a scenario in which a non-deterministic choice leads to an undesired outcome the user wishes to identify. To support both traditional correctness reasoning (*a.k.a.* over-approximation) and reasoning in the style of Incorrectness Logic (*a.k.a.* under-approximation) [98, 138], Velvet treats non-determinism symbolically in two ways: *angelic* or *demonic* [18]. Angelic semantics existentially quantifies over the outcome of a choice in the VCs needed to establish the postcondition, while demonic semantics uses universal quantification, requiring the postcondition to hold for *any* outcome of the choice operator.

6.1.6 *Using Mathematical Libraries in Program Specifications*

Velvet can combine program verification with reasoning about pure Lean datatypes and mathematical theories from mathlib [81]. Consider the program in Fig. 6.4 that approximates $\int_0^1 x^2\,dx$ with a left Riemann sum over a partition into n equal segments. The `while`-loop accumulates the sum of the values at the left endpoints of the partition. Velvet verifies the program automatically after adding hints to `grind` about `Finset.range`. It then remains to show that the computed sum approximates the integral, which amounts to proving $\lim_{n\to\infty} \sum_{j=0}^{n-1} j^2/n^3 = \frac{1}{3}$. This fact is proven interactively and involves no reasoning about code; in principle, such proofs could be delegated to AI systems optimised for mathematical reasoning in Lean.

```
noncomputable method oneThird (n : Nat) return (res : ℝ)
require n > 0
ensures res = ∑ j ∈ range n, (j^2/n^3 : ℝ) do
  let mut res : ℝ := 0
  let mut i := 0
  while i < n
  invariant i ≤ n
  invariant res = ∑ j ∈ range i, (j^2/n^3 : ℝ)
  decreasing n - i do
    let x : ℝ := i / n
    res := res + (x * x) / n
    i := i + 1
  return res
```

Figure 6.4: Approximating $\int_0^1 x^2\,dx = \frac{1}{3}$ in Velvet

```
def Predicate.toArray {α} s : VelvetM ((Array α) × (α → Bool)) := do
  let sOld := s;
  let res := #[];
  VelvetM.repeat (res, s) fun (res, s) => do
    invGadget ∀ x, sOld x ↔ x ∈ res ∨ s x
    if ∃ x, s x = true then do
      let x <- pickSuchThat α fun x => s x = true
      let s := fun y => if y = x then false else s y
      pure (ForInStep.yield (res.push x, s))
    else pure (ForInStep.done (res, s))
```

Figure 6.5: Monadic form of the code from Fig. 6.3

6.2 Elements of Velvet Implementation

6.2.1 *Semantic Foundations*

Velvet is built as an instance of Loom (Chapter 4) — a general Lean library for implementing provably correct multi-modal verifiers by means of a monadic *shallow* embedding (*i.e.*, Velvet programs are Lean *programs*, not data instances). Although Velvet's syntax resembles regular imperative code, under the hood its programs are encoded as monadic computations. For example, after a series of macro expansions, the code from Fig. 6.3 is translated into the monadic representation shown in Fig. 6.5. This is a computation in the `VelvetM` monad, which supports effects such as non-deterministic choice and divergence.

Mutable variables in Velvet are supported by a specialised macro expansion that "threads" their values through the monadic computation. For instance, to mimic the mutation of `s` in Fig. 6.3, Velvet adds an extra value of type $\alpha \to$ `Bool` to the return type of the computation. This value is then threaded — together with the mutated result array `res` — through the `while`-loop. The loop itself is modelled by the monadic `repeat` function, which iterates the loop body, passing through the intermediate values of the affected variables (here `res` and `s`):

```
repeat (init : β) (body : β → VelvetM (ForInStep β)) : VelvetM β
```

At each iteration, the `body` produces a `ForInStep` value, either `ForInStep.yield x` to continue the loop with a new value `x`, or `ForInStep.done x` to terminate with a final value `x`. Here, if the loop condition holds, the loop resumes, updating the result array with

a non-deterministically chosen `x` (`:|` is represented by `pickSuchThat`); otherwise the loop terminates. In addition to the computational payload, the code in Fig. 6.5 features several annotations useful for VC generation, which we discuss next.

6.2.2 *Verification Conditions Generation*

A verification-condition generator for Velvet is obtained "for free" from the underlying Loom framework, which automatically derives VC generators for computations expressed by monads capturing common effects. In particular, `VelvetM` is defined as `NonDetT DivM`, where `NonDetT` is the monad transformer for non-determinism (Sec. 4.6) and `DivM` the monad for divergence (Sec. 4.5). This combination provides exactly the operations we need: (a) `NonDetT` supports non-deterministic choice (such as `pickSuchThat` from Fig. 6.5), and (b) `DivM` supports divergence (such as the potentially non-terminating `repeat` loop). Both are supported by Loom out of the box, so Loom automatically derives a weakest-precondition transformer `wp` for `VelvetM` of type `wp : VelvetM β → (β → Prop) → Prop`. The `velvet_solve` tactic then uses this transformer to convert a Velvet program into a formula and split it into separate goals — one per verification condition.

6.2.3 *Annotations for Intrinsic Program Verification*

In the examples above we have seen various program annotations: `require`/`ensures` clauses, assertions, loop invariants, and termination measures. These annotations are verification-only: they do not change the executable behaviour of the code. Velvet's VC generator collects the conjunction of all `require` and `ensures` clauses into the program's Hoare-style correctness statement. Likewise, loop `invariant` annotations are compiled into additional hypotheses for the respective VCs, thanks to the no-op combinator `invGadget` (*cf.* Fig. 6.5):

```
def invGadget (inv : Prop) : VelvetM Unit := pure ()
```

6.2.4 *Non-Deterministic Choice Operator*

The general form of Velvet's non-deterministic choice operator is:

```
def pickSuchThat {τ} (p : τ → Prop) [Findable p] : VelvetM τ := ...
```

This operator chooses an arbitrary value of type τ satisfying the predicate `p`; following Dafny notation, in program code it is written as `let x :| p x`. To execute code featuring it, we require `p` to come with a `Findable` instance, which provides a value `set` together with a proof that, if the predicate is satisfiable, then `find`'s result satisfies `p` (otherwise the value can be anything). In practice, constructing a `Findable` instance is rarely a problem: if `p` is decidable and τ is countable and non-empty, such an instance is inferred automatically.

The weakest-precondition transformer for this operator depends on the verification style chosen by the user. For example, to prove that the specification holds for *all* choices of `pickSuchThat`, the weakest precondition should ensure that the predicate is satisfiable and that the postcondition holds for all values satisfying `p`:

$$wp\ (pickSuchThat\ p)\ post = (\forall x \in p,\ post\ x) \wedge (\exists x,\ x \in p). \tag{6.1}$$

The last conjunct ensures that the value returned by `find` always satisfies `p`, and hence that execution of the Velvet program is guaranteed to satisfy the specification. Alternatively, the user might wish to prove a reachability property, *i.e.*, that the specification holds for *some* choice; in that case the weakest precondition asserts that there *exists* a value satisfying both the predicate and the postcondition. Velvet supports both styles via an option flag.

6.3 Evaluation

Velvet is available online.[1] We have evaluated it with respect to three research questions:

RQ1: How do Velvet's effectiveness and performance as an automated program verifier compare to those of Dafny?

RQ2: How much manual proof effort is required to complete Velvet proofs when its automation fails?

RQ3: What are characteristic examples where Velvet's capabilities as an interactive prover offer a better verification experience than Dafny?

All experiments below were run on a 2026 MacBook Pro with 64 GB of RAM and an Apple M4 chip, with cvc5 version 1.3.1, Z3 version 4.15.4, and Lean version 4.32.0.

6.3.1 RQ1: Automated Program Verification

To test RQ1, we manually selected 27 medium-complexity case studies from the VERINA benchmark suite [137]: each is a single program containing at least one loop requiring non-trivial invariant annotations. The case studies total 618 non-empty, non-comment lines of Lean code (on average 23 lines per program), of which 303 lines are specifications, invariant annotations, and proofs. Each benchmark has an equivalent Dafny implementation. We aimed to automate verification as much as possible, inserting similar manual proofs for both Velvet and Dafny when necessary. For all programs we used Lean's `grind` tactic to discharge the VCs; one program, `isPrime`, whose invariants involve non-linear arithmetic, additionally required supplying two existential witnesses that `grind` does not guess on its own. All reported times are averaged over three runs.

Our results are shown in Fig. 6.6. The case studies fall into two groups: 21 discharged automatically by Velvet and 6 that required some manual proof effort. Notably, Velvet and Dafny required manual proofs for the same case studies, suggesting comparable effectiveness. In terms of performance, Velvet surpasses Dafny on 25 of the 27 benchmarks, and verifies every case study in under two seconds; on average it is about 1.2× faster (the geometric mean of the per-benchmark ratios), totalling 17.6 s against Dafny's 20.0 s over the whole suite. Per-phase timing shows that VC generation takes 20–70 ms per method throughout: on the easy benchmarks the wall-clock time is dominated by a roughly constant per-file overhead (Lean start-up, elaboration, and kernel checking), while the two exceptions where Dafny is faster, `findEvenNumbers` and `mergeSorted`, are dominated by

[1] Sources: https://github.com/verse-lab/velvet; archived artifact: https://doi.org/10.5281/zenodo.19801401.

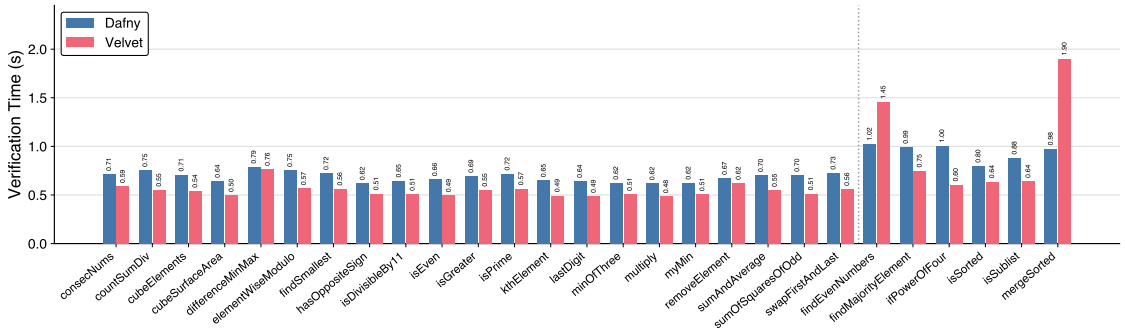

Figure 6.6: Comparison of end-to-end verification times between Dafny and Velvet on the 27 VERINA case studies. The dashed line separates the 21 case studies discharged fully automatically from the six that required manual proof effort.

`grind` discharging genuinely hard proof obligations rather than by infrastructure cost. We conclude that Velvet can verify a variety of programs with the same level of user effort as Dafny — and, on this suite, faster.

6.3.2 *RQ2: Combining Automated and Interactive Proofs*

To answer RQ2, we selected six case studies from VERINA that required manual proof effort in Dafny. We manually inserted additional assertions and proved the required lemmas in Dafny to help the SMT solver discharge the VCs. Notably, Velvet was likewise unable to discharge the same VCs automatically and required similar additional lemmas. Unlike Dafny, however, when VCs cannot be discharged automatically Velvet shows the proof-goal context, letting users inspect available facts and guide the proof. Furthermore, Velvet leverages datatypes from the rich Lean standard library, whose extensive set of lemmas helps discharge residual VCs with less manual effort — which is why Velvet's proofs of the required lemmas were shorter than Dafny's.

Fig. 6.7 breaks down the manual effort for these six benchmarks into two categories: *annotations*, comprising function pre- and postconditions (`requires`/`ensures`) and loop invariants; and *proof*, counting auxiliary definitions, lemmas, and `assert` statements interleaved with the program. Across all six benchmarks, Velvet requires on average 3.3× fewer total lines than Dafny, and the gap is driven almost entirely by the proof component: Velvet's proofs are 4.9× shorter on average. Annotation counts, by contrast, are nearly identical, indicating that the savings come from Lean's standard library and tactic ecosystem rather than from a less verbose specification language. For instance, `findMajorityElement` computes the majority array element; in Dafny one must define a counting function `count` on lists and prove it distributes over concatenation, whereas Velvet reuses `List.count` from the standard library, for which this lemma is already proven. Adding such lemmas to the automation database discharges the VCs automatically. We conclude that Velvet offers a better experience than Dafny for programs with complex mathematical specifications that cannot be discharged automatically.

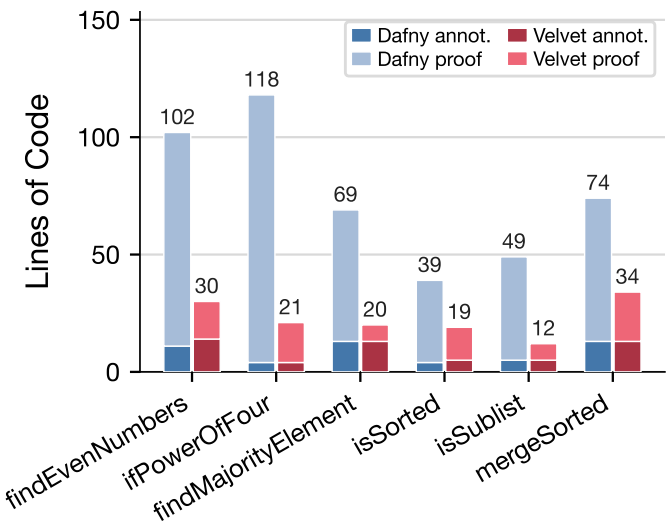

Figure 6.7: Lines of code for the six case studies that required manual proof effort, split into specification and invariant annotations and proof lines, in Dafny vs. Velvet.

6.3.3 *RQ3: Increased Expressivity beyond Auto-Active Verifiers*

For RQ3 we considered three case studies: `memAlloc`, a classic first-fit memory allocator with a free list; `isNonPrime`, an efficient primality check; and `oneThird`, the left-Riemann-sum approximation of $\int_0^1 x^2\,dx$. Each requires substantial manual proof effort. We verified all three in Velvet and report the results below.

Memory allocation. This procedure implements a classic first-fit memory allocator that searches a linked list of free memory blocks in a memory graph for one large enough to satisfy an allocation request. The memory graph is encoded functionally, via a function `next : addr → addr` returning the next address and a function `block_size : addr → Nat` mapping addresses to sizes. Verifying this program in Velvet requires developing a custom theory of paths in the graph, amounting to 15 lemmas (250 lines of interactive proof in total), such as `path_append` (path concatenation) and `path_split_at_p` (path splitting at a given position). As Velvet is built on Lean, all such lemmas can be proven in Lean's native interactive mode, with access to the proof context at each step and a rich set of automation tactics like `aesop` [77], `simp`, and `grind` [88]; the lemmas then discharge the remaining VCs in Velvet. This example shows that Velvet can verify programs whose proofs require developing custom theory libraries.

Prime number checking. This procedure determines whether a number is prime by checking that it is not divisible by any number up to its square root. Verifying it requires extensive reasoning about multiplication and division of integers, known to be hard for SMT solvers and therefore difficult in SMT-based verifiers such as Dafny. In Velvet, however, the proof amounts to just nine lines, as we can simply use the `Nat.prime_def_le_sqrt` lemma from mathlib.[2] This shows that, by leveraging Lean's li-

[2]This lemma states that a number is prime if and only if it is greater than one and not divisible by any number from 2 up to its square root.

brary ecosystem, even programs whose correctness relies on complex mathematical reasoning can be verified with minimal manual effort.

Integral approximation. This procedure was shown in Sec. 6.1.6: it approximates $\int_0^1 x^2\,dx$ with a left Riemann sum. Specifying and verifying `oneThird` requires working with complex mathematical objects such as limits, summations, and integrals, many of which cannot be encoded from first principles in SMT-based verifiers such as Dafny — so even *stating* the specification is beyond their reach. As a Lean library, Velvet accommodates arbitrarily complex mathematical specifications while still providing Dafny-level automation for the program-relevant VCs, letting the user focus on the problem-specific mathematical parts of the proof.

6.4 Discussion

Expressivity. Velvet is a shallow verifier on top of Lean: programs are encoded directly using Lean's own datatypes and language features. As a result, Velvet natively supports reasoning about Lean's functional features, including local mutable state, type classes, and pattern matching. This contrasts with auto-active verifiers like Dafny and Why3 [38], which model imperative programs in an object-oriented setting built around a global heap and reference-based classes. The flip side of Velvet's design is that it does not currently support global heap state or OOP-style classes; programs that essentially rely on shared mutable references (*e.g.*, doubly-linked lists with aliasing) are outside its present scope.

Early adoption. Even though the first version of Velvet was released only in October 2025, it has already been used in verification projects outside the core development team. In November 2025 it was used to win first place in one category of the regional VeHa verification competition: a one-person team implemented and verified in Velvet an efficient procedure for finding, in an arbitrary string, the longest substring consisting of digits only. A verification problem stated in Velvet (`memAlloc` from RQ3 of Sec. 6.3) was also offered as one of the tasks in the 2025 CCF ChinaSoft Theorem Proving Contest.

6.5 Related and Future Work

Conceptually, Velvet's approach to multi-modal program verification is similar to Why3 [38], which provides WhyML — a programming language with verification support that uses SMT solvers and interactive provers (*e.g.*, Rocq) to discharge VCs. Unlike Velvet, Why3 is not foundational: while its logic fragment has been mechanised in Rocq [29], the mechanisation does not yet cover the semantics of WhyML or VC generation. Both Dafny [72] and F* [120], while positioned as auto-active verifiers, offer experimental support for tactic-based interactive proofs [26, 80], but those modes are not easily extensible with user-defined tactics and provide no access to mathematical libraries such as mathlib. Daenerys [117] is a recent attempt to supplement interactive proofs in the Rocq-based foundational program logic Iris [61, 66] with SMT-enabled automation; unlike Velvet, it does not generate SMT-friendly VCs automatically and requires manually translating

them from Rocq to SMT-LIB. Lean's own VC generator for monadic computations in the Std.Do library provides facilities for reasoning about imperative computations, but does not support programs with non-terminating loops or non-determinism.

Veil [106] is another multi-modal verifier built on top of Loom (Chapter 4). Unlike Velvet, Veil does not offer a general-purpose imperative language; it targets verification of distributed protocols expressed in a high-level DSL optimised for emitting SMT-friendly first-order VCs. In the future, we plan to tackle multi-layer verification of large-scale systems in the style of IronFleet [54, 68] or Verdi [135] by combining Velvet and Veil.

6.6 Summary

We have presented Velvet, a foundational multi-modal verifier for imperative programs built on Loom, integrating SMT-style automation, interactive proofs, and property-based testing in a single Lean environment with seamless access to mathlib. Our evaluation on VERINA and three case studies shows that this design matches state-of-the-art auto-active verifiers in effectiveness — and surpasses Dafny in verification time on 25 of 27 benchmarks — while enabling proofs whose specifications rely on mathematical theories beyond the reach of SMT solvers. Velvet is a step toward verification environments in which push-button automation and interactive foundational reasoning coexist within a single unified workflow.

7
Conclusion

In this work, we have asked whether foundational guarantees, generality, and scalability can coexist in a single formal verification framework. The best success stories of formal methods typically score at most two of these criteria, and hence inevitably face strong limitations. The effort required to understand and work with foundational and general frameworks usually becomes overwhelming well before the problems we tackle reach real-world demands. Building a system that is practical in terms of automation and performance requires a huge amount of engineering, which is hard to justify foundationally, so the natural temptation is to give up on foundational guarantees. However, as soon as we try to make a non-foundational framework general, accommodating a broad range of metatheory features, its TCB becomes too large — weakening our certainty in program correctness to a level unacceptable in the reality of the AI era. Another way out is to sacrifice generality and build a highly specialised yet somewhat practical system. While there have been relatively successful stories of such developments, the amount of required effort is still too large to be justified by the single narrow application such projects target.

We set ourselves the goal of developing a system that does not compromise on any of the three criteria from the very first step. What largely made such a framework possible at all was the recent tremendous progress in program verification. First, seL4 demonstrated that the monadic shallow embedding approach can scale to large systems. Then, advances in the theory of Dijkstra monads showed that there is a strong, coherent story behind reasoning about monadic code. Finally, the last missing component was a system in which both the metatheory of monadic reasoning and the best practices of building program verifiers could be embedded: such a system is Lean4 — a recently developed programming language *and* proof assistant. The main reason we could answer the opening question positively is that we stood on the shoulders of three giants.

7.1 Summary of contributions

The first step was to develop foundations which can serve practical needs, while being general enough to accommodate the semantic features of real-world languages. In a sense, we wanted to build a metatheory which, in each particular instance, is as simple as the one seL4 had, but is also as general as Dijkstra monads. This would give us confidence that each particular instantiation can scale to industrial-size projects, while the common foundation would provide a shared vocabulary that can be reused across

multiple applications. As we have explained, the main reason why Dijkstra monads do not immediately fit as a silver-bullet solution here is the encoding overhead they introduce into the generated VCs. In other words, if one were to redo the seL4 verification effort as an instance of the Dijkstra monads formalism, it would have been significantly harder: most of the generated VCs would not fit into the first-order fragment of separation logic, and most of the automation (developed for the simple Isabelle metatheory) would simply not work. To overcome this problem, we have reverse-engineered the most general subset of the Dijkstra monads theory that can be formalised without the encoding overhead — monads with a weakest-precondition transformer into PredTrans — and adapted the existing verification techniques to it. In particular, we have formalised reasoning about effects and effect compositions in general, and developed several concrete but important instances for effects such as state, exceptions, partiality, ghost code, and non-determinism. For some of these effects, such as partiality and ghost code, to the best of our knowledge, no practical foundational formalisations were previously available. This was our first contribution (Chapter 4).

The main reason we picked the theory of Dijkstra monads over other Hoare-triple-based foundations, such as Hoare Type Theory [94], was to support VC generation through weakest-precondition calculation at the level of the foundations. Building a verifier around a weakest-precondition calculus was already a well-known best practice in formal verification. Our second contribution was to build the first foundational verification framework that matches the performance of state-of-the-art non-foundational ones (Chapter 5). The reason this was hard is that we had to balance on three pillars. The first, and the most well-known, is the efficiency of the verifier itself. The second is the efficiency of constructing the certificate that justifies each step of the verifier. And the last is making sure that the certificates produced by the verifier — in the form of Lean proof terms — can be checked by the Lean kernel in a reasonable amount of time. These pillars are exactly why every benchmark in Chapter 5 reported its three phases — generation, discharge, and kernel time — separately. Of course, thinking about these three problems every time we write a new line of code would not get us far. The only way to develop code under such tight constraints is to come up with invariants on the framework that, once maintained, keep all three pillars standing, and then with coding patterns that preserve these invariants while remaining flexible enough to express the logic of the verifier. Luckily, half of these problems had already been solved by SymM — Lean's framework for building efficient symbolic execution engines (Sec. 5.2). SymM defines precisely the invariants a Lean metaprogram should maintain to achieve maximal performance in certificate construction and kernel checking; among them, the monotonic growth of the local context and the hash-consing of every expression appearing in the goal. On top of SymM, we have identified typical code patterns, common in verifier engineering, that satisfy these invariants by default. The first such pattern is organising the verifier as a ladder of *strategies*, each responsible for handling its own kind of goal, with a high-level scheduler running them (Sec. 5.1). This separates the concerns and offloads the work of preserving the invariants to each strategy in isolation. The second pattern is the unification of individual verifier steps into cached rule applications within a single strategy (Sec. 5.4). This pairs the implementation of a strategy with the construction of its certificates, in isolation from the rest of the code. If we fix the type of the produced rules,

rule application can be unified across multiple strategies. Then, since the constructed rules are cached anyway, we can separate the concern of producing a robust certificate from that of fitting the constructed rule into the SymM invariants: the rule-construction procedure is not on the hot path and does not need to be extremely efficient, so there we can focus on robustness and readability, while fitting the rule's expressions into the SymM invariants happens entirely at the caching level — a procedure unified across all rule-based strategies. The last pattern concerns the automation: internalising the `grind` context at each split, at the level of the strategy scheduler (Sec. 5.6). This way, when extending the verifier, one does not have to worry about the performance of VC discharge and can focus entirely on designing the strategy at hand.

Finally, our last contribution was Velvet (Chapter 6): an auto-active verifier built on Loom, demonstrating that the techniques described above indeed yield a foundational verifier with performance comparable to state-of-the-art non-foundational tools — on the VERINA benchmarks, Velvet beats Dafny on all but two case studies, and is less than a second slower on those two.

7.2 Future Directions

In this work, we have shown that shallow monadic embedding is a way to combine foundational guarantees, generality, and scalability. Such a combination is therefore possible — but this is, of course, not the end of the story. We have only made the first step into the uncharted territory of a new era of program verifiers, and the real journey starts from here. Let us list a set of future directions, each of which starts at a limitation of the current development.

7.2.1 Deep Embeddings

The shallow embedding story does not entirely solve the problem of faithfully modelling programming-language semantics. Even in the seL4 verification effort, although the actual verification was done at the shallow-embedding level, there was still a need for a deep embedding of a subset of the C language, together with a proper definition of its semantics, which was then shown to refine the abstract shallow monad. seL4 demonstrated that, to make formal proofs feasible, they should be carried out at the level of the abstract shallow layer; however, to transfer those proofs down to the deeply embedded code, one has to either develop a formal technique for proving refinements, or somehow generate VCs for deep embeddings directly, distilling the functional-correctness VCs from the low-level semantic details. The first step towards the latter approach would be to define an analogue of the `WPMonad` class for deep-embedding models — which is already being prototyped in Lean today.

The prototype modularises the `WPMonad` type class into two, as shown in Fig. 7.1: a `WP` class over arbitrary program representations, which provides the weakest-precondition transformer together with its monotonicity law, and a `WPMonad` class for monadic programs, which carries a `WP` instance for every result type and adds the soundness laws for `pure` and `bind`.

Notably, for `vcgen` to run, it suffices to require only the first class, so the automation story

```
class WP (Prog : Type u) (Value : outParam (Type v)) (Pred : outParam (Type w))
    (EPred : outParam (Type w')) [Assertion Pred] [Assertion EPred] where
  wpTrans : Prog → PredTrans Pred EPred Value
  wp_trans_monotone (x : Prog) : wpTrans x |>.monotone

class WPMonad (m : Type u → Type v) (Pred : outParam (Type w))
    (EPred : outParam (Type w')) [Monad m] [Assertion Pred] [Assertion EPred]
    extends LawfulMonad m where
  toWP : ∀ α, WP (m α) α Pred EPred
  pure_le_wp_pure (x : α) (post : α → Pred) (epost : EPred) :
    post x ⊑ (toWP α).wp (pure (f := m) x) post epost
  bind_le_wp_bind (x : m α) (f : α → m β) (post : β → Pred) (epost : EPred) :
    (toWP α).wp x (fun a => (toWP β).wp (f a) post epost) epost
      ⊑ (toWP β).wp (x >>= f) post epost
```

Figure 7.1: The prototype refactoring of `WPMonad` into a program-level `WP` class and a monad-level `WPMonad` class.

transfers as-is. Yet it is not entirely clear how well the restricted `WP` class will cope with the full low-level complexity of real-world programming languages; further experimentation with this interface should help refine it, pushing the limits of the framework.

7.2.2 *Relational Properties*

Unfortunately, even if we managed to embed two versions of the same program (say, an abstract shallow monadic model and a deep low-level implementation), our framework would not be of much use in proving the refinement between them.

Right now, the class of properties our metatheory can support is limited to unary properties of a single program run. Extending the Loom metatheory to express relational properties — properties relating *two* runs, of the same program or of different ones, such as refinement or non-interference — and providing proof principles for them is another fruitful and promising direction. Maillard et al. develop a relational version of the Dijkstra monads theory [79]. What is not yet clear is how to map their ideas onto the Loom foundations and, even more so, how to extend the `vcgen` automation to handle relational proofs. For the latter problem, one might consider the *product program* approach [11]. A product of two programs is a single program that interleaves the steps of both, so that one run of the product simulates a pair of runs of its components. This reduction turns a relational verification problem into a unary one: a relational specification of the pair becomes an ordinary pre-/postcondition on the product, which the existing unary machinery — including `vcgen` — can then handle. The proof of the product typically goes through a "lock-step" invariant: an assertion over the states of both components that describes how the two states *relate* to each other (say, that two loop counters stay equal), rather than what each state actually is. The price of this reduction is that constructing a good product is not automatic — it requires finding an *alignment* between the two programs, *e.g.* synchronising their loops, so that the lock-step invariant stays simple.

7.2.3 *Interaction Trees*

If we aim to precisely model the semantics of the actual syntax of a source language, but do not want to leave the monadic interface, one approach is Interaction Trees (ITrees) [136].

ITrees are a coinductive data structure representing possibly-infinite trees of computations: an ITree either returns a value, performs an internal (silent) step — which makes it possible to represent divergence — or issues a visible *event* and continues depending on the environment's answer. Crucially, the ITree data type is itself a monad, so the techniques developed in this work would, in principle, apply to generate VCs for programs encoded this way. The first obstacle is that ITrees are defined coinductively, and Lean, at the time of writing, has no native support for coinductive types.

Vistrup et al. [131] develop an Iris-based [61, 66] way to build separation logics for ITrees, modelling such advanced semantic features as both angelic and demonic non-deterministic choice and concurrency. Their weakest-precondition calculus is much more complicated than ours, and, since it is a separation logic, using it in automation would require some form of frame inference.

While this approach can model many advanced semantic features out of the box, it is not yet clear how to implement automation for it that matches the performance of non-foundational auto-active verifiers.

7.2.4 *Separation Logic and Viper*

Implementing automation for the full-blown generality of Iris might be an overwhelmingly difficult task to begin with. Another approach would be to tackle the automation for smaller subsets, such as separation logics with fractional permissions [17] or Implicit Dynamic Frames (IDF) [116]. One of the prominent implementations of the latter is Viper [90]: an intermediate verification language built exactly around IDF. Viper programs are not meant to be executable; the two primitives the language is built on are the inhale and exhale commands. Intuitively, inhale H *obtains* the assertion H: it adds the permissions and assumptions described by H to the current state, as when assuming a precondition on entry to a procedure. Dually, exhale H *gives up* H: it checks that the current state contains a fragment satisfying H and removes it, as when passing ownership to a callee at a call site.

One can view Viper programs as computations in the $(\alpha \to \mathsf{hprop}) \to \mathsf{hprop}$ monad — the monad of predicate transformers over heap predicates, effectively the same one that CFML uses (Sec. 2.3.2). In this setting, one can express inhale as

$$\mathsf{inhale}\ H \triangleq \lambda post,\ H \mathrel{-\!\!*} post\ ()$$

where $-\!\!*$ is the *magic wand*: the weakest precondition of obtaining H is a heap that, once *extended* with any fragment satisfying H, satisfies the postcondition. Dually, exhale becomes

$$\mathsf{exhale}\ H \triangleq \lambda post,\ H * post\ ()$$

where the separating conjunction requires the current heap to split into a fragment satisfying H — the part being given up — and a remainder satisfying the postcondition. Then, since Viper programs become ordinary monadic computations, one could implement Viper-style automation on top of `vcgen`.

7.2.5 *Complexity Reasoning*

A simpler kind of resource logic is one that tracks a time counter, to reason about program complexity [6, 24, 51]. One way to do that would be to piggy-back on the ghost-state monad transformer from Sec. 4.7. Such a design would be both lightweight and composable: the ghost state is erased automatically by the Lean compiler, and the transformer can be stacked on top of any monadic computation. The challenge we foresee here is figuring out how to develop a comprehensive frame rule — at the level of both the metatheory and the `vcgen` automation — to cover complexity reasoning for more interesting monadic computations, *e.g.*, the probability monad. Another challenge worth highlighting is the interaction between shallow embedding and cost annotations. In a deep-embedding setting, the intended complexity model can be baked directly into the semantics of the language: if we then ask an AI agent to generate code, it has no way to cheat and underestimate the complexity of the program. With shallow embedding, this gets more complicated: the only way to count the cost of a function is to literally insert "ticks" into the program, as in the instrumented linear search of Fig. 4.11 — and then, without manual inspection, there is no way to ensure that the claimed complexity of the code matches its actual one.

7.2.6 *Immediate Loom Clients*

An orthogonal future direction is to reuse the framework we have built in the existing verifiers already organised around the idea of monadic shallow embedding in Lean. The two most prominent ones are Veil and Aeneas.

Veil has already undergone a migration of its metatheory to an early version of the Loom library [46]. However, we still see value in porting Veil to the latest Loom.

The first reason is that the older Loom metatheory was fully based on ordered monad algebras (*cf.* Sec. 4.4), and hence did not support separate postconditions for program paths terminating with exceptions. The assertions in Veil's state transitions, however, require a parametrised treatment — as successes or as failures — depending on the kind of property we wish to verify. For instance, for invariant preservation, assertion violations can be tolerated, while for reachability properties we need to ensure that no assertion ever fails. To model this in the ordered-monad-algebra setting, Veil defines multiple weakest-precondition transformer instances, each with its own treatment of assertion failure, and then provides bespoke proofs of how those instances relate to each other. In the new Loom foundations, we believe it is possible to develop all of this metatheory within a single instance, by simply adjusting the postconditions applied to failed assertions.

The second advantage of building Veil on top of the latest Loom is that it would then reuse `vcgen`. Right now, Veil implements its own VC-generation algorithm, which is not based on the SymM framework and, as a consequence, its performance is several times worse than that of state-of-the-art non-foundational verifiers in the field, such as IVy [101, 108].

Aeneas The story with Aeneas is quite similar: it develops its own metatheory and implements its own VC-generation framework. Unlike Veil, however, Aeneas's automation has already been migrated to the SymM framework. Nevertheless, building on top of `vcgen` would allow Aeneas to offload the burden of maintaining its own framework and would help develop Loom and Aeneas in sync.

7.3 Building Verifiers as We Go

We have presented various future directions for the work described in this thesis. We would like to conclude by sharing our vision of how all these directions can be reconciled, together constituting a *Universal Formal Verification Platform*.

Crucially, none of the future directions we have described are mutually exclusive. Verifying more complicated correctness properties, such as refinement or non-interference, would require extending the Loom metatheory while maintaining support for the existing fragment. Verifying properties other than functional correctness (*e.g.*, complexity) would require developing tools for particular effects (*e.g.*, a ghost resource-tracking state), which should compose freely with the others. Supporting more advanced effects of the underlying programs, such as concurrency, would require developing a dedicated automation layer for `vcgen` (*e.g.*, automated frame inference). The latter need not interfere with the current `vcgen` tactic, because it is designed as a ladder of strategies, each triggered when a particular program construct shows up — nothing, in principle, prevents us from parameterising the set of such strategies, making it possible for users to extend the tactic with domain-specific ones.

Making all this variety of Loom extensions compatible with each other, we can add each of them on demand, modularly, in sync with the evolution of the verifier that needs it. Every extension becomes a separate library, implementing its own concern like relational reasoning, resource accounting or concurrency. A user of such a platform can then start building a verifier for a simple subset of their source language with the core Loom and `vcgen` alone, and extend it as they go, adding complexity reasoning here and concurrency support there, one library at a time. This is the same "build the verifier as you go" experience we demonstrated in Chapter 3, elevated from a single verifier to a truly universal verification platform, which, by construction, keeps scoring all three criteria this thesis began with. Foundational guarantees are preserved because every component is foundational on its own, and so is any composition of them. Generality follows because the platform's only limit is the expressivity of Lean itself. And Lean's dependent type theory can embed computational models from the entirety of Computer Science. Scalability is maintained because each new component is developed together with its extension of `vcgen`, following the SymM invariants and the coding patterns identified in this thesis. The big, complex parts of a verifier are thus built once, foundationally, and shared. Thus the three criteria that before seemed mutually exclusive coexist not within a single verifier, but across an entire Ultimate Formal Verification Platform, where every new verifier starts not from scratch, but from the shoulders of all the previous ones.

Bibliography

[1] Smbat Abian and Arthur B. Brown. A theorem on partially ordered sets, with applications to fixed point theorems. *Canadian Journal of Mathematics*, 13:78–82, 1961. doi: 10.4153/CJM-1961-007-5. Cited on page 38.

[2] Alejandro Aguirre. Towards a provably correct encoding from F* to SMT. Technical report, Inria, 2016. https://catalin-hritcu.github.io/students/alejandro/report.pdf. Cited on pages 13 and 47.

[3] Danel Ahman, Cătălin Hriţcu, Kenji Maillard, Guido Martínez, Gordon Plotkin, Jonathan Protzenko, Aseem Rastogi, and Nikhil Swamy. Dijkstra monads for free. In *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 515–529, 2017. doi: 10.1145/3093333.3009878. Cited on pages 13, 16, and 17.

[4] Anthropic. Building a C compiler with parallel AI agents. Anthropic Engineering Blog, 2026. https://www.anthropic.com/engineering/building-c-compiler. Cited on page 1.

[5] Anthropic. Project Glasswing: An initial update, May 2026. https://www.anthropic.com/research/glasswing-initial-update. Cited on page 1.

[6] Robert Atkey. Amortised Resource Analysis with Separation Logic. *Log. Methods Comput. Sci.*, 7(2:17), 2011. doi: 10.2168/LMCS-7(2:17)2011. Cited on page 86.

[7] Chris Bailey. nanoda: An external type checker for Lean, in Rust. https://github.com/ammkrn/nanoda_lib. Independent re-implementation of the Lean kernel. Cited on page 2.

[8] Haniel Barbosa, Clark W. Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. cvc5: A Versatile and Industrial-Strength SMT Solver. In *TACAS*, volume 13243 of *LNCS*, pages 415–442. Springer, 2022. doi: 10.1007/978-3-030-99524-9_24. Cited on pages 47 and 71.

[9] Michael Barnett, Bor-Yuh Evan Chang, Robert DeLine, Bart Jacobs, and K. Rustan M. Leino. Boogie: A Modular Reusable Verifier for Object-Oriented Programs. In *Formal Methods for Components and Objects (FMCO)*, volume 4111 of *LNCS*, pages 364–387. Springer, 2006. doi: 10.1007/11804192_17. Cited on page 47.

[10] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. SMT-LIB: The Satisfiability Modulo Theories Library. Available at https://smt-lib.org/, 2025. Cited on pages 47 and 71.

[11] Gilles Barthe, Juan Manuel Crespo, and César Kunz. Relational Verification Using Product Programs. In *FM*, volume 6664 of *LNCS*, pages 200–214. Springer, 2011. doi: 10.1007/978-3-642-21437-0_17. Cited on page 84.

[12] Yves Bertot and Vladimir Komendantsky. Fixed point semantics and partial recursion in coq. In *PPDP*, pages 89–96. ACM, 2008. doi: 10.1145/1389449.1389461. Cited on page 38.

[13] Karthikeyan Bhargavan, Barry Bond, Antoine Delignat-Lavaud, Cédric Fournet, Chris Hawblitzel, Cătălin Hriţcu, Samin Ishtiaq, Markulf Kohlweiss, Rustan Leino, Jay Lorch, Kenji Maillard, Jianyang Pan, Bryan Parno, Jonathan Protzenko, Tahina Ramananandro, Ashay Rane, Aseem Rastogi, Nikhil Swamy, Laure Thompson, Peng Wang, Santiago Zanella-Béguelin, and Jean-Karim Zinzindohoué. Everest: Towards a verified, drop-in replacement of HTTPS. In *Summit on Advances in Programming Languages (SNAPL)*, 2017. Cited on pages 4, 13, and 47.

[14] Big Sleep Team. From Naptime to Big Sleep: Using large language models to catch vulnerabilities in real-world code. Google Project Zero Blog, November 2024. https://projectzero.google/2024/10/from-naptime-to-big-sleep.html. Cited on page 1.

[15] Richard Boulton, Andrew Gordon, Michael Gordon, John Harrison, John Herbert, and John Van Tassel. Experience with embedding hardware description languages in HOL. In *Theorem Provers in Circuit Design: Theory, Practice and Experience (TPCD)*, pages 129–156. North-Holland, 1992. Cited on page 6.

[16] Ana Bove, Peter Dybjer, and Ulf Norell. A brief overview of Agda — a functional language with dependent types. In *Theorem Proving in Higher Order Logics (TPHOLs)*, volume 5674 of *LNCS*, pages 73–78. Springer, 2009. doi: 10.1007/978-3-642-03359-9_6. Cited on page 3.

[17] John Boyland. Checking Interference with Fractional Permissions. In *SAS*, volume 2694 of *LNCS*, pages 55–72. Springer, 2003. doi: 10.1007/3-540-44898-5_4. Cited on page 85.

[18] Manfred Broy and Martin Wirsing. On the algebraic specification of nondeterministic programming languages. In *CAAP*, volume 112 of *LNCS*, pages 162–179. Springer, 1981. doi: 10.1007/3-540-10828-9_61. Cited on pages 41 and 73.

[19] Mario Carneiro. The type theory of Lean. Master's thesis, Carnegie Mellon University, 2019. https://github.com/digama0/lean-type-theory. Cited on page 2.

[20] Mario Carneiro. Lean4lean: Verifying a typechecker for Lean, in Lean. arXiv:2403.14064, 2024. Independent typechecker for Lean, written and verified in Lean. Confirm published venue before final submission. Cited on page 2.

[21] Oliver Chang, Dongge Liu, and Jonathan Metzman. Leveling up fuzzing: Finding more vulnerabilities with AI. Google Security Blog, November 2024. https://security.googleblog.com/2024/11/leveling-up-fuzzing-finding-more.html. Cited on pages 1 and 2.

[22] Arthur Charguéraud. Characteristic formulae for the verification of imperative programs. In *Proc. ACM Program. Lang. (ICFP)*, 2011. doi: 10.1145/2034773.2034828. Cited on pages 3 and 11.

[23] Arthur Charguéraud. Separation logic for sequential programs (functional pearl). *Proc. ACM Program. Lang.*, 4(ICFP), 2020. doi: 10.1145/3408998. Cited on page 11.

[24] Arthur Charguéraud and François Pottier. Verifying the Correctness and Amortized Complexity of a Union-Find Implementation in Separation Logic with Time Credits. *J. Autom. Reason.*, 62(3):331–365, 2019. doi: 10.1007/s10817-017-9431-7. Cited on page 86.

[25] Haogang Chen, Daniel Ziegler, Tej Chajed, Adam Chlipala, M. Frans Kaashoek, and Nickolai Zeldovich. Using crash Hoare logic for certifying the FSCQ file system. In *ACM Symposium on Operating Systems Principles (SOSP)*, pages 18–37, 2015. doi: 10.1145/2815400.2815402. Cited on page 29.

[26] Stefan Ciobâc, K. Rustan M. Leino, Stefan Merca, and Roxana-Mihaela Timon. The Design of an Interactive Proof Mode for Dafny. In *Proceedings of the Third Workshop on Dafny*, 2026. Cited on page 79.

[27] Koen Claessen and John Hughes. QuickCheck: a lightweight tool for random testing of Haskell programs. In *ICFP*, pages 268–279. ACM, 2000. doi: 10.1145/351240.351266. Cited on page 72.

[28] David Cock, Gerwin Klein, and Thomas Sewell. Secure microkernels, state monads and scalable refinement. In *Theorem Proving in Higher Order Logics (TPHOLs)*, volume 5170 of *LNCS*, pages 167–182. Springer, 2008. doi: 10.1007/978-3-540-71067-7_16. Cited on pages 3 and 11.

[29] Joshua M. Cohen and Philip Johnson-Freyd. A Formalization of Core Why3 in Coq. *Proc. ACM Program. Lang.*, 8(POPL):1789–1818, 2024. doi: 10.1145/3632902. Cited on page 79.

[30] CSO Online. AI agent finds 18-year-old remote code execution flaw in Nginx, May 2026. CVE-2026-42945 ("NGINX Rift"). https://www.csoonline.com/article/4171437/ai-agent-finds-18-year-old-remote-code-execution-flaw-in-nginx.html. Cited on page 2.

[31] Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Rungta, Emina Torlak, and Andrew M. Wells. Cedar: A new language for expressive, fast, safe, and analyzable authorization. *Proc. ACM Program. Lang. (OOPSLA)*, 8, 2024. doi: 10.1145/3649835. Cited on page 3.

[32] Leonardo de Moura. SymM: Scaling verification in Lean 4, 2026. Invited talk at SVIL '26. Slides: https://leodemoura.github.io/static/svil26/. Cited on page 53.

[33] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Conference on Automated Deduction (CADE)*, 2021. doi: 10.1007/978-3-030-79876-5_37. Cited on pages 2 and 47.

[34] Leonardo Mendonça de Moura and Nikolaj Bjørner. Z3: an efficient SMT solver. In *TACAS*, volume 4963 of *LNCS*, pages 337–340. Springer, 2008. doi: 10.1007/978-3-540-78800-3_24. Cited on pages 47 and 71.

[35] Edsger W. Dijkstra. Notes on structured programming. Technical Report EWD249, Technological University Eindhoven, 1970. Cited on pages 2 and 21.

[36] Edsger W. Dijkstra. On a political pamphlet from the Middle Ages. *ACM SIGSOFT Software Engineering Notes*, 3(2):14–16, 1978. doi: 10.1145/1005888.1005890. Cited on page 2.

[37] Yueyang Feng, Dipesh Kafle, Vladimir Gladshtein, Vitaly Kurin, George Pîrlea, Qiyuan Zhao, Peter Müller, and Ilya Sergey. Certified program synthesis with a multi-modal verifier. In *Automated Software Engineering (ASE)*, 2026. Cited on page 5.

[38] Jean-Christophe Filliâtre and Andrei Paskevich. Why3 - Where Programs Meet Provers. In *ESOP*, volume 7792 of *LNCS*, pages 125–128. Springer, 2013. doi: 10.1007/978-3-642-37036-6_8. Cited on pages 43 and 79.

[39] Jean-Christophe Filliâtre, Léon Gondelman, and Andrei Paskevich. The Spirit of Ghost Code. In *CAV*, volume 8559 of *LNCS*, pages 1–16. Springer, 2014. doi: 10.1007/978-3-319-08867-9_1. Cited on pages 42 and 43.

[40] Martin Giese and Wolfgang Ahrendt. Hilbert's epsilon-terms in automated theorem proving. In *TABLEAUX*, volume 1617 of *LNCS*, pages 171–185. Springer, 1999. doi: 10.1007/3-540-48754-9_17. Cited on page 40.

[41] Vladimir Gladshtein and K. Rustan M. Leino. Formalization of a realistic verification-condition generator for an intermediate verification language. In *Interactive Theorem Proving (ITP)*, 2026. To appear. Cited on pages 5 and 29.

[42] Vladimir Gladshtein, George Pîrlea, and Ilya Sergey. Small Scale Reflection for the Working Lean User. arXiv preprint arXiv:2403.12733, 2024. Cited on page 5.

[43] Vladimir Gladshtein, Qiyuan Zhao, Willow Ahrens, Saman P. Amarasinghe, and Ilya Sergey. Mechanised Hypersafety Proofs about Structured Data. *Proc. ACM Program. Lang.*, 8(PLDI):647–670, 2024. doi: 10.1145/3656403. Cited on page 5.

[44] Vladimir Gladshtein, Qiyuan Zhao, Willow Ahrens, Saman P. Amarasinghe, and Ilya Sergey. LGTM: The Logic for Graceful Tensor Manipulation (Software Artifact). https://doi.org/10.5281/zenodo.10951930, 2024. Cited on page 5.

[45] Vladimir Gladshtein, Vitaly Kurin, Yueyang Feng, Dipesh Kafle, George Pîrlea, Qiyuan Zhao, and Ilya Sergey. Velvet: A foundational multi-modal verifier for imperative programs in Lean. In *Computer Aided Verification (CAV)*, 2026. doi: 10.1007/978-3-032-32526-6_11. Cited on page 5.

[46] Vladimir Gladshtein, George Pîrlea, Qiyuan Zhao, Vitaly Kurin, and Ilya Sergey. Foundational multi-modal program verifiers. *Proc. ACM Program. Lang.*, 10(POPL), 2026. doi: 10.1145/3776719. Cited on pages 4 and 86.

[47] Vladimir Gladshtein, Qiyuan Zhao, Yuxi Ling, Sean Wang, and Ilya Sergey. Infinitary relational logic. In *International Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, 2026. To appear. Cited on page 5.

[48] Harrison Goldstein, Joseph W. Cutler, Daniel Dickstein, Benjamin C. Pierce, and Andrew Head. Property-based testing in practice. In *ICSE*, pages 187:1–187:13. ACM, 2024. doi: 10.1145/3597503.3639581. Cited on page 72.

[49] Georges Gonthier, Beta Ziliani, Aleksandar Nanevski, and Derek Dreyer. How to make ad hoc proof automation less ad hoc. In *ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 163–175, 2011. doi: 10.1145/2034574.2034798. Cited on page 12.

[50] David Greenaway, Japheth Lim, June Andronick, and Gerwin Klein. Don't sweat the small stuff: Formal verification of C code without the pain. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 429–439, 2014. doi: 10.1145/2594291.2594296. Cited on pages 3 and 11.

[51] Armaël Guéneau, Arthur Charguéraud, and François Pottier. A Fistful of Dollars: Formalizing Asymptotic Complexity Claims via Deductive Program Verification. In *ESOP*, volume 10801 of *LNCS*, pages 533–560. Springer, 2018. doi: 10.1007/978-3-319-89884-1_19. Cited on page 86.

[52] David Harel, Dexter Kozen, and Jerzy Tiuryn. *Dynamic Logic*. Foundations of Computing. MIT Press, 2000. doi: 10.7551/mitpress/2516.001.0001. Cited on page 10.

[53] Chris Hawblitzel, Jon Howell, Jacob R. Lorch, Arjun Narayan, Bryan Parno, Danfeng Zhang, and Brian Zill. Ironclad Apps: End-to-End Security via Automated Full-System Verification. In *OSDI*, pages 165–181. USENIX Association, 2014. Cited on page 47.

[54] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath T. V. Setty, and Brian Zill. IronFleet: proving practical distributed systems correct. In *SOSP*, pages 1–17. ACM, 2015. doi: 10.1145/2815400.2815428. Cited on pages 47 and 80.

[55] Paul N. Hilfinger. A high-level language and silicon compiler for digital signal processing. In *IEEE Custom Integrated Circuits Conference (CICC)*, 1985. Cited on page 6.

[56] Son Ho and Jonathan Protzenko. Aeneas: Rust verification by functional translation. In *ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 2022. doi: 10.1145/3547647. Cited on pages 3 and 13.

[57] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969. doi: 10.1145/363235.363259. Cited on page 28.

[58] C. A. R. Hoare. How did software get so reliable without proof? In *FME'96: Industrial Benefit and Advances in Formal Methods*, volume 1051 of *LNCS*, pages 1–17. Springer, 1996. doi: 10.1007/3-540-60973-3_77. Cited on page 1.

[59] IEEE. *IEEE Standard VHDL Language Reference Manual*. IEEE, 1988. IEEE Std 1076-1987. Cited on page 6.

[60] JetBrains Research. The Arend proof assistant. https://arend-lang.github.io, 2015. Cited on page 3.

[61] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *J. Funct. Program.*, 28:e20, 2018. doi: 10.1017/S0956796818000151. Cited on pages 79 and 85.

[62] Andrej Karpathy. Post on x, February 2025. https://x.com/karpathy/status/1886192184808149383. Cited on page 1.

[63] Shin-ya Katsumata. Parametric effect monads and semantics of effect systems. In *POPL*, pages 633–646. ACM, 2014. doi: 10.1145/2535838.2535846. URL https://doi.org/10.1145/2535838.2535846. Cited on page 36.

[64] Donnacha Oisín Kidney, Zhixuan Yang, and Nicolas Wu. Algebraic effects meet Hoare logic in cubical Agda. *Proc. ACM Program. Lang.*, 8(POPL), 2024. doi: 10.1145/3632898. Cited on pages 4 and 15.

[65] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal verification of an OS kernel. In *ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, pages 207–220, 2009. doi: 10.1145/1629575.1629596. Cited on pages 10, 11, and 47.

[66] Robbert Krebbers, Amin Timany, and Lars Birkedal. Interactive proofs in higher-order concurrent separation logic. In *POPL*, pages 205–217. ACM, 2017. doi: 10.1145/3009837.3009855. Cited on pages 79 and 85.

[67] Saunders Mac Lane. *Categories for the Working Mathematician*, volume 5 of *Graduate Texts in Mathematics*. Springer, 2 edition, 1998. Cited on page 7.

[68] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying rust programs using linear ghost types. In *Proc. ACM Program. Lang. (OOPSLA)*, 2023. doi: 10.1145/3586037. Cited on pages 70 and 80.

[69] leanprover-community. Plausible: A property testing framework for Lean 4. https://github.com/leanprover-community/plausible, 2025. Accessed on 28 Jan 2026. Cited on pages 31 and 72.

[70] Nico Lehmann, Rose Kunkel, Jordan Brown, Jean Yang, Niki Vazou, Nadia Polikarpova, Deian Stefan, and Ranjit Jhala. STORM: Refinement types for secure web applications. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 441–459, 2021. Cited on page 45.

[71] K. Rustan M. Leino. *Toward Reliable Modular Programs*. PhD thesis, California Institute of Technology, 1995. Cited on pages 28 and 29.

[72] K. Rustan M. Leino. Dafny: An Automatic Program Verifier for Functional Correctness. In *LPAR*, volume 6355 of *LNCS*, pages 348–370. Springer, 2010. doi: 10.1007/978-3-642-17511-4_20. Cited on pages 43, 47, 70, and 79.

[73] K. Rustan M. Leino. Compiling hilbert's epsilon operator. In *LPAR*, volume 35 of *EPiC Series in Computing*, pages 106–118. EasyChair, 2015. doi: 10.29007/rkxm. Cited on pages 41 and 73.

[74] K. Rustan M. Leino and Michał Moskal. Usable Auto-Active Verification. In *Usable Verification Workshop (UV)*, 2010. Cited on page 47.

[75] Thomas Letan, Yann Régis-Gianas, Pierre Chifflier, and Guillaume Hiet. Modular verification of programs with effects and effects handlers. *Formal Aspects Comput.*, 33(1):127–150, 2021. doi: 10.1007/s00165-020-00523-2. Cited on page 40.

[76] Sheng Liang, Paul Hudak, and Mark Jones. Monad transformers and modular interpreters. In *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 333–343, 1995. doi: 10.1145/199448.199528. Cited on pages 20 and 22.

[77] Jannis Limperg and Asta Halkjær From. Aesop: White-Box Best-First Proof Search for Lean. In *CPP*, pages 253–266. ACM, 2023. doi: 10.1145/3573105.3575671. Cited on pages 72 and 78.

[78] Kenji Maillard, Danel Ahman, Robert Atkey, Guido Martínez, Catalin Hritcu, Exequiel Rivas, and Éric Tanter. Dijkstra monads for all. In *Proc. ACM Program. Lang. (ICFP)*, 2019. doi: 10.1145/3341708. Cited on pages 13, 15, 16, 17, and 36.

[79] Kenji Maillard, Cătălin Hriţcu, Exequiel Rivas, and Antoine Van Muylder. The Next 700 Relational Program Logics. *Proc. ACM Program. Lang.*, 4(POPL):4:1–4:33, 2020. doi: 10.1145/3371072. Cited on page 84.

[80] Guido Martínez, Danel Ahman, Victor Dumitrescu, Nick Giannarakis, Chris Hawblitzel, Catalin Hritcu, Monal Narasimhamurthy, Zoe Paraskevopoulou, Clément Pit-Claudel, Jonathan Protzenko, Tahina Ramananandro, Aseem Rastogi, and Nikhil Swamy. Meta-F*: Proof Automation with SMT, Tactics, and Metaprograms. In *ESOP*, volume 11423 of *LNCS*, pages 30–59. Springer, 2019. doi: 10.1007/978-3-030-17184-1_2. Cited on page 79.

[81] The mathlib Community. The Lean mathematical library. In *CPP*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824. https://github.com/leanprover-community/mathlib4. Cited on pages 70 and 73.

[82] Valentin Mikhalchuk, Vladimir Gladshtein, and Ilya Sergey. Lazy Proof Automation for Separation Logic. In *Interactive Theorem Proving (ITP)*, 2026. To appear. Cited on page 5.

[83] Richard A. De Millo, Richard J. Lipton, and Alan J. Perlis. Social processes and proofs of theorems and programs. *Communications of the ACM*, 22(5):271–280, 1979. doi: 10.1145/359104.359106. Cited on page 2.

[84] E. Moggi. Computational lambda-calculus and monads. In *Proceedings of the Fourth Annual Symposium on Logic in Computer Science*, page 1423. IEEE Press, 1989. ISBN 0818619546. doi: 10.1109/lics.1989.39155. Cited on pages 7 and 9.

[85] Eugenio Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991. doi: 10.1016/0890-5401(91)90052-4. Cited on pages 3, 9, 15, and 18.

[86] Abdalrhman Mohamed, Tomaz Mascarenhas, Muhammad Harun Ali Khan, Haniel Barbosa, Andrew Reynolds, Yicheng Qian, Cesare Tinelli, and Clark W. Barrett. lean-smt: An SMT Tactic for Discharging Proof Goals in Lean. In *CAV*, volume 15933 of *LNCS*, pages 197–212. Springer, 2025. doi: 10.1007/978-3-031-98682-6_11. Cited on pages 63 and 71.

[87] J. D. Morison and A. S. Clarke. *ELLA 2000: A Language for Electronic System Design*. McGraw-Hill, 1993. Cited on page 6.

[88] Kim Morrison and Leonardo de Moura. grind: An SMT-Inspired Tactic for Lean 4. In *IJCAR*, LNCS. Springer, 2026. System description. To appear. Cited on pages 48, 63, 70, and 78.

[89] Till Mossakowski, Lutz Schröder, and Sergey Goncharov. A generic complete dynamic logic for reasoning about purity and effects. *Formal Aspects of Computing*, 22 (3):363–384, 2010. doi: 10.1007/s00165-010-0153-4. URL https://doi.org/10.1007/s00165-010-0153-4. Cited on pages 4, 10, and 15.

[90] Peter Müller, Malte Schwerhoff, and Alexander J. Summers. Viper: A verification infrastructure for permission-based reasoning. In *VMCAI*, volume 9583 of *LNCS*, pages 41–62. Springer, 2016. doi: 10.1007/978-3-662-49122-5_2. Cited on pages 47 and 85.

[91] Toby Murray, Daniel Matichuk, Matthew Brassil, Peter Gammie, Timothy Bourke, Sean Seefried, Corey Lewis, Xin Gao, and Gerwin Klein. seL4: from general purpose to a proof of information flow enforcement. In *IEEE Symposium on Security and Privacy (S&P)*, pages 415–429, 2013. doi: 10.1109/sp.2013.35. Cited on page 11.

[92] Satya Nadella. Remarks at LlamaCon, April 2025. Reported by CNBC: 20–30% of Microsoft's code is written by AI. https://www.cnbc.com/2025/04/29/satya-nadella-says-as-much-as-30percent-of-microsoft-code-is-written-by-ai.html. Cited on page 1.

[93] Gal Nagli. Hacking Moltbook: The AI social network any human can control. Wiz Research Blog, February 2026. https://www.wiz.io/blog/exposed-moltbook-database-reveals-millions-of-api-keys. Cited on page 1.

[94] Aleksandar Nanevski, Greg Morrisett, and Lars Birkedal. Polymorphism and separation in Hoare type theory. In *ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 62–73, 2006. doi: 10.1145/1160074.1159812. Cited on pages 12 and 82.

[95] Aleksandar Nanevski, Greg Morrisett, Avraham Shinnar, Paul Govereau, and Lars Birkedal. Ynot: Dependent types for imperative programs. In *ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 229–240, 2008. doi: 10.1145/1411204.1411237. Cited on page 12.

[96] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*, volume 2283 of *LNCS*. Springer, 2002. doi: 10.1007/3-540-45949-9. Cited on page 3.

[97] Peter W. O'Hearn. Separation logic. *Communications of the ACM*, 62(2):86–95, 2019. doi: 10.1145/3211968. Cited on page 11.

[98] Peter W. O'Hearn. Incorrectness logic. *Proc. ACM Program. Lang.*, 4(POPL):10:1–10:32, 2020. doi: 10.1145/3371078. Cited on page 73.

[99] Peter W. O'Hearn. Incorrectness logic. *Proc. ACM Program. Lang.*, 4(POPL), 2020. doi: 10.1145/3371078. Cited on page 25.

[100] Susan S. Owicki and David Gries. An Axiomatic Proof Technique for Parallel Programs I. *Acta Informatica*, 6(4):319–340, 1976. doi: 10.1007/BF00268134. Cited on pages 42 and 43.

[101] Oded Padon, Kenneth L. McMillan, Aurojit Panda, Mooly Sagiv, and Sharon Shoham. Ivy: Safety Verification by Interactive Generalization. In *PLDI*, pages 614–630. ACM, 2016. doi: 10.1145/2908080.2908118. Cited on page 86.

[102] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. The impact of AI on developer productivity: Evidence from GitHub Copilot. *arXiv preprint arXiv:2302.06590*, 2023. doi: 10.48550/arXiv.2302.06590. Cited on page 1.

[103] Tomas Petricek and Don Syme. The F# computation expression zoo. In *Practical Aspects of Declarative Languages (PADL)*, volume 8324 of *LNCS*, pages 33–48. Springer, 2014. doi: 10.1007/978-3-319-04132-2_3. Cited on page 9.

[104] Simon L. Peyton Jones and Philip Wadler. Imperative functional programming. In *Proceedings of the 20th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '93, page 7184, New York, NY, USA, 1993. Association for Computing Machinery. ISBN 0897915607. doi: 10.1145/158511.158524. URL https://doi.org/10.1145/158511.158524. Cited on page 9.

[105] Sundar Pichai. Remarks on the Alphabet Q3 2024 earnings call, October 2024. "More than a quarter of all new code at Google is generated by AI.". Cited on page 1.

[106] George Pîrlea, Vladimir Gladshtein, Elad Kinsbruner, Qiyuan Zhao, and Ilya Sergey. Veil: A Framework for Automated and Interactive Verification of Transition Systems. In *CAV*, volume 15933 of *LNCS*, pages 26–41. Springer, 2025. doi: 10.1007/978-3-031-98682-6_2. Cited on page 80.

[107] George Pîrlea, Vladimir Gladshtein, Elad Kinsbruner, Qiyuan Zhao, and Ilya Sergey. Veil: A Framework for Automated and Interactive Verification of Transition Systems (Software Artifact). https://doi.org/10.5281/zenodo.15208271, 2025. Cited on page 5.

[108] George Pîrlea, Vladimir Gladshtein, Elad Kinsbruner, Qiyuan Zhao, and Ilya Sergey. Veil: A framework for automated and interactive verification of transition systems. In *Computer Aided Verification (CAV)*, LNCS. Springer, 2025. doi: 10.1007/978-3-031-98682-6_2. Cited on pages 3, 5, 14, and 86.

[109] George Pîrlea, Vladimir Gladshtein, Qiyuan Zhao, and Ilya Sergey. Lessons from building an auto-active verifier in lean. In *The Dafny Workshop (Dafny 2026), co-located with POPL*, 2026. Cited on pages 2, 14, and 47.

[110] Jonathan Protzenko, Jean-Karim Zinzindohoué, Aseem Rastogi, Tahina Ramananandro, Peng Wang, Santiago Zanella-Béguelin, Antoine Delignat-Lavaud, Cătălin Hriţcu, Karthikeyan Bhargavan, Cédric Fournet, and Nikhil Swamy. Verified low-level programming embedded in F*. *Proc. ACM Program. Lang.*, 1(ICFP), 2017. doi: 10.1145/3110261. Cited on page 3.

[111] Jonathan Protzenko, Bryan Parno, Aymeric Fromherz, Chris Hawblitzel, Marina Polubelova, Karthikeyan Bhargavan, Benjamin Beurdouche, Joonwon Choi, Antoine Delignat-Lavaud, Cédric Fournet, Natalia Kulatova, Tahina Ramananandro, Aseem Rastogi, Nikhil Swamy, Christoph M. Wintersteiger, and Santiago Zanella-Béguelin. EverCrypt: A fast, verified, cross-platform cryptographic provider. In *IEEE Symposium on Security and Privacy (S&P)*, pages 983–1002, 2020. doi: 10.1109/sp40000.2020.00114. Cited on pages 4 and 13.

[112] Yicheng Qian, Joshua Clune, Clark W. Barrett, and Jeremy Avigad. Lean-Auto: An Interface Between Lean 4 and Automated Theorem Provers. In *CAV*, volume 15933 of *LNCS*, pages 175–196. Springer, 2025. doi: 10.1007/978-3-031-98682-6_10. Cited on page 72.

[113] Lutz Schröder and Till Mossakowski. Monad-independent hoare logic in hascasl. In *Proceedings of the 6th International Conference on Fundamental Approaches to Software*

Engineering, FASE'03, page 261277, Berlin, Heidelberg, 2003. Springer-Verlag. ISBN 3540008993. doi: 10.1007/3-540-36578-8_19. Cited on pages 9, 10, and 15.

[114] Lutz Schröder and Till Mossakowski. Monad-independent dynamic logic in hascasl. In Martin Wirsing, Dirk Pattinson, and Rolf Hennicker, editors, *Recent Trends in Algebraic Development Techniques*, pages 425–441, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg. ISBN 978-3-540-40020-2. doi: 10.1093/logcom/14.4.571. Cited on pages 10 and 15.

[115] Thomas Arthur Leck Sewell, Magnus O. Myreen, and Gerwin Klein. Translation validation for a verified OS kernel. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 471–482, 2013. doi: 10.1145/2499370.2462183. Cited on page 11.

[116] Jan Smans, Bart Jacobs, and Frank Piessens. Implicit Dynamic Frames: Combining Dynamic Frames and Separation Logic. In *ECOOP*, volume 5653 of *LNCS*, pages 148–172. Springer, 2009. doi: 10.1007/978-3-642-03013-0_8. Cited on page 85.

[117] Simon Spies, Niklas Mück, Haoyi Zeng, Michael Sammler, Andrea Lattuada, Peter Müller, and Derek Dreyer. Destabilizing Iris. *Proc. ACM Program. Lang.*, 9(PLDI), 2025. doi: 10.1145/3729284. URL https://doi.org/10.1145/3729284. Cited on page 79.

[118] Stack Overflow. 2025 developer survey: AI, 2025. https://survey.stackoverflow.co/2025/ai. Cited on page 2.

[119] Nikhil Swamy, Joel Weinberger, Cole Schlesinger, Juan Chen, and Benjamin Livshits. Verifying higher-order programs with the Dijkstra monad. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 387–398, 2013. doi: 10.1145/2499370.2491978. Cited on pages 12 and 15.

[120] Nikhil Swamy, Catalin Hritcu, Chantal Keller, Aseem Rastogi, Antoine Delignat-Lavaud, Simon Forest, Karthikeyan Bhargavan, Cédric Fournet, Pierre-Yves Strub, Markulf Kohlweiss, Jean Karim Zinzindohoue, and Santiago Zanella-Béguelin. Dependent types and multi-monadic effects in F*. In *POPL*, pages 256–270. ACM, 2016. doi: 10.1145/2837614.2837655. Cited on page 79.

[121] Nikhil Swamy, Cătălin Hriţcu, Chantal Keller, Aseem Rastogi, Antoine Delignat-Lavaud, Simon Forest, Karthikeyan Bhargavan, Cédric Fournet, Pierre-Yves Strub, Markulf Kohlweiss, Jean-Karim Zinzindohoué, and Santiago Zanella-Béguelin. Dependent types and multi-monadic effects in F*. In *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 256–270, 2016. doi: 10.1145/2914770.2837655. Cited on pages 4, 12, 13, and 47.

[122] Technical Correspondence. Comments on "social processes and proofs of theorems and programs". *Communications of the ACM*, 22(11), 1979. Letters in response to De Millo, Lipton, and Perlis; exact page range to be confirmed. Cited on page 2.

[123] The Guardian. Rogue AI coding agent deletes firm's database, April 2026. https://www.theguardian.com/technology/2026/apr/29/claude-ai-deletes-firm-database. Cited on page 1.

[124] The OCaml Manual. Binding operators. https://ocaml.org/manual/bindingops.html, 2019. Introduced in OCaml 4.08. Cited on page 9.

[125] The Register. Vibe coding service Replit deleted production database, July 2025. https://www.theregister.com/2025/07/21/replit_saastr_vibe_coding_incident/. Cited on page 1.

[126] The Rocq Development Team. *The Rocq Prover (formerly Coq)*, 2025. https://rocq-prover.org. Cited on page 3.

[127] The Scala Documentation. For comprehensions. https://docs.scala-lang.org/tour/for-comprehensions.html, 2024. Cited on page 9.

[128] Sebastian Ullrich. *An Extensible Theorem Proving Frontend*. PhD thesis, Karlsruhe Institute of Technology, 2023. Cited on page 2.

[129] Sebastian Ullrich and Leonardo de Moura. Counting immutable beans: Reference counting optimized for purely functional programming. In *Implementation and Application of Functional Languages (IFL)*, 2019. doi: 10.48550/arXiv.1908.05647. arXiv:1908.05647. Cited on page 2.

[130] Sebastian Ullrich and Leonardo de Moura. 'do' unchained: Embracing local imperativity in a purely functional language. *Proc. ACM Program. Lang.*, 6(ICFP), 2022. doi: 10.1145/3547640. Cited on pages 3 and 9.

[131] Max Vistrup, Michael Sammler, and Ralf Jung. Program logics à la carte. *Proc. ACM Program. Lang.*, 9(POPL), 2025. doi: 10.1145/3704847. Cited on pages 4 and 85.

[132] Philip Wadler. Comprehending monads. In *Proceedings of the 1990 ACM Conference on LISP and Functional Programming*, LFP '90, page 6178, New York, NY, USA, 1990. Association for Computing Machinery. ISBN 089791368X. doi: 10.1145/91556.91592. URL https://doi.org/10.1145/91556.91592. Cited on pages 7 and 9.

[133] Philip Wadler. The essence of functional programming. In *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 1–14, 1992. doi: 10.1145/143165.143169. Cited on pages 3, 9, and 28.

[134] Kent Walker. A summer of security: Empowering cyber defenders with AI. Google Keyword Blog, July 2025. https://blog.google/innovation-and-ai/technology/safety-security/cybersecurity-updates-summer-2025/. Cited on page 1.

[135] James R. Wilcox, Doug Woos, Pavel Panchekha, Zachary Tatlock, Xi Wang, Michael D. Ernst, and Thomas E. Anderson. Verdi: a framework for implementing and formally verifying distributed systems. In *PLDI*, pages 357–368. ACM, 2015. doi: 10.1145/2737924.2737958. Cited on page 80.

[136] Li yao Xia, Yannick Zakowski, Paul He, Chung-Kil Hur, Gregory Malecha, Benjamin C. Pierce, and Steve Zdancewic. Interaction trees: Representing recursive and impure programs in Coq. *Proc. ACM Program. Lang.*, 4(POPL), 2020. doi: 10.1145/3371119. Cited on pages 22, 39, 40, 41, and 84.

[137] Zhe Ye, Zhengxu Yan, Jingxuan He, Timothe Kasriel, Kaiyu Yang, and Dawn Song. VERINA: Benchmarking Verifiable Code Generation, 2025. URL https://arxiv.org/abs/2505.23135. Cited on page 76.

[138] Noam Zilberstein, Derek Dreyer, and Alexandra Silva. Outcome Logic: A Unifying Foundation for Correctness and Incorrectness Reasoning. *Proc. ACM Program. Lang.*, 7(OOPSLA1):522–550, 2023. doi: 10.1145/3586045. Cited on page 73.

[139] Jean-Karim Zinzindohoué, Karthikeyan Bhargavan, Jonathan Protzenko, and Benjamin Beurdouche. HACL*: A verified modern cryptographic library. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1789–1806, 2017. doi: 10.1145/3133956.3134043. Cited on pages 4 and 13.